

Permissionless Clock Synchronization with Public Setup

Juan Garay¹, Aggelos Kiayias² and Yu Shen³ 

¹Texas A&M University, garay@cse.tamu.edu

²University of Edinburgh and IOG, aggelos.kiayias@ed.ac.uk

³University of Edinburgh, shenyu.tcv@gmail.com

Abstract

The permissionless clock synchronization problem asks how it is possible for a population of parties to maintain a system-wide synchronized clock, while their participation rate fluctuates — possibly very widely — over time. The underlying assumption is that parties experience the passage of time with roughly the same speed, but however they may disengage and engage with the protocol following arbitrary (and even chosen adversarially) participation patterns. This (classical) problem has received renewed attention due to the advent of blockchain protocols, and recently it has been solved in the setting of proof of stake, i.e., when parties are assumed to have access to a trusted PKI setup [Badertscher *et al.*, Eurocrypt '21].

In this work, we present the first proof-of-work (PoW)-based permissionless clock synchronization protocol. Our construction assumes a public setup (e.g., a CRS) and relies on an honest majority of computational power that, for the first time, is described in a fine-grain timing model that does not utilize a global clock that exports the current time to all parties. As a secondary result of independent interest, our protocol gives rise to the first PoW-based ledger consensus protocol that does not rely on an external clock for the time-stamping of transactions and adjustment of the PoW difficulty.

Contents

1	Introduction	3
1.1	Overview of Our Results	4
2	Model and Building Blocks	6
2.1	Imperfect Local Clocks	6
2.2	Other Core Functionalities	7
2.3	Dynamic Participation	8
3	The Clock Synchronization Protocol with Public Setup	9
3.1	Timekeeper Timestamps	10
3.2	2-for-1 Proofs of Work and Synchronization Beacons	11
3.3	Clock Synchronization Intervals and the Synchronization Procedure	12
3.3.1	Interval Structure	12
3.3.2	The Synchronization Procedure	15
3.4	The Target Recalculation Function	16
3.5	Newly Joining Parties	17
4	Protocol Analysis	18
4.1	Notation, Definitions and Preliminary Propositions	18
4.2	Protocol Parameters and Their Conditions	22
4.3	Typical Executions	23
4.4	Proof Roadmap	24
4.5	Typical Executions Maintain a Steady Block Generation Rate	24
4.6	Typical Executions Maintain Good Skews	28
A	Mathematical Facts	36
B	Model and Building Blocks (Cont'd)	37
C	The Timekeeper Protocol	42
D	Proofs Omitted from the Main Body	54
E	Glossary	56

1 Introduction

In the classical clock synchronization problem, thoroughly studied over the past four decades by the distributed computing community—non-exhaustively, [Lam78, LMS84, HSSD84, DHS86, ST87, WL88, ADD⁺19, LL22]—, a set of processors, each one possessing a timer that is within a bounded rate of drift from “nominal time” (the real time—called Newtonian time in [DHS86]), should realize logical clocks that are within a distance $\text{Skew} \in \mathbb{N}$ of each other and within a linear envelope of nominal time. The typical threat model involves a subset of parties who deviate arbitrarily either from correct protocol execution or in terms of their clock speed and may as a result prevent synchronization from happening. A clock synchronization protocol has parties exchanging messages to suitably adjust their clocks so that the synchronization condition is achieved.

Up until the work of [BGK⁺21] all prior work in clock synchronization assumed that the number of parties are known during the protocol execution (and available, unless they are assumed adversarial¹). This standard assumption in Byzantine fault tolerance protocols was challenged first with the advent of the Bitcoin blockchain and related “permissionless” protocols. As exemplified in [GKL17, GKL20], the Bitcoin blockchain operates in a setting where the number of active parties may be unknown and continuously fluctuating throughout the protocol execution. While such results paved the way to rethink the problem of consensus in this setting (cf. [PS17b, GK20]), near perfectly synchronized clocks remained a central assumption in all previous security analyses of blockchain protocols (cf. [GKL15, BMTZ17, PSs17, GKL17, GKL20]).

In the setting where participation is dynamic and fluctuating over time, the adversary can introduce and remove honest parties at will without notifying the existing participants. As a result, existing clock synchronization algorithms (e.g., [ST87, ADD⁺19, LL22]) do not directly translate to such permissionless setting because they fundamentally rely on the fact that the parties are aware of the number of parties as well as of the number of tolerated corruptions/faults—i.e., they are able to *count*—and a different protocol design technique is needed.

The main challenge in this transient participation setting shifts from correcting the bounded-rate drift occurring between the ever connected honest parties over time to the task of bringing up to sync freshly joining parties who start without any information about nominal time, while accommodating for the fact that a (possibly large) number of honest parties is no longer active. In [BGK⁺21], assuming a so-called private-state setup [GK20] (specifically, a PKI), a protocol called “Ouroboros Chronos” is presented that can synthesize a notion of global time using a continuous flow of clock measurements that are provided by parties who only transiently participate in the protocol and their local clocks are assumed to be correct up to a bound. The level of participation fluctuates broadly with the only requirements that (i) it does not become negligible, and (ii) honest majority is preserved in terms of stake (all parties have a number of coins associated to their public keys that amount to their individual stake). Given this, their result leaves open the question of only utilizing a public(-state) setup.

To our knowledge, the only known result with a public setup in the permissionless setting, again from [BGK⁺21], is that parties may use a Nakamoto-style longest chain blockchain without difficulty adjustment and use the block index to define a concept of global time. The obvious downside of this idea is that the protocol execution speeds up and slows down as participation fluctuates and, most importantly, it will be entirely insecure when there is a steady increase (or decrease) of participants, making the construction essentially only suitable for a static model where the number of parties (i.e., the computational power invested in the system for proof of work) remains fixed.

¹We note that the problem of joining parties in the context of clock synchronization was considered, but only conditionally on the new party agreed upon and approved by a sufficient number of participants; see [HSSD84].

This motivates the current work, where the following open question is being tackled:

Is it possible for a dynamically changing population of peers to synchronize their clocks utilizing only a public setup and assuming PoW?

One apparent difficulty in answering this question is that using a blockchain protocol to derive consistency for clock adjustments runs into the complication that the blockchain protocol itself utilizes a clock to adjust the PoW difficulty at regular intervals. Indeed, the Bitcoin blockchain [Nak08] relies fundamentally on a global clock being available to all parties.² It follows that this observation suggests also a secondary open question that will be tackled as well:

Is there a blockchain protocol in the PoW setting that has no dependency on a publicly accessible global clock?

1.1 Overview of Our Results

The clock synchronization problem asks parties to report clocks that satisfy two properties³ (i) *Precision*: the parties maintain logical clocks whose difference is upper bounded, and (ii) *Accuracy*: the logical clock reported by a party is always within a *linear envelope* of the nominal time. Note that we are interested in a formulation of this problem in a very general setting where some parties are adversarial and hence deviate from the protocol arbitrarily, while honest parties may come and go following arbitrary participation patterns. Given this setting we formulate the *desideratum* of a synchronized clock only with respect to a class of parties we call *alert*, which are honest parties that have also been online for a sufficiently long time to catch up with all protocol messages. More formally, the clock synchronization problem is stated as follows.

Definition 1 (Clock Synchronization). *There exist constants $\text{Skew} \in \mathbb{N}$, $\text{shiftLB}, \text{shiftUB} \in (0, 1)$ such that honest parties' logical clocks satisfy the following two properties:*

- **Precision.** *Let $\mathbf{r}_1, \mathbf{r}_2$ be the reported logical clocks of two alert parties at any nominal time r . Then $|\mathbf{r}_1 - \mathbf{r}_2| \leq \text{Skew}$.*
- **Accuracy.** *Each alert party's logical clock stays in a (U, L) -linear envelope⁴ with respect to the nominal time r , where $U = 1/(1 - \text{shiftUB})$ and $L = 1/(1 + \text{shiftLB})$.*

Solving the clock synchronization problem asks for a protocol that within a certain threat model achieves the two properties. This brings us to our first main result.

A model for permissionless clock synchronization in the PoW setting. Our model (Section 2) simultaneously facilitates (i) the dynamic participation of parties, (ii) imperfect local clocks, and (iii) resource bounding by restricting parties' queries to a random oracle (cf. [GKO⁺20]). Specifically, we extend the previous model of the global imperfect clock of [BGK⁺21] to the PoW setting by introducing a random oracle functionality that apportions random oracle queries per unit of time between the honest parties and the adversary in a manner consistent with an honest majority assumption in terms of computational power. The concept of time provided by the imperfect local clock functionality of [BGK⁺21] enables parties to advance their local clocks and experience time at roughly the same speed (a maximum drift of Φ_{clock} is allowed). Note that the environment is

²The protocol implements such clock by having nodes querying other nodes in the network and possibly seeking user input — it has no way of deriving a clock from the protocol operation itself. See [GKL20] for more details.

³In the conference version of this paper, we used term “bounded skews” and “linear envelope synchronization” following [DHS86].

⁴A function $f : \mathbb{R} \rightarrow \mathbb{R}$ is within a (U, L) -linear envelope if and only if it holds that $L \cdot x - c \leq f(x) \leq U \cdot x + c$, where c is a constant and $x \in \mathbb{R}^+$.

allowed to introduce new parties and remove old parties at will, something that results in them being de-registered from the clock functionality; when this happens the clock functionality is not responsible for keeping them up to speed with the rest of the honest parties. In this way, parties can be seen as entirely transiently engaging with the protocol—each individual party may only engage for a small fraction of the total execution time as adaptively decided by the environment. Armed with our model, we then present our second main result.

A new protocol for permissionless clock synchronization in the PoW model. We describe our new PoW-based clock synchronization protocol *Timekeeper* in Section 3. The construction is based on three key ingredients: (i) A mechanism that repurposes the concept of 2-for-1 PoWs, introduced in [GKL15] and subsequently utilized to achieve various properties such as fairness in [PS17a] and high throughput in [BKT⁺19], to the setting of time-keeping by employing it to enable the collection of “timing beacons” from the active parties in a rolling window process; (ii) a PoW-based longest-chain type of blockchain that enables parties at regular intervals to reach consensus about the timing beacons that are shared and extract a suitable correction to their local clocks taking into account the arrivals of the beacons; and (iii) a novel target-recalculation function that can be thought of as the *reverse* of the one used in Bitcoin, that uses protocol recorded timestamps as a means to define the length of an epoch, and then uses the number of blocks produced in that period of time to adjust the PoW difficulty accordingly.

Putting these elements together, our clock synchronization protocol instructs parties when their local clock passes some specific moment (which happens periodically with respect to the interval length) to execute an adjustment on their local clock based on the median value of the beacon timestamps and their corresponding arrival time. Moreover, towards the goal of letting newly joining parties become synchronized with the protocol time, we present a joining procedure which requires the fresh parties to passively listen to the protocol execution for a while and then synchronize with other honest participants.

Based on the ledger consensus function offered by our protocol it is easy to derive also the following result.

A new PoW-based blockchain protocol without a global clock. Given that our protocol is a Nakamoto-style “most-difficult chain” type of protocol that facilitates clock synchronization, it is easy to transform it to a full-fledged blockchain protocol that admits transactions as in Bitcoin script or Ethereum smart contracts. The resulting blockchain has the novel property that it does not depend on accessing a globally available clock. Instead, parties utilize their local clocks which may be drifting or be out of sync, but thanks to the synchronization (sub-)protocol that is offered by our construction they can adjust their local clocks periodically. This eliminates time as an attack vector in the context of PoW-based blockchain protocols and demonstrates that it is possible to achieve ledger consensus using merely local clocks in a fully dynamic setting where parties may come and go adaptively per the adversary’s instructions.

Security analysis. We present the full security analysis of *Timekeeper* in Section 4. As a high-level overview, we proceed to adapt the analytical toolset from [GKL17, GKL20] to the imperfect-local-clock model. Notably, we modify the concept of target recalculation epoch boundaries (from “point” to “zone”) and the concept of isolated successes (which addresses the question of under what circumstances can a hash success guarantee the increase of accumulated difficulty). As an intermediate step, we study several predicates aiming at providing the “good” properties of an execution starting from the onset and until a given nominal round.

Our inductive-style proof works in the following manner. We prove that if at the onset, the PoW difficulty is appropriately set and the steady block-generation rate lasts during the whole clock synchronization interval, parties can maintain good skews after they enter the next interval and the

shift value they compute to adjust their clocks is properly bounded. In addition, if good skews and certain time adjustment calculations are maintained during a target recalculation epoch, the block production rate will be properly controlled in the next epoch. To sum up, this guarantees that “good” properties can be achieved during the whole execution given a “safe” start and a bounded change in the number of parties (which can nevertheless still be exponential). We also provide an analysis of the joining procedure showing that joining parties starting with no *a-priori* knowledge of global time, can listen in and bootstrap their logical clock, turning themselves into *alert* parties being capable of fully engaging with the protocol.

In summary, Timekeeper solves the clock synchronization problem as defined above as follows.

Theorem (Informal, restated from Theorem 18). *Let Φ_{clock} be the maximum drift allowed on parties’ local clocks and Δ the maximum (and unknown) message transmission delay. Then Timekeeper solves the clock synchronization problem assuming bounded dynamic participation and an honest majority in terms of random oracle queries, with parameter values*

$$\text{Skew} = 2\Phi, \quad \text{shiftLB} = 3\Phi/R, \quad \text{and} \quad \text{shiftUB} = 2\Phi/R,$$

where $\Phi = \Delta + \Phi_{\text{clock}}$ and $R \in \mathbb{N}^+$ is a parameter chosen sufficiently large w.r.t. the security parameter and reflects the time required for an honest party to become alert.

Organization of the paper. The rest of the paper is organized as follows. In Section 2 we present our model, relevant definitions and building blocks. We describe our Timekeeper protocol in Section 3 and present the full analysis in Section 4. Detailed description of protocols, functionalities, and some of the proofs can be found in the appendix.

2 Model and Building Blocks

In this paper we adapt the timing and networking model of [BGK⁺21] to the setting of proof of work, obviating the requirement for a PKI as a setup assumption. In more detail, in the model there is an upper bound Δ in message transmission (cf. [DLS88, PSs17, BMTZ17, GKL20]), and parties do not have access to a global clock, but instead rely on their local clocks, whose drift is assumed to be upper-bounded by Φ_{clock} . What complicates matters is that the model supports dynamic participation where parties may join and leave during the protocol execution without warning (it is worth noting here that this is where the difficulty of our setting is derived from: indeed if all honest parties were online throughout then it would be trivial to implement a logical clock by incrementing a counter). For succinctness, we choose to express primitives and building blocks (see below) in our execution model utilizing the ideal functionality language of [Can01], but we do not pursue a composability analysis for our security properties, which are expressed in a game based manner as in [GKL15, PSs17].

2.1 Imperfect Local Clocks

As in [BGK⁺21], and as mentioned above, in this paper we remove the assumption that parties have access to a global clock, as in [GKL15, GKL17, PSs17, BMTZ17, GKL20], and instead assume *imperfect local clocks*. In a nutshell, every honest party maintains a local clock variable by communicating with an imperfect local clock functionality $\mathcal{G}_{\text{ImpClock}}$. In contrast to the global-setup clock functionality in [KMTZ13], where parties learn the exact global time and thus strong synchrony is guaranteed, parties registered with $\mathcal{G}_{\text{ImpClock}}$ will only receive “ticks” from the functionality to indicate that they should update their own clocks. In addition, $\mathcal{G}_{\text{ImpClock}}$ issues “imperfect” ticks,

i.e., the adversary is allowed to set a bounded additive drift to each party by manipulating its corresponding status variable in $\mathcal{G}_{\text{ImpClock}}$. $\mathcal{G}_{\text{ImpClock}}$ can be viewed as a variant from [BGK⁺21]’s with adaptations to provide a more natural clock model with real-world resources and in the proof-of-work setting.

For a detailed description of the functionality, see Appendix B. Here we just elaborate on the “imperfect” aspect of the clock and on the adversarial manipulation of clock drifts. Specifically, we allow the adversary to set some drifts to parties’ local clocks, which will accelerate or stall their progress; such values are globally bounded by Φ_{clock} . This assumption allows local clocks to proceed at “roughly” the same speed.

Further, the adversary $\mathcal{A}$ can adaptively manipulate the drift of honest parties’ clocks by sending CLOCK-FORWARD and CLOCK-BACKWARD messages to the functionality⁵ after they conclude the current round. If $\mathcal{A}$ issues CLOCK-FORWARD for party P , it will enter a new local round before $\mathcal{G}_{\text{ImpClock}}$ updates the nominal time, and this can be repeated as long as P ’s drift is not Φ_{clock} rounds larger than other honest parties. On the other hand, if $\mathcal{A}$ issues CLOCK-BACKWARD, it will set P ’s budget to a negative value, thus preventing $\mathcal{G}_{\text{ImpClock}}$ from updating d_P at the end of the nominal round (d_P is the functionality variable that captures whether the party P has made its move for the clock tick). I.e., P will still be in the same logical round during these two nominal rounds. Again, this process can be repeated by $\mathcal{A}$ as long as the drift on P is not Φ_{clock} rounds smaller than others. As a consequence, the targeted party’s local clock may remain static for several nominal rounds.

2.2 Other Core Functionalities

Common Reference String. We model a public-state setup by the CRS functionality $\mathcal{F}_{\text{CRS}}$. The functionality is parameterized with some distribution $\mathcal{D}$ with sufficiently high entropy. Once $\mathcal{F}_{\text{CRS}}$ receives (RETRIEVE, sid) from either the adversary $\mathcal{A}$ or a party P for the first time, it generates a string $d \leftarrow \mathcal{D}$ as the common reference string. In addition, $\mathcal{F}_{\text{CRS}}$ will immediately send a message (RETRIEVED, sid) to functionality $\mathcal{W}(\mathcal{F}_{\text{RO}})$ (described next) to indicate that $\mathcal{W}(\mathcal{F}_{\text{RO}})$ should start to limit the adversarial RO queries. For all subsequent activations, $\mathcal{F}_{\text{CRS}}$ simply returns d to the requester.

(Wrapped) Random Oracle. By convention, we model parties’ calls to the hash function used to generate proofs of work as assuming access to a random oracle; this is captured by the functionality $\mathcal{F}_{\text{RO}}$. Notice that with regards to bounding access to real-world resources, functionality $\mathcal{F}_{\text{RO}}$ as defined fails to limit the adversary on making a certain number of queries per round. Hence, we adopt a functionality wrapper [BMTZ17, GKO⁺20] $\mathcal{W}(\mathcal{F}_{\text{RO}})$ that wraps the corresponding resource to capture such restrictions. We highlight that our wrapper $\mathcal{W}(\mathcal{F}_{\text{RO}})$ improves on previous wrappers in two aspects, in order to provide a more natural model of the real world: (i) We capture the pre-mining stage by letting the adversary query the RO with no restrictions (albeit polynomially bounded) before the CRS is released; and (ii) The wrapper limits adversarial access per nominal round by bounding the total number of queries that $\mathcal{A}$ can make. The second aspect allows us to dispose the “flat” computational model and define the computational power in terms of the number of RO queries per round, which makes it possible to further refine the notion of a “respecting environment” (see below) that is suited for imperfect local clocks.

⁵As such, our clock functionality is a more natural model of the real world compared to [BGK⁺21]’s, as it allows $\mathcal{A}$ to manipulate the clock in both directions, backward, and forward; in [BGK⁺21], only forward manipulation is allowed. Nonetheless, this does not result in a more powerful adversary.

Diffusion network. We adopt the peer-to-peer communication functionality $\mathcal{F}_{\text{Diffuse}}$ (cf. [BGK⁺21]), which guarantees that an honestly sent message will be delivered to all the protocol participants within Δ rounds. Moreover, for those adversarially generated messages, $\mathcal{F}_{\text{Diffuse}}^\Delta$ forces them to be delivered to all the honest parties within Δ rounds after they are learnt by at least one honest participant. This captures the natural behavior of honest parties that they will forward all the messages that they have not yet seen to their peers.

We refer to Appendix B for a detailed description of the above functionalities.

2.3 Dynamic Participation

In [BGK⁺18, BGK⁺21], the notion of *dynamic participation* was introduced aiming at describing the protocol execution in a more realistic fashion. Here we present a further refined classification of possible *types* of honest parties. See Table 1.

Resource	Basic types of <i>honest</i> parties	
	Resource unavailable	Resource available
random oracle $\mathcal{F}_{\text{RO}}$	<i>stalled</i>	<i>operational</i>
network $\mathcal{F}_{\text{Diffuse}}$	<i>offline</i>	<i>online</i>
clock $\mathcal{G}_{\text{ImpClock}}$	<i>time-unaware</i>	<i>time-aware</i>
synchronized state	<i>desynchronized</i>	<i>synchronized</i>

Table 1: A classification of protocol participants.

Consider an honest party P at a given point of the protocol execution. We say P is *operational* if P is registered with the random oracle $\mathcal{F}_{\text{RO}}$; otherwise, we say it is *stalled*. We say P is *online* if P is registered with the network; *offline* otherwise. We say P is *time-aware* if P is registered with the *imperfect* clock functionality $\mathcal{G}_{\text{ImpClock}}$, and *time-unaware* otherwise.

Further, we say P is *synchronized* if P has been participating in the protocol for sufficiently long time and achieves a “synchronized state” as well as a “synchronized clock.” “Synchronized state” means P holds a chain that shares a common prefix (cf. [GKL15]) with other *synchronized* parties; “synchronized clock” refers to that P maintains a local clock with time close to other *synchronized* parties. Otherwise, P is *desynchronized*. Additionally, P is aware of whether it is synchronized or not, and maintains a local variable `isSync` serving as an indicator for other actions.

Based on the above classification, we now define the notion of *alert* parties:

$$\text{alert} \stackrel{\text{def}}{=} \text{operational} \wedge \text{online} \wedge \text{time-aware} \wedge \text{synchronized}.$$

In short, alert parties are those who have access to all the resources and are synchronized; this requires them to join the protocol execution passively for some period of time. They constitute the core set of parties that carry out the protocol.

In addition, we define *active* parties to include all parties that are alert, adversarial, and time-unaware.

$$\text{active} \stackrel{\text{def}}{=} \text{alert} \vee \text{adversarial} \vee \text{time-unaware}.$$

Respecting environment in terms of computational power. The notion of a “respecting environment” was introduced in [GKL17] to model the varying number of participants in a protocol execution.

We provide the following generalization of “respecting environment” to relate it to computational power as opposed to number of parties. Our assumption is that during the whole protocol execution, the honest computational power is higher than the adversarial one (cf. the “honest majority” condition in [GKL15] and follow-ups). The computational power is captured by counting the number of RO (hash) queries that parties make in each round. Further, we restrict the environment to fluctuate the number of such queries in a certain limited fashion.

Definition 2. For $\gamma \in \mathbb{R}^+$ we call the sequence $(h_r)_{r \in [0, B)}$, where $B \in \mathbb{N}$, (γ, s) -respecting if for any set $S \subseteq [0, B)$ of at most s consecutive integers, $\max_{r \in S} h_r \leq \gamma \cdot \min_{r \in S} h_r$.

We say that *environment $\mathcal{Z}$ is (γ, s) -respecting* if for all $\mathcal{A}$ and coins for $\mathcal{Z}$ and $\mathcal{A}$ the sequence of honest hash queries (h_r) is (γ, s) -respecting.

Note that the notion of respecting environment here is different from the “flat” model adopted in [GKL15, GKL17, GKL20, BMTZ17]. In a flat model, honest parties are assumed to have the same computational power, hence the total number of RO queries is a direct 1-to-1 map from the number of parties. The new respecting environment allows some subset of the honest parties to query the RO multiple times or stay stalled during a nominal round and hence it adapts to the “imperfect local clock” model used in this paper.

3 The Clock Synchronization Protocol with Public Setup

In this section we present the general approach and the various core building blocks of the new clock synchronization protocol—Timekeeper. For a complete description, refer to Appendix C. At a high level, Timekeeper is a Nakamoto-style PoW-based blockchain protocol together with time synchronization functionalities. Readers can think of it as a Bitcoin protocol with the following modifications (See Figure 1 for an illustration of the protocol execution).

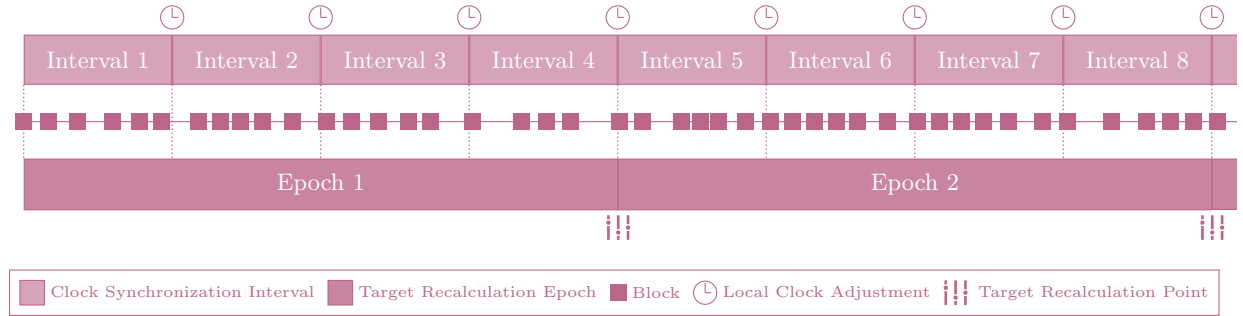


Figure 1: An illustration of the clock synchronization protocol execution with one target recalculation epoch consisting of four clock synchronization intervals.

- It replaces Bitcoin’s original clock maintenance solution⁶ with a new clock synchronization scheme, which requires parties to use 2-for-1 PoWs [GKL15] to mine and emit clock synchronization beacons and include them in an upcoming block. Furthermore, protocol participants will periodically adjust their local clock values based on the beacons collected in the blockchain and their (local) receiving time.

⁶In Bitcoin’s original implementation, miners will adjust their time based on three different sources: (i) their local system clock; (ii) the median of clock values from peers; (iii) the human operator (if the first two disagree).

- Events are triggered by counting the number of local rounds (which is different from the convention that events in PoW-based blockchains are triggered by the arrival of blocks). In other words, the protocol has a clock synchronization *interval* of length R and a target recalculation *epoch* of length M that are defined in terms of the number of rounds; in addition, M is a multiple of R . Both of these values are hardcoded in the protocol. More precisely, parties will call the `SyncProcedure` sub-protocol (see Protocol 12) when their local clock enters round $\langle \text{itvl}, \text{itvl} \cdot R \rangle$ (this represents the last round in interval itvl ; see below for details on the round structure); and for target in the next epoch they will call `UpdateMiningTarget` (Protocol 6) when their local clock enters $\langle \text{itvl}, \text{itvl} \cdot R \rangle$, where $(\text{itvl} \bmod (M/R)) = 0$ (i.e., at the boundary of every (M/R) synchronization intervals).

Next, we present the basic components that are employed in `Timekeeper`.

3.1 Timekeeper Timestamps

As opposed to the conventional approach where blocks' timestamps are integer values, timestamps (both blocks' and beacon values) in `Timekeeper` are represented by a pair of values interval number and round number $\langle \text{itvl}, r \rangle \in \langle \mathbb{N}^+, \mathbb{N}^+ \rangle$. Note that (ideally) one synchronization interval would last for R rounds (i.e., rounds $((i-1) \cdot R, i \cdot R]$ would belong to the i -th interval). However, in `Timekeeper` we let the lower bound be 0, which means that timestamps with a somewhat small round number are still valid. Specifically, a timestamp $\langle \text{itvl}, r \rangle$ is considered valid if and only if it satisfies the predicate $\text{validTS}(\text{itvl}, r) \triangleq r \leq \text{itvl} \cdot R$. We note that this new treatment allows for some small distortion at the end of each interval—i.e., the round number of a few blocks at the beginning of the next interval may be smaller than the last block of the previous interval (we call these “retorted” timestamps); see Figure 2.



Figure 2: An illustration of a segment of the blockchain with synchronization interval length $R = 100$. Blocks can have timestamp values equal to blocks in the previous interval.

Consider a chain of blocks in `Timekeeper`. Their timestamps should increase monotonically in terms of their interval number, and the round number in a single interval should also increase monotonically. More specifically, given two timestamps $\langle \text{itvl}_i, r_i \rangle, \langle \text{itvl}_j, r_j \rangle$ of two blocks $\mathcal{B}_i, \mathcal{B}_j$ respectively, if $\mathcal{B}_i$ is an ancestor block of $\mathcal{B}_j$, they should satisfy the following predicate:

$$\text{validTSOrder}(\langle \text{itvl}_i, r_i \rangle, \langle \text{itvl}_j, r_j \rangle) \triangleq \left\{ \begin{array}{l} \text{validTS}(\text{itvl}_i, r_i) \wedge \text{validTS}(\text{itvl}_j, r_j) \\ \wedge [(\text{itvl}_i \leq \text{itvl}_j) \vee (\text{itvl}_i = \text{itvl}_j \wedge r_i < r_j)] \end{array} \right\}.$$

Furthermore, we will overload the notation of comparison operators based on the valid order of timestamps. E.g., “=” will denote that two timestamps are identical, and $\langle \text{itvl}_1, r_1 \rangle < \langle \text{itvl}_2, r_2 \rangle$ if and only if $\text{validTSOrder}(\langle \text{itvl}_1, r_1 \rangle, \langle \text{itvl}_2, r_2 \rangle)$ holds. Other operators $>, \leq, \geq, \neq$ are defined similarly.

We also redefine “+,” “−” to describe the timestamp that is $k \in \mathbb{N}$ rounds before (resp., after) $\langle \text{itvl}, r \rangle$. Regarding addition, $\langle \text{itvl}, r \rangle + k = \langle \max\{\text{itvl}, \lceil (r+k)/R \rceil\}, r+k \rangle$. Intuitively, the additive operation simply adds k to r , and only increments itvl when it is going to become invalid. For subtraction, $\langle \text{itvl}, r \rangle - k = \langle \max\{1, \lceil (r-k)/R \rceil\}, \max\{1, r-k\} \rangle$. In other words, regarding the subtraction operation, we only apply the operation on round, and the interval number is derived

from the round after calculation. It does not output timestamps that do not “normally” belong to an interval. In case we do the subtraction operation for $k \geq r$, it will return $\langle 1, 1 \rangle$.

Timekeeper’s new approach to timestamps raises questions regarding the “trimming” of blockchains by counting the number of rounds. Recall that in [GKL15] the notation $\mathcal{C}^{\lceil k}$ represents the chain that results from removing the k rightmost blocks. In this paper, we overload this notation to denote the chain that results from removing blocks with timestamps in the last k rounds with respect to the current time. Specifically, for $\mathcal{C} = \mathcal{B}_1 \mathcal{B}_2 \dots \mathcal{B}_n$ and local time $\langle \text{itv1}, r \rangle$, $\mathcal{C}^{\lceil k} = \mathcal{B}_1 \mathcal{B}_2 \dots \mathcal{B}_m$ is the longest chain such that $\forall \mathcal{B} \in \mathcal{C}^{\lceil k}, \text{Timestamp}(\mathcal{B}) < \langle \text{itv1}, r \rangle - k$. In other words, $\mathcal{B}_{m+1}$ is the first block (if it exists) such that $\text{Timestamp}(\mathcal{B}_{m+1}) \geq \langle \text{itv1}, r \rangle - k$ holds.

3.2 2-for-1 Proofs of Work and Synchronization Beacons

2-for-1 PoW is a technique that allows protocols to utilize a single random oracle $H(\cdot)$ to compose two separate PoW sub-procedures involving two distinct and independent random oracles $H_0(\cdot), H_1(\cdot)$. It was first proposed in [GKL15] in order to achieve a better/optimal corruption threshold (from one-third to one-half) for the solution of the traditional consensus problem using a blockchain.

We refer to [GKL15] for more details, and here we present a simple implementation with the clock synchronization application in mind. In order to do the 2-for-1 mining, a party P prepares a composite input w that is a concatenation of two inputs w_0, w_1 of two different sub-procedures S_0, S_1 , respectively. I.e., $w = w_0 \parallel w_1$. After selecting a nonce ctr , querying the random oracle with $ctr \parallel w$ and getting result u , P checks if $u < T$ which implies success in sub-procedure S_0 ; P also checks if $[u]^R < T$ (where $[u]^R$ denotes the reverse of a bit-string u) which indicates success in sub-procedure S_1 . After successfully generating a PoW for S_0 (resp., S_1), in order to let other parties check validity, the proof will include the nonce and the entire composite input $ctr \parallel w$. Note that sub-procedure S_0 (resp., S_1) only cares about its corresponding part w_0 (resp., w_1), and treats the other part as dummy information.

The 2-for-1 PoW technique has several advantages when compared with the straightforward approach that would simply utilize two different random oracles. The most prominent advantage is that it prevents the adversary $\mathcal{A}$ from concentrating its computational power on one RO and thus gain advantage in the corresponding sub-procedure.

Synchronization beacons. In addition to the conventional blocks constituting the blockchain, protocol participants in Timekeeper also produce another type of “tiny” blocks using 2-for-1 PoWs. We call these blocks *clock synchronization beacons* (“beacons” for short) since they are used to report parties’ local time and synchronize their clocks.

In more detail, one clock synchronization beacon SB is a tuple with the following structure:

$$\text{SB} \triangleq \langle \langle \text{itv1}, r \rangle, P, \eta_{\text{itv1}}, ctr, \text{blockLabel} \rangle,$$

where $\langle \text{itv1}, r \rangle$ is the local time SB reports; P denotes the identity of its miner; η_{itv1} is some fresh randomness in the current interval; ctr represents the nonce of the PoW and blockLabel is the associated block input. Note that SB must record the identity of its miner because there might be multiple beacons, mined by different parties, reporting the same timestamp as well as nonce value; otherwise, it would be impossible for the parties to distinguish such beacons. Worse still, other participants would not be able to distinguish the same beacon SB when they receive SB multiple times. Regarding η_{itv1} , it is a string associated with interval itv1 for the purpose of preventing the adversary $\mathcal{A}$ from mining beacons with *future* timestamps. In other words, protocol participants (including $\mathcal{A}$) can only learn η_{itv1} after they have (almost) finished interval $\text{itv1} - 1$. We present

the structure of intervals in detail and how we compute η_{itv1} in Section 3.3 and treat it as a communal bit-string here. We note that parties can learn η_{itv1} from their local chain, and indeed SB does not need to include η_{itv1} (P can prune those beacons that are invalid with η_{itv1} in their local view). We keep η_{itv1} in the description for convenience.

Regarding the structure of a blockchain block $\mathcal{B}$, we adopt the similar structure as in [GKL17] (with the dummy information in the 2-for-1 PoWs):

$$\mathcal{B} \triangleq \langle h, \text{st}, \langle \text{itv1}, \mathbf{r} \rangle, \text{ctr}, \text{txLabel} \rangle,$$

where h is the reference to the previous block, st the Merkle root of the block content, $\langle \text{itv1}, \mathbf{r} \rangle$ its timestamp, ctr the nonce of PoW, and txLabel the bound beacon input.

We are now ready to describe how the parties in Timekeeper do the 2-for-1 PoW mining. The composite input prepared in Timekeeper is different from the trivial instance above, in that the term $\langle \text{itv1}, \mathbf{r} \rangle$ appears in both blocks and beacons. Hence, simply concatenating two inputs introduces redundant information in the PoW. When a party P is ready to perform the mining procedure, P binds the nonce, the blocks' input and beacon input together as

$$\langle \text{ctr}, h, \text{st}, \langle \text{itv1}, \mathbf{r} \rangle, \text{P}, \eta_{\text{itv1}} \rangle$$

and hand them over to random oracle $\mathcal{F}_{\text{RO}}$. Let u denote the result from $\mathcal{F}_{\text{RO}}$. If $u < T$ (i.e., the block query succeeds), P finds a new block $\mathcal{B} = \langle h, \text{st}, \langle \text{itv1}, \mathbf{r} \rangle, \text{ctr}, \text{txLabel} \rangle$ where $\text{txLabel} = \langle \text{P}, \eta_{\text{itv1}} \rangle$; if $[u]^R < T$ (the beacon query succeeds), P gets a new beacon $\text{SB} = \langle \langle \text{itv1}, \mathbf{r} \rangle, \text{P}, \eta_{\text{itv1}}, \text{ctr}, \text{blockLabel} \rangle$, where $\text{blockLabel} = \langle h, \text{st} \rangle$. Note that for the sake of presentation, we reorder the content of blocks and beacons so that they are inconsistent with the input to the PoW.

After receiving the result from $\mathcal{F}_{\text{RO}}$, P checks if it was able to successfully generate a new block. In addition, P checks if he successfully produces a beacon but only when P's local clock stays in the *beacon mining and inclusion* phase. Namely, P reports a timestamp that satisfies a certain criterion (details in Section 3.3).

3.3 Clock Synchronization Intervals and the Synchronization Procedure

As mentioned earlier, Timekeeper participants will periodically adjust their local clock. We call the time interval between two adjustment points⁷ a *clock synchronization interval* (or “interval” for short). Ideally, one interval will last for R rounds. The actual number of local rounds that parties observe may differ according to the shift computed in the previous interval (we will show later that the shift computed in every interval is well-bounded). When party P's local clock gets to the last round of an interval, it will call SyncProcedure (Protocol 12) which adjusts its local clock and gets the fresh randomness to run the next interval.

3.3.1 Interval Structure

Timekeeper divides one interval into three phases: (i) *view convergence*, (ii) *beacon mining and inclusion*, and (iii) *beacon-set convergence*. For each party P, the phase that P stays in depends on P's local clock. Furthermore, parties will keep track of the (local) arriving time of a synchronization beacon as long as it is online. In this section we describe these three phases as well as the bookkeeping function and explain the design intention behind them.

⁷The first interval in particular lies between the beginning of the execution and the first time parties adjust their clock.

View convergence. When a party P 's local clock reports a time $\langle \text{itv1}, r \rangle$ such that $r < (\text{itv1} - 1) \cdot R + K$, P is in the *view convergence* phase. Note that this also includes rounds with potentially retorted timestamps. In this phase, if P is alert, it will try to mine the next block with the 2-for-1 PoW technique (i.e., the input information that P forwards to the $\mathcal{F}_{\text{RO}}$ functionality does not need to be changed); nonetheless, P will not check if he successfully mines a beacon after P acquires the output. This is because all the beacons obtained in this phase are *invalid* in that they report an undesirable timestamp.

The general motivation for introducing the view convergence phase and letting parties wait for some period of time at the beginning of an interval is that we would like parties to start mining beacons with a *consistent* view on the previous interval. Since K is larger than the common prefix parameter (we will quantify K later, in Section 4.2), at the end of the view convergence phase of interval $\text{itv1} + 1$, alert parties will have a common view of interval itv1 . In other words, they will agree on all the blocks in interval itv1 , and the adversary $\mathcal{A}$ will not be able to apply any changes to these blocks. Hence, alert parties agree on the number of blocks in the previous interval, which decides the mining difficulty within the current interval. (This will be used in our new target recalculation function, presented in Section 3.4.) Parties will mine beacons with the same difficulty, and this simplifies the protocol description as well as its analysis. Furthermore, alert parties will compute the same fresh randomness as

$$\eta_{\text{itv1}+1} \triangleq G(\eta_{\text{itv1}} \parallel (\text{itv1} + 1) \parallel v), \quad (1)$$

where v is the concatenation of all block hashes in interval itv1 . Note that we adopt a different hash function $G(\cdot)$ (as opposed to $H(\cdot)$) to compute the next fresh randomness that is not used in the 2-for-1 PoW, which does not consume any queries to random oracle $\mathcal{F}_{\text{RO}}$.

Recall that by assumption the adversary $\mathcal{A}$ has full knowledge of the network, and hence it can learn all honest blocks from the previous interval immediately and manipulate the chain at will for up to a number of rounds bounded by the common prefix parameter, allowing $\mathcal{A}$ to mine the synchronization beacons before the alert parties start to mine. We call this period where $\mathcal{A}$ starts ahead of time the *pre-mining* stage. Nonetheless, we will show later that there will be at least one block generated by an alert party near the end of interval itv1 , which prevents the adversary from pre-mining for too long a time.

Remark 1. We note that, with some modifications, it is safe to get rid of the view convergence phase. The fresh randomness will still need to be extracted from the settled part of the chain, so we will replace it with the randomness generated in previous beacon mining and inclusion phase. This can be implemented by modifying some parameters but does not change the protocol framework. The main difference lies in how validity of a timestamp beacon is checked. At the beginning of a target recalculation epoch (which is also the beginning of an interval), parties may be mining on different chains, and hence mining beacons under different targets. In order to check if the beacon target is correctly computed, some additional information (e.g., the block height of the previous target recalculation epoch) should be included in the block header (and the height should not be less than the number of blocks in the settled part). Moreover, when parties run the synchronization procedure at the first interval in an epoch, they should use the “weighted” median timestamp instead of the plain median (and “weighted” means we would assign a weight to each timestamp based on its difficulty). Details on applying the median timestamp are given in Section 3.3.2.

Beacon mining and inclusion. When a party P 's local clock is in rounds $\langle \text{itv1}, r \rangle$ satisfying $(\text{itv1} - 1) \cdot R + K \leq r \leq \text{itv1} \cdot R - K$, P is in the *beacon mining and inclusion* phase. Next, we define the predicate $I_{\text{sync}}(\text{itv1})$ to extract the set of timestamps in this phase. Formally,

$$I_{\text{sync}}(\text{itv1}) \triangleq \{(\text{itv1} - 1) \cdot R + K, \dots, \text{itv1} \cdot R - K\}. \quad (2)$$

For convenience, we slightly overload this predicate. When the input is a timestamp, $I_{\text{sync}}(\langle \text{itvl}, \mathbf{r} \rangle)$ outputs whether $\langle \text{itvl}, \mathbf{r} \rangle$ stays in a beacon mining and inclusion phase. I.e., $I_{\text{sync}}(\langle \text{itvl}, \mathbf{r} \rangle) = \text{true}$ if $\mathbf{r} \in I_{\text{sync}}(\text{itvl})$, and false otherwise.

After entering this phase, P will use a 2-for-1 PoW to mine both blocks and clock synchronization beacons. During interval itvl , the output will be a beacon which indicates its local time and value $\text{SB} \triangleq \langle \langle \text{itvl}, \mathbf{r} \rangle, P, \text{ctr}, \text{blockLabel} \rangle$. Regarding the mining difficulty, Timekeeper will set the same target value for blocks and beacons.⁸ In other words, the expected number of blocks and of beacons in this phase are equal.

After a beacon is successfully generated, it will be diffused into the network via $\mathcal{F}_{\text{Diffuse}}^{\text{sync}}$. P will include a beacon SB into the pending block content if SB is valid w.r.t. the current interval. Next, we describe how to check the validity of a beacon. The format of a beacon SB with respect to interval itvl is correct if and only if it reports a timestamp $\langle \text{itvl}, \mathbf{r} \rangle$ such that $\mathbf{r} \in I_{\text{sync}}(\text{itvl})$. We say a beacon SB is *valid w.r.t. chain $\mathcal{C}$* if and only if its format is correct and the hash value of SB (after concatenating with the fresh randomness in $\mathcal{C}$) is smaller than the corresponding mining target. P will try to include all the (valid) beacons mined in the current interval itvl with timestamps earlier than the current local time but which have not yet been included in the blockchain. Specifically, at round $\langle \text{itvl}, \mathbf{r} \rangle$, all valid beacons recording timestamp $\langle \text{itvl}, u \rangle$ with $u \leq \mathbf{r}$ will get into P 's pending block content.

When P 's local clock goes past the last round of beacon mining and inclusion phase, it stops checking the beacon hash output and it no longer includes beacons in the next block. Beacons that are generated and diffused right at the end of this phase get dropped.

Beacon-set convergence. The third and last phase—*beacon-set convergence*—consists of the last K rounds in an interval. In other words, a party P is in this phase when P reports a timestamp $\langle \text{itvl}, \mathbf{r} \rangle$ with $\mathbf{r} > \text{itvl} \cdot R - K$. During this phase, P behaves similar to the first phase. I.e., it will not check for the 2-for-1 PoW result to see if the beacon generation succeeds.

Parties have to wait for at least K rounds to ensure that they share a *consistent* view of the set of beacons included in the current interval (except with some negligible probability). This phase cannot be omitted (as opposed to the case of first phase mentioned in Remark 1) since only when parties agree on the same beacon set can the synchronization procedure maintain the protocol's security properties (Section 4).

Beacon arrival booking. In order to adjust its clock, P also needs the local receiving time of all beacons that have been included in the chain. Hence, P will maintain a local registry that records the beacons it receives as well as their arrival time. More specifically, this local beacon ledger is an array of synchronization beacons. For each beacon SB , a pair $(a, \text{flag}) \in \langle \mathbb{N}^+, \mathbb{N}^+ \rangle \times \{\text{final}, \text{temp}\}$ is assigned to it. Consider a round $\langle \text{itvl}, \mathbf{r} \rangle$ when P receives a beacon SB with $\text{Timestamp}(\text{SB}) = \langle \text{itvl}', \mathbf{r}' \rangle$.

- If $\text{itvl}' \leq \text{itvl}$, which means the beacon SB is generated in the current or previous interval.⁹ P will drop SB if it is not valid w.r.t. its local chain; otherwise, P will assign $(\langle \text{itvl}, \mathbf{r} \rangle, \text{final})$ to SB . This means that all the information gathering regarding this beacon has been finalized and it is ready to be used.
- If $\text{itvl}' > \text{itvl}$, the beacon is generated in the future. P will assign $(\langle \text{itvl}, \mathbf{r} \rangle, \text{temp})$ to SB , which indicates that modifications on the receiving time may be applied in the future. Note that parties may not know the fresh randomness in future intervals (for example, if they are

⁸We will adopt the same target for simplicity. Indeed, maintaining a constant ratio between the difficulty level of blocks and that of beacons will work.

⁹Beacons generated in previous intervals are stale in that P has already passed the synchronization point associated with these beacons, and they will never be used in the future. We list them for completeness.

newly joint parties and have not yet synchronized with the blockchain or they are alert but receive forthcoming beacons). Hence they cannot check the validity of beacons with **temp** flag. Nevertheless, invalid beacons would be excluded from the registry after P learns the upcoming fresh randomness.

If P receives multiple beacon messages with the same creator and time reported, P will adopt the first one it receives as its arrival time.

3.3.2 The Synchronization Procedure

At the end of an interval (i.e., when the local time reports $r = \text{itvl} \cdot R$), parties will use the beacons information to compute a value **shift** that indicates how much the logical clock should be adjusted. (See Protocol 12 for the complete specification.)

Adjusting the local clock. When a party P 's local clock reaches round $\langle \text{itvl}, \text{itvl} \cdot R \rangle$ and P has finished the round's regular mining procedure, P will adjust its local clock based on the beacons recorded on chain and their local receiving time. More specifically, P will extract all beacons from blocks in the beacon mining and inclusion phase of interval itvl in her local chain (we denote this set of beacons by $\mathcal{S}_{\text{itvl}}^P$), and, for each beacon SB , compute the difference between its timestamp and local receiving time $\text{Timestamp}(SB) - \text{arrivalTime}(SB)$. Since the timestamp of SB and its arrival time share the same interval index, we only need to compute the difference between their round numbers. Subsequently, all the beacons will be ordered based on this difference and a **shift** will be computed by selecting the median difference therein. Formally,

$$\text{shift}_{\text{itvl}}^P \triangleq \text{med}\{\text{Timestamp}(SB) - \text{arrivalTime}(SB) \mid SB \in \mathcal{S}_{\text{itvl}}^P\}. \quad (3)$$

In case there are two median beacons SB_1, SB_2 , parties will adjust $\text{shift}_{\text{itvl}}^P \triangleq \lceil (\text{Timestamp}(SB_1) - \text{arrivalTime}(SB_1) + \text{Timestamp}(SB_2) - \text{arrivalTime}(SB_2)) / 2 \rceil$. Afterwards, P will update its local clock to $\langle \text{itvl} + 1, r + \text{shift}_{\text{itvl}}^P \rangle$. Later we show that this update strategy in the synchronization procedure allows parties' clocks to remain in a narrow interval and stay close to the nominal time.

Note that parties will enter local round $\langle \text{itvl}, r \rangle$ where $r = \text{itvl} \cdot R$ only once. If they enter some time $\langle \text{itvl}', r \rangle$ in the future, we will get $\text{itvl}' > \text{itvl}$ and they will never revert back.

Mining with backward-set clocks. After the adjustment at the end of intervals, and **shift** is added to P 's local clock, it may set its local time to values $\langle \text{itvl}, r \rangle$ such that $r \leq (\text{itvl} - 1) \cdot R$ (i.e., the retortion effect that was mentioned earlier). Nonetheless, P can continue to mine blocks with this timestamp and its local clock will eventually proceed to a time value of regular format (i.e., $r > (\text{itvl} - 1) \cdot R$).

We compare this treatment with the similar scenario in a PoS blockchain [BGK⁺21]. In [BGK⁺21], setting local clocks backward is never a problem since parties can keep silent during this period. Due to the nature of PoS-based blockchains, parties do not need to do anything if they are not assigned the leader slot. In our context, however, adopting the same 'silence' policy contradicts the basic nature of PoW-based blockchains as parties will forfeit the chance to extend their local chain. In other words, there is no point for an activate party to not make RO queries. This is taken care of by Timekeeper's timestamping scheme.

Updating the beacon arrival time registry. Notice that the beacon information stored in a party P 's arrival time registry is closely related to which interval P stays in; after P enters the next interval, it needs to update the beacon bookkeeping. P will apply a shift computation for all beacons with flag **temp**. Furthermore, for those beacons that report a timestamp with interval equal to the incoming one, their flag will be set to **final**. In more detail, at the end of interval itvl ,

for all eligible SB in the beacon registry, their associated pair $(\langle \text{itvl}_{\text{SB}}, \mathbf{r}_{\text{SB}} \rangle, \text{temp})$ will be updated to $(\langle \text{itvl}_{\text{SB}}, \mathbf{r}_{\text{SB}} + \text{shift} \rangle, \text{final})$ if $\text{itvl}_{\text{SB}} = \text{itvl} + 1$. Note that for those beacons whose flags are set to final, P will remove all invalid ones from the registry after the update.

3.4 The Target Recalculation Function

If the mining target is not set appropriately (“appropriately” means that the block generation rate according to the current hashing power and target is somewhat steady; see [GKL17]), PoW-based blockchain protocols fail to maintain any of the security properties in a permissionless environment. In Bitcoin, the target is adjusted after receiving the last block of the current epoch (and an epoch consists of 2016 blocks). Based on the time elapsed to mine these blocks, a new target is set based on the previous target value and the variation is proportional to the time elapsed. Note that Bitcoin’s target recalculation function is not the only way to adjust the difficulty level. A large number of other recalculation functions have been proposed in alternate blockchains (e.g., Ethereum, Bitcoin Cash, Litecoin), with their security asserted by either theoretical analysis or empirical data.

In *Timekeeper*, we propose a new target recalculation function that is suitable for the new setting. Intuitively, our function is a reversed version of Bitcoin’s original function, namely, protocol participants wait for some fixed number of rounds M (in their local view) to update the difficulty level. We call such M number of rounds a *target recalculation epoch*. Moreover, *Timekeeper* sets M as a multiple of R , which makes the target recalculation epoch consist of several clock synchronization intervals, and the start and end point of an epoch coincide with the start and end of different synchronization intervals. Recall that in the *Timekeeper* timestamp scheme introduced in Section 3.1, the first term in $\langle \text{itvl}, \mathbf{r} \rangle$ does not directly reflect which target recalculation epoch it is in. For simplicity, we introduce function *TargetRecalcEpoch* that maps the protocol timestamp to the target recalculation epoch it belongs to:

$$\text{TargetRecalcEpoch}(\langle \text{itvl}, \mathbf{r} \rangle) \triangleq \lceil \text{itvl} / (M/R) \rceil.$$

In addition, we introduce a function *EpochBlocks* which extracts all the blocks in chain $\mathcal{C}$ that belong to target recalculation epoch ep . Formally, given $\text{ep} \geq 1$,

$$\text{EpochBlocks}(\mathcal{C}, \text{ep}) \triangleq \{\mathcal{B} : \mathcal{B} \in \mathcal{C} \wedge \text{TargetRecalcEpoch}(\text{Timestamp}(\mathcal{B})) = \text{ep}\}.$$

Also for convenience, we let *EpochBlockCount* be a function that returns the number of blocks in chain $\mathcal{C}$ that belong to epoch ep . We also extend the input domain of epoch numbers to 0 and let it output Λ_{epoch} (the ideal number of blocks) to capture the fact that the target at the beginning of an execution is set appropriately and hence maintains the ideal block generation rate. Formally,

$$\text{EpochBlockCount}(\mathcal{C}, \text{ep}) \triangleq \begin{cases} |\text{EpochBlocks}(\mathcal{C}, \text{ep})| & \text{if } \text{ep} \geq 1 \\ \Lambda_{\text{epoch}} & \text{if } \text{ep} = 0 \end{cases} \quad (4)$$

Going back to the algorithm, for the first epoch ($\text{ep} = 1$) parties will adopt the target value of the genesis block (T_0). I.e., $T_1 = T_0$. Regarding other epochs ($\text{ep} > 1$), parties will figure out how many blocks are produced in the previous epoch, and set the next target based on the previous one. This variation is proportional to the ratio of expected number of blocks Λ_{epoch} and the actual number. I.e., for epoch $\text{ep} + 1$,

$$T_{\text{ep}+1} \triangleq \frac{\Lambda_{\text{epoch}}}{\Lambda} \cdot T_{\text{ep}}, \quad \text{ep} \in \mathbb{N}^+, \quad (5)$$

where Λ is the number of blocks in epoch ep —in other words, the size of $\text{EpochBlocks}(\mathcal{C}, \text{ep})$.

In order to prevent the “raising difficulty attack” [Bah13], the maximal target variation in a single recalculation step still needs to be bounded (we denote this bound by τ). Specifically, if $\Lambda > \tau \cdot \Lambda_{\text{epoch}}$, $T_{\text{ep}+1}$ will be set as T_{ep}/τ ; on the other hand, if $\Lambda < \Lambda_{\text{epoch}}/\tau$, $T_{\text{ep}+1}$ will be set as $\tau \cdot T_{\text{ep}}$.

Remark 2. We observe that, compared to the Bitcoin case, the adversary $\mathcal{A}$ in **Timekeeper** is in a much worse position to carry out the raising difficulty attack. This is because in Bitcoin, in order to significantly raise the difficulty in the next epoch, $\mathcal{A}$ only needs to mine 2016 blocks with close timestamps; in the case of **Timekeeper**, however, the adversary has to mine $\tau \cdot \Lambda_{\text{epoch}}$ blocks (with fake timestamps) in order to raise the same level of difficulty. The number of blocks that $\mathcal{A}$ needs to prepare is τ times larger than that in Bitcoin (assuming both protocols share the same number of expected blocks in an epoch).

3.5 Newly Joining Parties

Recall that **Timekeeper** runs in a permissionless environment where parties can join and leave at will. As such, it is essential that newly joining parties can learn the protocol time to become alert and participate in the core mining process. More specifically, after the joining procedure, newly joining party P ’s local clock should report a time in a sufficiently narrow interval with all other alert parties, at which point P can claim also being alert.

Based on the fine-grained classification of types of parties in our dynamic participation model (Section 2.3), newly joining parties can be classified into two types: (i) parties that are temporarily de-registered from $\mathcal{F}_{\text{RO}}$, and (ii) parties that start with bootstrapping from the genesis block, or parties that temporarily lose the network connection (i.e., de-registered from $\mathcal{F}_{\text{Diffuse}}$), or parties that are temporarily de-registered from $\mathcal{G}_{\text{ImpClock}}$.

For parties that are stalled for a while, since they do not miss any clock tick or other necessary information from the network, they can easily re-join by calling the procedure **SimulateClockAdjustments** (Protocol 14). For the rest of newly joining parties, they will be classified as de-synchronized (note that parties are aware of their synchronization status), and will run the joining procedure **JoinProc**, which we now describe.

Procedure JoinProcedure. In order to synchronize its clock, a newly joining party P needs to “listen” to the protocol for sufficiently long time. We describe the joining process below, which is similar to that in [BGK⁺21]. The main difference is that we adopt the heaviest-chain selection rule in order to adapt to the PoW context. The complete specification of this protocol is presented in Protocol 16, and the default parameters values are summarized in Table 2.

Parameter	Default	Phase
t_{off}	2K	B
t_{gather}	5R/2	C
t_{pre}	3K	D

Table 2: *Parameters of the joining procedure and their corresponding phases.*

- **Phase A (state reset).** When all resources are available to P , after resetting all its local variables, P invokes the main round procedure triggering the join procedure.
- **Phase B (chain convergence, with parameter t_{off}).** In the second activation upon a **MAINTAIN-LEDGER** command, the party will jump to phase B and stay in phase B for t_{off}

rounds. During this phase, the party applies the *heaviest-chain selection rule* `maxvalid` to filter its incoming chains. The motivation behind Phase B is to let P build a chain that shares a sufficiently long common prefix with all alert parties. Note that since P has not yet learnt the protocol time, it cannot filter out chains that should be put aside in the `futureChains`. Hence, the chain held by P may still contain a long suffix built entirely by the adversary. However, it can be guaranteed (Lemma 15(a)) that this adversarial fork can happen for up to k rounds ahead. Thus, the beacons recorded before the fork can be used to compute the adjustment and their local arrival times will be reliable.

- **Phase C (beacon gathering, with parameter t_{gather}).** Once a party P has finished Phase B, it continues with Phase C, the beacon-gathering phase. During this phase, P continues to collect and filter chains as in Phase B. In addition, P now processes and bookkeeps the beacons received from $\mathcal{F}_{\text{Diffuse}}^{\text{sync}}$. At a high level, this phases' length parameter t_{gather} guarantees that: (i) enough beacons are recorded to compute a reliable time shift; and (ii) enough time has elapsed so that the blockchain reaches agreement on the set of (valid) beacons to use. At the end of Phase C, P is able to reliably judge valid arrival times.
- **Phase D (shift computation, with parameter t_{pre}).** Since party P has now built a blockchain sharing a common prefix with alert parties, and has bookkeept synchronization beacons for a sufficiently long time, P starts from the earliest interval i^* such that (i) the arrival times of all beacons included in blocks within the beacon mining and inclusion phase of interval i^* have been locally bookkeept; and (2) all of these beacons arrived sufficiently later than the start of Phase C (parameterized by t_{pre} rounds). Based on this information, P computes the shift value as alert parties do at the boundary of synchronization interval i^* . P concludes Phase D when the adjusted time is a valid timestamp in interval $i^* + 1$ (in other words, r does not exceed $(i^* + 1)R$); otherwise, P updates the local arrival time of beacons with flag `temp` and repeats the above process with interval $i^* + 1$. We note that if Phase D involves the computation w.r.t. multiple intervals, the local time may temporarily be set as an invalid timestamp. Nevertheless, eventually after P has passed $(2K + 5R/2)$ (local) rounds, P will end up with a valid timestamp that with overwhelming probability is close enough to those of all alert parties.

4 Protocol Analysis

Our ultimate goal is to show that, at any point of the protocol's execution, the timestamps reported by all alert protocol participants of `Timekeeper` will satisfy the properties defined in Definition 1. We start off with some additional definitions and preliminary results.

4.1 Notation, Definitions and Preliminary Propositions

We note that several of the analytical tools proposed in [GKL17, GKL20] do not directly apply in the environment (with $\mathcal{G}_{\text{ImpClock}}$) where `Timekeeper` runs in. Therefore, we first extend and enhance these tools to adapt them to this new environment.

Our probability space is over all executions of length at most some polynomial in κ and λ ; we use $\mathbf{Pr}$ to denote the probability measure of this space. Furthermore, let $\mathcal{E}$ be a random variable taking values on this space and with a distribution induced by the random coins of all entities (adversary, environment, parties) and the random oracle.

For the sake of convenience, we define a *nominal time* that coincides with the internal variable τ_{sid} in $\mathcal{G}_{\text{ImpClock}}$. Recall that τ_{sid} aims at recording how many times the functionality sends clock ticks to all registered honest parties.

Definition 3 (Nominal Time). *Given an execution of Timekeeper, any prefix of the execution can be mapped deterministically to an integer r , which we call nominal time, as follows: r is the value of variable τ_{sid} in the clock functionality at the final step of the execution prefix which is obtained by parsing the prefix from the genesis block and keeping track of the honest party set registered with the clock functionality (bootstrapped with the set of inaugural alert parties). (In case no honest party exists in the execution, r is undefined).*

Note that we adopt r to denote the nominal time, which is different from the protocol timestamp $\langle \text{itvl}, r \rangle$.

If at a nominal round r exactly h parties query the oracle with target T , the probability of at least one of them will succeed is

$$f(T, h) = 1 - (1 - pT)^h \leq pTh, \text{ where } p = 1/2^\kappa.$$

During nominal round r , alert parties might be querying the random oracle for various targets. We denote by $T_r^{\min}$ and $T_r^{\max}$ the minimum and maximum of those targets. Moreover, the initial target T_0 implies in our model an initial estimate of the number of honest RO queries h_0 ; specifically, $h_0 = 2^\kappa \Lambda_{\text{epoch}} / (T_0 \cdot M)$, i.e., the number of parties it takes to produce Λ_{epoch} blocks of difficulty $1/T_0$ in time M . For convenience, we denote $f_0 = f(T_0, h_0)$ and simply refer to it as f . Also note that the ideal number of blocks $\Lambda_{\text{epoch}} = Mf$, so in the analysis we will use Mf to represent Λ_{epoch} .

“Good” properties. Next, we present some definitions which will allow us to introduce a few (“good”) properties, serving as an intermediate step towards proving the desired clock properties.

Let us consider the boundary of two target recalculation epochs. Recall that Bitcoin’s target recalculation algorithm defines epoch in terms of the number of blocks (m blocks forms an epoch). Thus, a block with block height a multiple of m is the last block of an epoch. While it might be manipulated, its timestamp naturally becomes the proof that miners have adjusted their difficulty and entered the next epoch (known as the *target recalculation point* [GKL17]). In contrast, Timekeeper adopts a new target recalculation function (see Section 3.4) that divides the epoch based on the parties’ local view. While we can still define a target recalculation point based on one party’s local view, parties can *never* agree on a point where they enter the next epoch based on nominal time.

In order to circumvent the above obstacle, we extend the notion of target recalculation point to target recalculation *zone*. See Figure 3 for an illustration. Intuitively, a “target recalculation zone” w.r.t. epoch ep is a sequence of consecutive nominal rounds such that during these nominal rounds, at least one alert party crosses its own target recalculation point w.r.t. epoch ep . For convenience, we assume a “safe” start—i.e., the first epoch also has a target recalculation zone, and it naturally satisfies all good properties we will later define.

Definition 4. — *Nominal time r is good if $f/2\gamma^2 \leq ph_r T_r^{\min}$ and $ph_r T_r^{\max} \leq (1 + \delta)\gamma^2 f$.*

- *Round $\langle \text{itvl}, r \rangle$ is a target-recalculation point w.r.t. epoch ep if $(r = \text{itvl} \cdot R) \wedge [\text{itvl} \bmod (M/R) = 0]$.*
- *A sequence of consecutive nominal rounds $Z_{\text{ep}} = \{r_i, \dots, r_j\}$ is a target recalculation zone w.r.t. target recalculation epoch ep if during Z_{ep} some subset of synchronized parties are in the logical round that is a target recalculation point w.r.t. $\text{ep} - 1$.*
- *A target-recalculation zone Z_{ep} is good if for all $h_r, r \in Z_{\text{ep}}$ the target T_{ep} satisfies $f/2\gamma \leq ph_r T_{\text{ep}} \leq (1 + \delta)\gamma f$.*
- *A chain is good if all its target-recalculation zones are good.*
- *A chain is stale if for some nominal time u it does not contain an honest block computed after nominal time $u - \ell - 2\Delta - 2\Phi$.*

time	$t-1$	t	$t+1$	$t+2$	$t+3$	$t+4$	$t+5$	$t+6$	$t+7$
P_1									
P_2									
P_3									
P_4									
P_5									
Env.	ep - 1	Target Recalculation Zone Z_{ep}							ep

Figure 3: An illustration of the target recalculation zone $Z_{ep} = \{t, \dots, t+6\}$. In the first and the last row (regarding [nominal] time and Env.), tan indicates the protocol execution is in epoch $ep-1$; blue implies the execution is in epoch ep ; and pink shows the target recalculation zone Z_{ep} . Regarding parties, tan represents RO queries w.r.t. $ep-1$, and blue represents RO queries w.r.t. ep .

- The blocklength of an epoch ep on a chain $\mathcal{C}$ is the number of blocks in $\mathcal{C}$ with timestamp $\langle \text{itvl}, \cdot \rangle$ such that $\text{TargetRecalcEpoch}(\langle \text{itvl}, r \rangle) = ep$.

We would like to prove that, at a certain nominal round r of the protocol execution, alert parties enjoy good properties on their local chains and reported timestamps. Towards this goal, we extract all chains that either belong to alert parties at r or have accumulated sufficient difficulty and thus might be adopted in the future. We denote this chain set by $\mathcal{S}_r$:

$$\mathcal{S}_r \triangleq \left\{ \mathcal{C} \in E_r \left| \begin{array}{l} \text{"}\mathcal{C} \text{ belongs to an alert party"} \text{ or} \\ \text{"}\exists \mathcal{C}' \in E_r \text{ that belongs to an alert party and } \text{diff}(\mathcal{C}) > \text{diff}(\mathcal{C}') \text{" or} \\ \text{"}\exists \mathcal{C}' \in E_r \text{ that belongs to an alert party and } \text{diff}(\mathcal{C}) = \text{diff}(\mathcal{C}') \\ \text{and head}(\mathcal{C}) \text{ was computed no later than head}(\mathcal{C}') \text{"} \end{array} \right. \right\}.$$

Next, we define a series of useful predicates with respect to the potential chain set $\mathcal{S}_r$ and parties' local clocks at nominal round r . Note that Φ is a constant that is the ideal maximal skew of all alert clocks, and $\Phi = \Delta + \Phi_{\text{clock}}$ where Δ is the network delay and Φ_{clock} is the maximal clock drift that $\mathcal{A}$ can set (see Section 2.1).

Definition 5. For a nominal round r , let:

- $\text{GOODCHAINS}(r) \triangleq$ "For all $u \leq r$, every chain in $\mathcal{S}_u$ is good."
- $\text{GOODROUND}(r) \triangleq$ "All rounds $u \leq r$ are good."
- $\text{NOSTALECHAINS}(r) \triangleq$ "For all $u \leq r$, there are no stale chains in $\mathcal{S}_u$."
- $\text{COMMONPREFIX}(r) \triangleq$ "For all $u \leq r$ and $\mathcal{C}, \mathcal{C}' \in \mathcal{S}_r$, $\text{head}(\mathcal{C} \cap \mathcal{C}')$ was created after nominal round $u - \ell - 2\Delta - 2\Phi$."
- $\text{BLOCKLENGTH}(r) \triangleq$ "For all $u < r$ and $\mathcal{C} \in \mathcal{S}_u$, the blocklength Λ of any epoch ep in $\mathcal{C}$ satisfies $\frac{1}{2(1+\delta)\gamma^2} \cdot \text{Mf} \leq \Lambda \leq 2(1+\delta)\gamma^2 \cdot \text{Mf}$ "
- $\text{GOODBEACONS}(r) \triangleq$ "For all $u < r$ and the beacon set $\mathcal{S}_{\text{itvl}}$ bookkeeping during any interval itvl , more than half of beacons within $\mathcal{S}_{\text{itvl}}$ are generated by honest parties."
- $\text{GOODSHIFT}(r) \triangleq$ "For all $u < r$, and the alert party P_i that adjusts its local clock at round u , P_i computes shift_i that $-2\Phi \leq \text{shift}_i \leq \Phi$."
- $\text{GOODSKEW}(r) \triangleq$ "For all alert parties in nominal time r , their local time in this round differs by at most Φ if they are in the same interval or differs by at most 2Φ if they are in different

intervals.” Formally,

$$\text{GOODSKEW}(r) :\Leftrightarrow \left(\forall P_1, P_2 \in \mathcal{P}_{\text{alert}}[r] : \begin{cases} |\mathbf{r}_1 - \mathbf{r}_2| \leq \Phi & \text{if } \text{itvl}_1 = \text{itvl}_2 \\ |\mathbf{r}_1 - \mathbf{r}_2| \leq 2\Phi & \text{if } \text{itvl}_1 \neq \text{itvl}_2 \end{cases} \right)$$

where $\langle \text{itvl}_1, \mathbf{r}_1 \rangle$ and $\langle \text{itvl}_2, \mathbf{r}_2 \rangle$ are the timestamps that P_1 and P_2 reports during r ¹⁰.

Random variables and (Δ, Φ) -isolated success. Next, for the purpose of estimating the difficulty acquired by honest parties during a sequence of rounds, we define the following random variables w.r.t. nominal round r .

- D_r : the sum of the difficulties of all blocks computed by alert parties at nominal round r .
- Y_r : the maximum difficulty among all blocks computed by alert parties at nominal round r .
- Q_r : equal to Y_r when $D_u = 0$ for all $r < u < r + \Delta + \Phi$ and 0 otherwise.

We call a nominal round r such that $D_r > 0$ *successful* and one wherein $Q_r > 0$ *isolated successful*. An isolated successful round guarantees the irreversible progress of the honest parties.

We highlight that, under the imperfect local clock model $\mathcal{G}_{\text{ImpClock}}$, the notion of an “isolated successful round” needs to be re-considered as parties’ local clocks may span some consecutive rounds. Assuming a Φ -drift is maintained during the sequence of rounds we are interested in, an irreversible contribution to the chain happens when the (nominal) distance between such success and the following success is at least $\Phi + \Delta$ rounds. This is because the block producer may have a local clock that is already Φ rounds behind other alert parties, and it takes Δ rounds to diffuse the block. This cancels out other parties’ successes for up to $\Phi + \Delta$ rounds. As a result, we call such event a (Δ, Φ) -*isolated successful*, which is for the new formulation of Q_r (cf. [GKL20]). Note that this (Δ, Φ) -isolated successful round is meaningful only when the protocol is able to maintain a Φ -bounded skew during the sequence of rounds we are considering.

Recall that the total number of hash queries alert parties (resp., the adversary) can make during nominal round r is denoted by h_r (resp., t_r). For a sequence of rounds S we write $n(S) = \sum_{r \in S} n_r$ and similarly, $t(S), D(S), Q(S)$.

Regarding the adversary $\mathcal{A}$, while $\mathcal{A}$ may query the random oracle for an arbitrarily low target and obtain blocks with arbitrarily high difficulty, we wish to upper-bound the difficulty it can accrue during a set of J queries. Consider a set of consecutive adversarial queries J and associate it with the target of the first query (this target is denoted by $T(J)$). We define $A(J)$ and $B(J)$ to be equal to the sum of the difficulties of all blocks computed by the adversary during queries in J for target at least $T(J)/\tau$ and $T(J)$, respectively.

Let $\mathcal{E}_{r-1}$ fix the execution just before (nominal) round r . In particular, a value E_{r-1} of $\mathcal{E}_{r-1}$ determines the adversarial strategy and so determines the targets against which every party will query the oracle at round r and the number of parties h_r and t_r , but it does not determine D_r or Q_r . For an adversarial query j we will write $\mathcal{E}_{j-1}$ for the execution just before this query.

Blockchain properties. We use blockchain properties as formulated in [GKL15, GKL17] as an intermediate step towards proving the clock properties and achieve our blockchain synchronizer. Next, we briefly describe these properties: common prefix, chain growth, chain quality and existential chain quality.

Notably, we consider common prefix in terms of number of rounds. I.e., honest parties will agree on a settled part of the blockchain with timestamps at most a given number of rounds before their

¹⁰If P passes multiple local rounds in nominal round r , we require that all of these timestamps should satisfy the predicate.

local time.¹¹ Let $\mathcal{C}^{\lceil k}$ denote the chain resulting from removing all rightmost blocks with timestamp larger than $\mathbf{r} - k$, where $\mathbf{r}$ is the current (local) time. We can now define common prefix as follows (we will quantify k in Corollary 6).

– **Common Prefix (with parameter $k \in \mathbb{N}$).** For any two alert parties P_1, P_2 holding chains $\mathcal{C}_1, \mathcal{C}_2$ at rounds r_1, r_2 , with $r_1 \leq r_2$, it holds that $\mathcal{C}_1^{\lceil k} \preceq \mathcal{C}_2$.

Regarding chain growth, the lemma below provides a lower bound on the irreversible progress of achieved by the honest parties regardless of any adversarial behavior. This lemma has appeared in previous analyses under varying settings, evolving from the synchronous network and static environment ([GKL15]), to a dynamic environment ([GKL17]), and further to a bounded-delay network setting ([GKL20]). The next lemma extends the chain growth property to a Δ -bounded network delay, Φ -bounded clock drift and dynamic environment.

Lemma 1 (Chain Growth). *Suppose that at nominal round u of an execution E an honest party diffuses a chain of difficulty d . Then, by (nominal) round v , every honest party has received a chain of difficulty at least $d + Q(S)$, where $S = [u + \Delta + \Phi, v - \Delta - \Phi]$.*

4.2 Protocol Parameters and Their Conditions

We summarize all Timekeeper parameters in Table 3 in Appendix E. It is worth noting that ϵ is a small constant regarding the quality of concentration of random variables (it will appear in the typical executions in Section 4.3). We introduce a parameter λ —which is related to the properties of the protocol—to simplify several expressions. Protocol parameter λ and the RO output length κ are the security parameters of Timekeeper.

In order to get desired convergence and perform meaningful analysis, we consider a sufficiently long consecutive sequence of at least

$$\ell = \frac{4(1 + 3\epsilon)}{\epsilon^2 f[1 - (1 + \delta)\gamma^2 f]^{\Delta + \Phi + 1}} \cdot \max\{\Delta + \Phi, \tau\} \cdot \gamma^3 \cdot \lambda \quad (6)$$

consecutive rounds.

We are now ready to discuss the conditions that protocol parameters should satisfy. We first quantify the length of a clock synchronization interval R , the length of a target recalculation interval M and the length of the convergence phase K . Specifically, we let one target recalculation epoch consists of 4 clock synchronization intervals, i.e., $M = 4R$; we set $K = \ell + 2\Delta + 4\Phi$ (this will coincide with our common prefix parameter and thus provide some desired properties; see Corollary 6).

Next, we will require that ℓ (defined in Equation (6)) is appropriately small compared to the length of an epoch and of an interval (note that $M = 4R$).

$$\ell + 2\Delta + 7\Phi \leq \epsilon M / (4\gamma) = \epsilon R / \gamma. \quad (C1)$$

Further, we require that the advantage of the honest parties is large enough to absorb the errors introduced by ϵ (from the concentration of random variables) and $[1 - (1 + \delta)\gamma^2 f]^{\Delta + \Phi}$ (from the network delay and clock skews).

$$[1 - (1 + \delta)\gamma^2 f]^{\Delta + \Phi} \geq 1 - \epsilon \quad \text{and} \quad \epsilon \leq \delta/12 \leq 1/12. \quad (C2)$$

¹¹While most of the previous work considers common prefix in terms of number of blocks, we note that these two definitions are equivalent. This is due to the fact that if the protocol guarantees security, then the block generation rate is somewhat steady (cf. [GKL17]) and thus the number of blocks generated during a period of time can be inferred from its length and the highest mining speed.

4.3 Typical Executions

We define the notion of *typical* executions following [GKL17, GKL20]. The idea here is that given a certain execution E , we compare the actual progress and the expected progress that parties will make under the success probabilities. If the difference and variance are reasonably small, and no bad events (see Definition 6) about the underlying hash function happen, we declare E *typical*.

Definition 6. *An insertion occurs when, given a chain $\mathcal{C}$ with two consecutive blocks $\mathcal{B}$ and $\mathcal{B}'$, a block $\mathcal{B}^*$ created after $\mathcal{B}'$ is such that $\mathcal{B}, \mathcal{B}^*, \mathcal{B}'$ form three consecutive blocks of a valid chain. A copy occurs if the same block exists in two different positions. A prediction occurs when a block extends one with later creation time.*

Note that in addition (compared to [GKL17, GKL20]), in Definition 7(a) we require that the difficulty of all blocks the alert parties can acquire during consecutive rounds S (i.e., $D(S)$) is well lower-bounded. This is because $D(S)$ also captures the beacon production process, where there is no loss incurred by the bounded-delay network as well as by skewed local clocks. Hence, a reasonably better lower-bound on $D(S)$ helps us get better results when arguing for the good properties of generated beacons by alert parties (Lemma 11).

Definition 7 (Typical Execution). *An execution E is typical if the following hold.*

(a) *For any set S of at least ℓ consecutive good rounds,*

$$(1 - \epsilon)[1 - (1 + \delta)\gamma^2 f]^\Delta ph(S) < Q(S) \leq D(S) < (1 + \epsilon)ph(S) \text{ and } D(S) > (1 - \epsilon)ph(S).$$

(b) *For any set J of consecutive adversarial queries and $\alpha(J) = 2(\frac{1}{\epsilon} + \frac{1}{3})\lambda/T(J)$,*

$$A(J) < p|J| + \max\{\epsilon p|J|, \tau\alpha(J)\} \text{ and } B(J) < p|J| + \max\{\epsilon p|J|, \alpha(J)\}.$$

(c) *No insertions, no copies, and no predictions occurred in E .*

In the next lemma, we establish the quantitative relation between honest and adversarial hashing power during consecutive rounds with length at least ℓ , as well as the relationship between the total difficulty acquired by all parties ($D(S) + A(J)$) and their hashing power.

Lemma 2. *Consider a typical execution in a (γ, s) -respecting environment. Let $S = \{r : u \leq r \leq v\}$ be a set of at least ℓ consecutive good rounds and J the set of adversarial queries in $U = \{r : u - \Delta - \Phi \leq r \leq v + \Delta + \Phi\}$. We have*

- (a) $(1 + \epsilon)p|J| \leq Q(S) \leq D(U) < (1 + 5\epsilon)Q(S)$.
- (b) $T(J)A(J) < \epsilon Mf/4(1 + \delta)$ or $A(J) < (1 + \epsilon)p|J|$; $\tau T(J)B(J) < \epsilon Mf/4(1 + \delta)$ or $B(J) < (1 + \epsilon)p|J|$.
- (c) *If w is a good round such that $|w - r| \leq s$ for any $r \in S$, then $Q(S) > (1 - \epsilon)[1 - (1 + \delta)\gamma^2 f]^\Delta |S|pn_w/\gamma$. If in addition $T(J) \geq T_w^{\min}$, then $A(J) < (1 - \delta + 3\epsilon)Q(S)$.*
- (d) *If w is a good round such that $|w - r| \leq s$ for any $r \in S$ and $T(J) \geq T_w^{\min}$, then $D(S) + A(J') < (1 + \epsilon)p(h(S) + |J'|)$ where J' denotes the set of adversarial queries in S .*

We conclude that almost all executions (that are polynomially bounded by κ and λ) are typical.

Theorem 3. *Assuming the ITM system $(\mathcal{Z}, C)$ runs for L steps, the probability of the event “ $\mathcal{E}$ is not typical” is bounded by $O(L^2)(e^{-\lambda} + 2^{-\kappa})$.*

4.4 Proof Roadmap

In the remainder of this section we present an overview of the analysis. Note that the predicates in Definition 5 are proved in an inductive way over the space of typical executions in a (γ, s) -respecting environment.

First, we focus on the steady block generation rate. For a warm-up, we argue that an adversarial fork cannot happen too long ago and then extract the common prefix parameter. Equipped with this knowledge, we show that if good skews and certain time adjustment calculations are maintained during a target recalculation epoch, the block production rate will be properly controlled in the next epoch.

Lemma 4. $\text{GOODROUND}(r-1) \implies \text{NOSTALECHAINS}(r)$.

Lemma 5. $\text{GOODROUND}(r-1) \wedge \text{GOODSKEW}(r-1) \implies \text{COMMONPREFIX}(r)$.

Lemma 7. $\text{GOODROUND}(r-1) \wedge \text{GOODCHAINS}(r-1) \wedge \text{GOODSKEW}(r-1) \wedge \text{GOODSHIFT}(r-1) \implies \text{BLOCKLENGTH}(r)$.

Lemma 8. $\text{GOODROUND}(r-1) \implies \text{GOODCHAINS}(r)$.

Corollary 9. $\text{GOODROUND}(r-1) \implies \text{GOODROUND}(r)$.

Next, we move to the properties w.r.t. clocks. We argue that if at the onset, the PoW difficult is appropriately set and the steady block generating rate lasts during the whole clock synchronization interval, the beacon set used by parties to update their clock will be identical and the majority of these beacons will be produced and emitted by alert parties. For synchronized parties, this good beacon set implies that the differences between alert parties' local clocks are still narrow after they enter the next interval and that the shift value they computed is well-bounded. Furthermore, regarding newly joining parties, we also provide an analysis of the joining procedure showing that joining parties starting with no *a-priori* knowledge of the global time, they can listen in and bootstrap their logical clock and become alert parties. The above two aspects imply that a bounded skew is maintained over the whole execution.

Lemma 11. $\text{GOODROUND}(r-1) \implies \text{GOODBEACONS}(r)$.

Lemma 13. $\text{GOODSKEW}(r-1) \implies \text{GOODSHIFT}(r)$

Lemma 16. $\text{GOODSKEW}(r-1) \wedge \text{GOODBEACONS}(r-1) \implies \text{GOODSKEW}(r)$.

To sum up, a “safe” start and a (γ, s) -respecting environment guarantee that good properties can be achieved during the whole execution. We work out the related parameters (in Theorem 18) and conclude that Timekeeper solves the clock synchronization problem.

4.5 Typical Executions Maintain a Steady Block Generation Rate

We first show that the adversary cannot create a fork that happened too long ago compared with the current time. We consider a novel partition strategy (different from the approach in [GKL20]) that is better suited to the new target recalculation function.

Lemma 4. $\text{GOODROUND}(r-1) \implies \text{NOSTALECHAINS}(r)$.

Proof. Suppose—towards a contradiction— $\mathcal{C} \in \mathcal{S}_r$ and has not been extended by an honest party for at least $\ell + 2(\Phi + \Delta)$ rounds and r is the least (nominal) round with this property. Let $\mathcal{B}$ be the last honestly-generated block of $\mathcal{C}$ (possibly the genesis) and let w be the (nominal) round it was computed. We consider $S = \{u : w + (\Phi + \Delta) \leq u \leq r - (\Phi + \Delta)\}$ and $U = \{u : w \leq u \leq r\}$ ($|S| \geq \ell$ by assumption). Suppose that the blocks of $\mathcal{C}$ after $\mathcal{B}$ (we denote these blocks by B) span k epochs with corresponding targets $T_1, \dots, T_k$. For $i \in [k]$ let m_i be the number of blocks with target T_i and set $M = m_1 + \dots + m_k$ and $d = m_1/T_1 + \dots + m_k/T_k$. Our plan is to contradict the assumption that $\mathcal{C} \in \mathcal{S}_r$ by showing that all chains in $\mathcal{S}_r$ have more difficulty than $\mathcal{C}$. By Chain-Growth Lemma 1, all the honest parties have advanced (in difficulty) during the rounds in U by $Q(S)$. Therefore, to reach a contradiction it suffices to show that $d < Q(S)$.

Consider the following partition on B : we partition B into u sections $B_v, v \in [u]$ and associate each section B_v with the target of its first block T_v . Section B_v starts with either the block after $\mathcal{B}$ (if $v = 1$) or the $\lceil m_i/2 \rceil$ -th block in an epoch (if $v > 1$); it ends at either the last block of the chain (if $v = u$) or the $\lfloor m_i/2 \rfloor$ -th block such that in epoch $i + 1$ the target is less than T_v/τ . Under such construction, the next block after partition B_v is exactly the first block of partition B_{v+1} .

For $u \geq 2$, we claim that for partition B_v , it has the following properties: (i) for all blocks in B_v , their target is at least T_v/τ ; (ii) the number of blocks in B_v is at least $Mf/2$.

Property (i) holds because of the strategy of our partition that will stop before it exceeds the lower bound for the targets and thanks to Equation (5) we need to pass at least two boundaries of epochs so the circumstance that no blocks exist in such partition will never happen. To reason why property (ii) stands, consider those epochs that are split into two different sections. For an epoch $\mathbf{ep}$ whose blocks are split into two sections B_v, B_{v+1} , since in epoch $\mathbf{ep} + 1$ the target is larger than that in $\mathbf{ep}$ (if not, it does not satisfy the criteria of the partition), there are at least Mf blocks in epoch $\mathbf{ep}$. Otherwise, Equation (5) will raise the target. By the rule of partition, at least $Mf/2$ blocks are in each sections. Hence for every partition, either its head or tail has at least $M/2$ blocks in the same epoch, and this implies the lower bound of the total number of blocks.

For each $v \in \{1, 2, \dots, u\}$, let $j_v \in J$ denote the index of the query during which the first block of the v -th section was computed and set $J_v = \{j : j_v \leq j < j_{i+1}\}$ (Definition 7(c) assures $j_i < j_{i+1}$). We have

$$d = \sum_{i=1}^k \frac{m_i}{T_i} < \sum_{v=1}^u (1 + \epsilon) |J_v| \leq (1 + \epsilon) p |J| \leq Q(S).$$

The difficulty of the blocks acquired in J_v is at most $A(J_v)$ by property (i) and their number at most $T_v A(J_v)$. Since property (ii) above shows that the adversary acquired at least $M/2$ blocks in J_v , the desired bound follows from Lemma 2(b). The final inequality is Lemma 2(a).

If $u = 1$, let J denote the queries in U starting from the first adversarial query attempting to extend $\mathcal{B}$. Then, $T_1 = T(J)$ and $T_i \geq T(J)/\tau (i > 1)$; thus, $d \leq A(J)$. If $A(J) < (1 + \epsilon)p|J|$, then $A(J) < Q(S)$ is obtained by Lemma 2(a). Otherwise, $A(J) < (\frac{1}{\epsilon} + 1)\tau\alpha(J) = 2(\frac{1}{\epsilon} + 1)(\frac{1}{\epsilon} + \frac{\epsilon}{3})\tau\lambda/T(J)$. However, we have

$$\begin{aligned} Q(S) &> (1 - \epsilon)[1 - (1 + \delta)\gamma^2 f]^{\Delta + \Phi} \cdot \frac{pn_u \ell T_1}{\gamma T_1} \\ &> \frac{(1 - \epsilon)[1 - (1 + \delta)\gamma^2 f]^{\Delta + \Phi} f \ell}{2\gamma^3 T(J)} \geq \frac{2(1 - \epsilon)(1 + 3\epsilon)\tau\lambda}{\epsilon^2 T(J)} \geq A(J) \end{aligned}$$

by considering only the first ℓ rounds in S hence $n(S) \geq n_u \ell / \gamma$. $\square$

Lemma 5. $\text{GOODROUND}(r-1) \wedge \text{GOODSKEW}(r-1) \implies \text{COMMONPREFIX}(r)$.

The proof is presented in Appendix D.

Previous works [GKL17, GKL20] consider the common prefix parameters in terms of the number of blocks parties have to prune, which directly maps to the number of blocks produced in the time period implied by Lemma 5. With an imperfect local clock $\mathcal{G}_{\text{ImpClock}}$ as well as the new definition of common prefix (in terms of the number of rounds that we are going to remove), parties have to prune more rounds in order to guarantee it. We establish the new Common Prefix parameter k in Corollary 6, which coincides with the K boundaries of $I_{\text{sync}}(\cdot)$ in Equation (2), and will be helpful in order to argue the good properties of clock skews.

Corollary 6. *For a typical execution in a (γ, s) -respecting environment, if all predicates in Definition 5 hold till $r-1$, for any two alert parties P_1, P_2 holding chains $\mathcal{C}_1, \mathcal{C}_2$ at round r , it holds that $\mathcal{C}_1^{[k]} \preceq \mathcal{C}_2$, where $k = \ell + 2\Delta + 4\Phi$.*

Proof. Lemma 5 shows that common prefix can be acquired by pruning the blocks produced in the last $\ell + 2\Delta + 2\Phi$ nominal rounds. We consider the timestamp of the first honest block $\mathcal{B}$ produced during such nominal rounds. $\text{GOODSKEW}(r-1)$ indicates that the timestamp of $\mathcal{B}$ can be up to Φ -rounds behind, if $\mathcal{B}$ is produced by an alert party that is Φ -rounds behind other alert parties. Meanwhile, $\text{GOODSHIFT}(r-1)$ implies that up to Φ logical rounds may be skipped at the boundary of an interval. Note that parties can pass at most two intervals within $\ell + 2\Delta + 2\Phi$ nominal rounds, and setting back local clocks does not hurt common prefix in that it is safe to prune more blocks. By summing them up, we conclude $k = \ell + 2\Delta + 4\Phi$. $\square$

Lemma 7. $\text{GOODROUND}(r-1) \wedge \text{GOODCHAINS}(r-1) \wedge \text{GOODSKEW}(r-1) \wedge \text{GOODSHIFT}(r-1) \implies \text{BLOCKLENGTH}(r)$.

Proof. Suppose—towards a contradiction—that $\text{BLOCKLENGTH}(r)$ is false. Then, there exists a $w \leq r$ and a chain $\mathcal{C} \in \mathcal{S}_w$ with an epoch of target T and duration Λ that does not satisfy

$$\frac{1}{2(1+\delta)\gamma^2} \cdot \text{Mf} \leq \Lambda \leq 2(1+\delta)\gamma^2 \cdot \text{Mf}.$$

We consider the earliest epoch ep with this property.

For the lower bound, we show that alert parties can alone produce enough blocks. Let u, v denote the first and last nominal round such that all alert parties are in epoch ep . Such u, v exist since $\text{GOODSKEW}(r-1)$ holds. Define $S = \{i : u + \ell + 2\Delta + 2\Phi \leq i \leq v - (\ell + 2\Delta + 2\Phi)\}$. We consider the length of S . Since one epoch consists of 4 intervals and $\text{GOODSHIFT}(r-1)$, $\text{GOODSKEW}(r-1)$ holds, we get $v - u \geq M - 4\Phi - 2\Phi = M - 6\Phi$ (we prune Φ rounds at the beginning and end to ensure that alert parties stay in ep). Further, by applying Condition (C1) we get $|S| \geq M - (2\ell + 4\Delta + 10\Phi) \geq (1 - \epsilon)M$. We have

$$Q(S) > (1 - \epsilon)[1 + (1 + \delta)\gamma^2 f]^{\Delta + \Phi} \cdot \frac{f|S|}{2\gamma^2 T} \geq (1 - \epsilon)^3 \cdot \frac{\text{Mf}}{2\gamma^2 T} \geq \frac{\text{Mf}}{2(1 + \delta)\gamma^2} \cdot \frac{1}{T}.$$

The first inequality follows that nominal round $u-1$ belongs to target recalculation zone Z_{ep} and is good (note that $h_{u-1} \geq h_w/\gamma, w \in S$ holds); the second and third inequalities comes from the lower bound on $|S|$ and Condition (C2). By Chain Growth (Lemma 1), this implies that alert parties has produced at least $\text{Mf}/2(1 + \delta)\gamma^2$ blocks.

For the upper bound, we show that even if the alert parties and the adversary join force, they cannot produce more than $2(1 + \delta)\gamma^2 \cdot \text{Mf}$ blocks with timestamp in epoch ep . Let u, v

denote the first and last nominal round such that at least one alert party is in epoch **ep**. Define $S = \{i : u \leq i \leq v\}$ and $S' = \{i : u - (\ell + 2\Delta + 2\Phi) \leq i \leq v + \ell + 2\Delta + 2\Phi\}$, and J the set of queries available to the adversary during the rounds in S' starting with the first query for target T (so that $T(J) = T$). $\text{COMMONPREFIX}(r)$ implies that all adversarial queries that contributed to epoch **ep** are all in J . By $\text{GOODSHIFT}(r-1)$ and $\text{GOODSKEW}(r-1)$ we get $|S| \leq M + 8\Phi + 2\Phi = M + 10\Phi$ (we add Φ rounds at the beginning and end to ensure that at least one alert parties stay in **ep**). Furthermore, $|S| < |S'| \leq M + 2\ell + 4\Delta + 14\Phi \leq (1 + \epsilon)M$ by Condition (C1). Since $u - 1$ is a nominal round within target recalculation zone Z_{ep} and $h_w \leq \gamma h_{u-1}$ for all $w \in S'$, it follows that $ph(S) \leq p\gamma h_{u-1}|S| \leq (1 + \delta)\gamma^2 f|S|/T$. Hence,

$$D(S) < (1 + \epsilon)ph(S) \leq (1 + \epsilon)(1 + \delta)\gamma^2 f|S| \cdot \frac{1}{T} < (1 + \epsilon)^2 \cdot (1 + \delta)\gamma^2 Mf \cdot \frac{1}{T}.$$

Regarding the adversary $\mathcal{A}$, if $\tau TB(J) < \epsilon Mf/4$, the total number of blocks is less than $2(1 + \delta)\gamma^2 Mf$ and we are done. Otherwise,

$$B(J) < (1 + \epsilon)p|J| \leq (1 + \epsilon)(1 - \delta)(1 + \delta)\gamma^2 f|S'| \cdot \frac{1}{T} \leq (1 + \epsilon)^2(1 - \delta)(1 + \delta)\gamma^2 Mf \cdot \frac{1}{T}.$$

The second inequality holds because $t_w < (1 - \delta)h_w$ for all $w \in S'$, and the last one follows from the upper bound on $|S'|$. Note that $\epsilon < \delta/4$ (by Condition (C2)), we have $(1 + \epsilon)^2(2 - \delta) < 2$. I.e., the total number of blocks is less than $2(1 + \delta)\gamma^2 Mf$. $\square$

Note that the length of S' in the above proof implies that we should require $s \geq M + 2(\ell + 2\Delta + 7\Phi)$.

Lemma 8. $\text{GOODROUND}(r-1) \implies \text{GOODCHAINS}(r)$.

Proof. Recall that the first target recalculation zone Z_1 (which consists of merely the first round) is good in our assumption, it suffices to show that if a recalculation zone Z_{ep} is good, then the next one $Z_{\text{ep}+1}$ is also good. Let Λ denote the number of blocks in epoch **ep**. we wish to show that, for all $z \in Z_{\text{ep}+1}$, $f/2\gamma \leq ph_z T_{\text{ep}+1} \leq (1 + \delta)\gamma f$.

We first consider the lower bound. Consider a round $w \in Z_{\text{ep}}$ and a round $z \in Z_{\text{ep}+1}$. If $\Lambda \leq Mf/\gamma$, we get $T_{\text{ep}+1} \geq \gamma T_{\text{ep}}$ according to the target recalculation function (see Equation (5)). Hence, $ph_z T_{\text{ep}+1} \geq ph_w T_{\text{ep}+1}/\gamma \geq ph_w T_{\text{ep}} \geq f/2\gamma$ in that w belongs a good recalculation zone.

If not, assume $\Lambda > Mf/\gamma$. Let u, v denote the first and last nominal round such that at least one alert party is in epoch **ep** and consider $S = \{i : u \leq i \leq v\}$, $S' = \{i : u - (\ell + 2\Delta + 2\Phi) \leq i \leq v + \ell + 2\Delta + 2\Phi\}$. Let J denote the set of queries available to the adversary in S' . By Lemma 5, all blocks contributed to Λ were computed during honest queries in S or adversarial ones in J . From the discussion in Lemma 7 we learn that $|S| < |S'| \leq M + 4\Delta + 14\Phi \leq (1 + \epsilon)M$. In addition, for any round $z \in Z_{\text{ep}+1}$, $h(S') \geq h_z|S'|/\gamma$ in that $|S'| < s$; similarly, $h(S) \geq h_z|S|/\gamma$. Thus, recall that $M = (T_{\text{ep}+1}/T_{\text{ep}})(1/f)\Lambda$, we have

$$B(J) < (1 - \delta)(1 + \epsilon)ph(S') \leq (1 - \delta)(1 + \epsilon)^2 p\gamma h_u M$$

and $D(S) < (1 + \epsilon)ph(S) \leq (1 + \epsilon)^2 p\gamma h_u M$. For any $z \in Z_{\text{ep}+1}$, assume $ph_z T_{\text{ep}+1} < f/2\gamma$, we get he following contradiction.

$$2p\gamma h_z M = 2p\gamma h_z \cdot \frac{T_{\text{ep}+1}}{T_{\text{ep}}} \cdot \Lambda \cdot \frac{1}{f} < \Lambda \cdot \frac{1}{T_{\text{ep}}} \leq D(S) + B(J) < (2 - \delta)(1 + \epsilon)^2 p\gamma h_z M < 2p\gamma h_z M.$$

In order to prove the upper bound ($f \leq (1 + \delta)\gamma f$), consider a round $w \in Z_{\text{ep}}$ and a round $z \in Z_{\text{ep}+1}$. If $\Lambda \geq \gamma M f$, we get $T_{\text{ep}+1} \leq T_{\text{ep}}/\gamma$. Thus, $ph_z T_{\text{ep}+1} \leq p\gamma h_w T_{\text{ep}+1} \leq ph_w T_{\text{ep}} \leq (1 + \delta)\gamma f$ in that w belongs a good recalculation zone and we are done.

Otherwise, assume $\Lambda < \gamma M f$. Let u, v denote the first and last nominal round such that all alert parties are in epoch ep and consider $S = \{i : u + \ell + 2\Delta + 2\Phi \leq i \leq v - (\ell + 2\Delta + 2\Phi)\}$. Following the argument in Lemma 7 we get $|S| \geq (1 - \epsilon)M$. For any $t \in Z_{\text{ep}+1}$, assume $ph_z T_{\text{ep}+1} > (1 + \delta)\gamma f$ and recall that $M = (T_{\text{ep}+1}/T_{\text{ep}})(1/f)\Lambda$, we obtain the following contradiction.

$$\frac{ph_z M}{(1 + \delta)\gamma} = \frac{ph_z}{(1 + \delta)\gamma} \cdot \frac{T_{\text{ep}+1}}{T_{\text{ep}}} \cdot \Lambda \cdot \frac{1}{f} > \Lambda \cdot \frac{1}{T_{\text{ep}}} \geq Q(S) \geq (1 - \epsilon)(1 - (1 + \delta)\gamma^2 f)^{\Delta + \Phi} \cdot \frac{ph_z |S|}{\gamma} \geq \frac{ph_z M}{(1 + \delta)\gamma}.$$

The second inequality comes from Chain Growth (Lemma 1) (recall that $\mathcal{C} \in \mathcal{S}_r$ and the adversary can discard alert parties for at most $\ell + 2\Delta + 2\Phi$ rounds at the beginning and end of epoch as a result of $\text{COMMONPREFIX}(r)$, Lemma 5); the next one holds in that $M < s$ and hence $h(S) \geq h_z |S|/\gamma$; the last inequality follows Condition (C2). $\square$

Corollary 9. $\text{GOODROUND}(r - 1) \implies \text{GOODROUND}(r)$.

Proof. Consider any $\mathcal{C} \in \mathcal{S}_r$. Let Z_{ep} be its last target recalculation zone before r . If $r \in Z_{\text{ep}}$, it follows directly by Lemma 8 that it is good. Otherwise, consider a round $w \in Z_{\text{ep}}$ (recall that $f/2\gamma \leq ph_w T_{\text{ep}} \leq (1 + \delta)\gamma f$). Since $\text{GOODSHIFT}(r - 1)$, $\text{GOODSKEW}(r - 1)$ implies $r - w < M + 8\Phi + \Phi = M + 9\Phi < s$, we have $h_r/\gamma \leq h_w \leq \gamma h_r$. Combining these two bounds we obtain the desired inequality. $\square$

4.6 Typical Executions Maintain Good Skews

In this section, we show that in a typical execution, alert parties maintain close local clocks, and that the shift that they compute at the end of an interval is well-bounded. Before we consider $\text{GOODBEACONS}(r)$, we establish the following lemma that bounds the adversary from pre-mining for too long a time.

Lemma 10. *Consider an interval itv1 , and suppose r is the smallest nominal round where all alert parties stay in the beacon mining and inclusion phase. Then the adversary can mine timestamp beacons w.r.t. itv1 no earlier than $r - (2\ell + 4\Delta + 9\Phi)$.*

Proof. Since we adopt the CRS recorded in the genesis block as the original fresh randomness, by our simultaneously-start assumption, in interval $\text{itv1} = 1$ the adversary can start to mine beacons at most $K = \ell + 2\Delta + 4\Phi$ rounds before the alert parties.

Consider an interval $\text{itv1} > 1$. Recall that in Equation (1), the fresh randomness w.r.t. interval itv1 is computed by hashing the concatenation of all blocks in the previous interval, in order to prove the lemma, it suffices to show that the production time of the last block is no earlier than $r - 2\ell + 4\Delta + 9\Phi$. Otherwise, it implies a prediction (cf. Definition 6), which contradicts the fact that execution is typical.

For the sake of a contradiction, assume that the last block $\mathcal{B}$ in interval $\text{itv1} - 1$ is computed at a nominal round $r' < r - 2\ell + 4\Delta + 9\Phi$. Let $\mathcal{B}'$ be the first honest block after $\mathcal{B}$. $\mathcal{B}'$ is produced at round at most $r'' = r' + \ell + 2\Delta + 2\Phi \leq r - (\ell + 2\Delta + 7\Phi)$; otherwise, the chain becomes stale at round r'' . We consider the timestamp of $\mathcal{B}'$. Since $\mathcal{B}'$ is at least $\ell + 2\Delta + 7\Phi$ (nominal) rounds before all honest parties start to mine beacons (which happens when local clocks of all alert parties pass $\langle \text{itv1}, (\text{itv1} - 1) \cdot R + \ell + 2\Delta + 4\Phi \rangle$), $\mathcal{B}'$ cannot report a timestamp $\langle \text{itv1}, \cdot \rangle$. This is because

$\text{GOODSHIFT}(r-1)$ implies that the backward shift at the end of interval $\text{itvl} - 1$ is at most 2Φ ; $\text{GOODSKEW}(r-1)$ indicates that all alert parties will start to mine beacons at most Φ rounds after at least one of them enter $\langle \text{itvl}, (\text{itvl} - 1) \cdot R + \ell + 2\Delta + 4\Phi \rangle$ and parties will wait for $\ell + 2\Delta + 4\Phi$ round at the beginning of interval itvl . After summing them up we get $\ell + 2\Delta + 7\Phi$. Hence, $\mathcal{B}'$ must report a timestamp $\langle \text{itvl} - 1, \cdot \rangle$. This contradicts our assumption that $\mathcal{B}$ is the last block in $\text{itvl} - 1$. $\square$

Next, we prove $\text{GOODBEACONS}(r)$. This predicate implies that at the boundary of every synchronization interval, alert parties will use the same set of beacons to update their local clock; further, the majority of these beacons are produced and issued by alert parties.

Note that, when the sequence of rounds S and queries J are appropriately selected, random variables $D(S)$ and $A(J)$ directly map to the number of synchronization beacons that protocol participants can produce during the beacon mining and inclusion phase. We are interested in using a typical execution to lower-bound the success of alert parties and upper-bound the success of the adversary (i.e., lower-bound $D(S)$ and upper-bound $A(J)$ in Definition 7).

Lemma 11. $\text{GOODROUND}(r-1) \implies \text{GOODBEACONS}(r)$.

Proof. Consider an interval itvl and a chain $\mathcal{C}$ held by an alert party at local round $\langle \text{itvl}, \text{itvl} \cdot R \rangle$. Let d denote the sum of difficulties of the synchronization beacons recorded in blocks with timestamp round $I_{\text{sync}}(\text{itvl})$.

Since $K = \ell + 2\Delta + 4\Phi$ by Corollary 6, honest parties agree on the beacon mining and inclusion phase in $\mathcal{C}$. Hence, every honest party would have the same view of the beacon set they are going to use. Now we prove that the majority of these beacons are produced by alert parties.

Let u denote the first (nominal) round such that all alert parties are mining beacons w.r.t. interval itvl . Consider a set of consecutive nominal rounds $S = \{i : u \leq i \leq v\}$ where $v = u + R - 2(\ell + 2\Delta + 4\Phi) - \Phi$, where all the honest queries in S are doing 2-for-1 PoW w.r.t. interval itvl and hence contribute to the honest beacon set. Let $\mathcal{B}$ be the last block produced by honest parties before round v and denote its production time by w (in terms of the nominal time index). Since $\mathcal{C}$ will become stale if there is no honest block since w for $\ell + 2\Delta + 2\Phi$ rounds, we get that $w < v - (\ell + 2\Delta + 2\Phi)$.

Let $S_1 = \{i : u \leq i \leq w - (\Delta + \Phi)\}$ and $S_2 = \{i : u - (2\ell + 4\Delta + 9\Phi) \leq i \leq w + (\ell + 2\Delta + 2\Phi)\}$. S_1 is the time interval that honest success can contribute to the beacon set w.r.t. interval itvl ; and S_2 is for the adversary. The lower bound of S_1 is derived from the definition of u and the upper bound is because it will take up to $\Delta + \Phi$ rounds for all beacons to be diffused to and accepted by all alert parties. The lower bound of S_2 is acquired due to the unpredictability discussed in Lemma 10. Regarding the upper bound of S_2 , it is achieved by considering the first honest block $\mathcal{B}'$ after v , which is produced no later than $w' = w + \ell + 2\Delta + 2\Phi$ (otherwise it violates NOSTALECHAINS). The adversary can no longer include beacons to the mining and inclusion phase after w' as it can no longer revert $\mathcal{B}'$, so all the subsequent beacons produced after w' are invalid w.r.t. the current chain. Note that $|S_1| \geq R - (3\ell + 7\Delta + 12\Phi) \geq R - 3(\ell + 2\Delta + 7\Phi)$ and $|S_2 \setminus S_1| = 3\ell + 7\Delta + 12\Phi \leq 3(\ell + 2\Delta + 7\Phi)$.

Let J denote the adversarial queries associated with S_2 . In order to prove that alert parties can produce at least half of synchronization beacons, it suffices to show that

$$D(S_1) > d/2.$$

We first show that the number of RO queries alert parties can make during S_2 is at most 4ϵ more than those in S_1 . We have

$$h(S_2) \leq (1 + \frac{\gamma|S_2 \setminus S_1|}{|S_1|})h(S_1) \leq (1 + \frac{3\gamma(\ell + 2\Delta + 7\Phi)}{R - 3(\ell + 2\Delta + 7\Phi)})h(S_1) \leq (1 + \frac{3\epsilon}{1 - 3\epsilon/\gamma})h(S_1) < (1 + 4\epsilon)h(S_1).$$

The first inequality follows from Fact 1(b); the third one holds since $\ell + 2\Delta + 7\Phi \leq \epsilon R/\gamma$ by Condition (C1); the last inequality is a consequence of Condition (C2) ($\epsilon < 1/12$). Next,

$$\begin{aligned} D(S_1) &\geq (1 - \epsilon)ph(S_1) > (1 - 5\epsilon)ph(S_2) > \frac{1 - 5\epsilon}{2 - \delta}p[h(S_2) + |J|] \\ &> \frac{1 - 6\epsilon}{2 - \delta}[D(S_2) + A(J)] > \frac{1 - 6\epsilon}{2 - \delta}[D(S_1) + A(J)] \geq \frac{1}{2}[D(S_1) + A(J)] = \frac{d}{2}. \end{aligned}$$

The first inequality follows from typical execution (Definition 7(a)); the second one is achieved by substituting $h(S_1)$ with $h(S_2)$; the next inequality follows from the honest majority assumption; and the forth one is by applying Lemma 2(d); the last inequality holds due to Condition (C2) ($\delta \geq 12\epsilon$). $\square$

Given an honest-majority beacon set, we are ready to prove $\text{GOODSKEW}(r)$ and $\text{GOODSHIFT}(r)$. Regarding the proof of $\text{GOODSKEW}(r)$, note that we will consider both the synchronized parties and the newly joining parties. The proof approach is as follows. We first prove that for parties in the same interval, their local clock can deviate for up to Φ rounds (Lemma 12). Next, we show that the shift computed at the boundary of each interval satisfies our GOODSHIFT predicate (Lemma 13). Then, we argue that the 2Φ -drift holds for parties in different intervals (Lemma 14). For those newly joining parties, we also prove that they can learn a clock value that is Φ -round close to any of the alert parties (Lemma 15). Finally, combining Lemmas 12, 14 and 15, we obtain the desired clock skews.

Lemma 12. *For a typical execution in a (γ, s) -respecting environment, if all predicates in Definition 5 hold till $r - 1$, then at round r , the local time of alert parties in the same interval differs by at most Φ .*

Proof. For nominal rounds that all alert parties stay in interval $\text{itv}1 = 1$, their local clock can differ with each other for up to $\Phi_{\text{clock}} < \Phi$ rounds in that no clock synchronization happened and only the adversary can set a Φ_{clock} -bounded drift. Recall that alert parties only adjust their local clock (call SyncProcedure , see Protocol 12) when it enters the last round of current interval.

Now we consider a nominal round r such that at least one alert party enters the next interval $\text{itv}1 + 1$. We show that for those parties that have finished adjusting their clock, the logical time that they report can deviate from each other for up to Φ rounds.

According to Lemma 11, alert parties will agree on the same synchronization beacon set (denoted by $SB_{\text{itv}1}$) at the end of $\text{itv}1$. Fix a beacon $SB \in SB_{\text{itv}1}$. Consider two alert parties P_1 and P_2 that have entered $\text{itv}1 + 1$. We are going to show that the quantity

$$\mu(P_i, SB) \triangleq r_i + \text{Timestamp}(SB) - P_i.\text{arrivalTime}(SB)$$

will differ by at most Φ between any two alert parties P_1 and P_2 , where r_i refers to the time that party P_i reports at nominal time r if it does not call SyncProcedure at the end of $\text{itv}1$. More precisely, $\mu(P_i, SB)$ is the logical time that P_i will report at nominal round r ¹² if it adopts SB and computes the corresponding shift shift_i to update its clock (at the end of $\text{itv}1$).

The arrival time of SB bookkept by P_i can be represented by

$$P_i.\text{arrivalTime}(SB) = r_i - (r - r_{SB}) + \delta_{P_i, SB} + \varphi_{P_i, SB}.$$

¹²We note that while an alert party may pass several logical rounds in a nominal round, it does not hurt our argument here as long as the adversarial drift is bounded by an absolute value.

r_{SB} is the nominal round that SB is emitted to the network if SB is honest, and is the first nominal round such that at least one honest party receives SB if SB is adversarial. $\delta_{P_i, \text{SB}} \in [\Delta]$ is the time elapsed (counted by rounds in terms of the nominal time) for SB to be delivered to P_i . $\varphi_{P_i, \text{SB}} \in [-\Phi_{\text{clock}}, \Phi_{\text{clock}}]$ is the drift of P_i when SB was sent (note that for two alert parties P_1, P_2 we have $|\varphi_{P_1, \text{SB}} - \varphi_{P_2, \text{SB}}| \leq \Phi_{\text{clock}}$ by the clock drift assumption). By substituting we get

$$\mu(P_i, \text{SB}) = r_i + \text{shift}_i = \text{Timestamp}(\text{SB}) + r - r_{\text{SB}} - \delta_{P_i, \text{SB}} - \varphi_{P_i, \text{SB}}. \quad (7)$$

Note that for different parties at nominal round r , $\text{Timestamp}(\text{SB}) + r - r_{\text{SB}}$ is a constant. Hence $|\mu(P_1, \text{SB}) - \mu(P_2, \text{SB})| \leq \Phi$. We highlight this holds for adversarially generated SB in that after SB is delivered to at least one honest party (denoted by r_{SB}), it will be delivered to all honest parties within Δ rounds.

Consider two alert parties $P_1, P_2 \in \mathcal{P}_{\text{alert}}[r]$ with timestamp $\langle \text{itv1} + 1, \cdot \rangle$ (i.e., they have finished synchronization). Tuples $(\mu(P_1, \text{SB}))_{\text{SB} \in \mathcal{S}_{\text{itv1}}}$ and $(\mu(P_2, \text{SB}))_{\text{SB} \in \mathcal{S}_{\text{itv1}}}$ are all the potential times that they will report. These two tuples are of the same size, and $\forall \text{SB} \in \mathcal{S}_{\text{itv1}}, |\mu(P_1, \text{SB}) - \mu(P_2, \text{SB})| \leq \Phi$. Due to Fact 2 we get

$$\left| \text{med}\left((\mu(P_1, \text{SB}))_{\text{SB} \in \mathcal{S}_{\text{itv1}}}\right) - \text{med}\left((\mu(P_2, \text{SB}))_{\text{SB} \in \mathcal{S}_{\text{itv1}}}\right) \right| \leq \Phi.$$

Recalling Equation (3), $\text{med}(\mu(P_i, \text{SB}))_{\text{SB} \in \mathcal{S}_{\text{itv1}}} = r_i + \text{med}\{\text{Timestamp}(\text{SB}) - P_i.\text{arrivalTime}(\text{SB}) \mid \text{SB} \in \mathcal{S}_{\text{itv1}}\} = r_i + \text{shift}_{\text{itv1}}^{P_i}$ is the logical time that P_i reports after the synchronization. Therefore alert clocks that have already entered $\text{itv1} + 1$ will deviate from each other for up to Φ rounds.

Finally, in order to conclude the proof, it suffices to show that during nominal rounds in interval $\text{itv1} > 1$ that no adjustment happens, the difference of alert parties' local clocks will not deviate for more than Φ rounds. This always holds in that the difference of honest parties' drifts in the execution of *Timekeeper* is an absolute term (recall Φ_{clock} and the condition check in *CLOCK-FORWARD* as well as *CLOCK-BACKWARD* in $\mathcal{G}_{\text{ImpClock}}$), thus adversarial manipulation cannot set $\varphi_{P_i, \text{SB}}$ to a value that violates the upper bound of clock skews. $\square$

Lemma 13. $\text{GOODSKEW}(r - 1) \implies \text{GOODSHIFT}(r)$

Proof. For a party P that calls *SyncProcedure* at nominal round r , we prove that $-2\Phi < \text{shift} < \Phi$.

Let SB denote the median beacon P adopted, and shift the corresponding shift value. Since $\text{GOODBEACONS}(r - 1)$ holds, we will have two alert parties P_1 and P_2 who produce synchronization beacons SB_1 and SB_2 , respectively. We denote the shifts w.r.t. SB_1, SB_2 by $\text{shift}_1, \text{shift}_2$ (based on P 's local arrival timetable) and learn that $\text{shift}_1 \leq \text{shift} \leq \text{shift}_2$. This implies $\mu(P, \text{SB}_1) \leq \mu(P, \text{SB}) \leq \mu(P, \text{SB}_2)$ (recall the definition of $\mu(P, \text{SB})$ in Lemma 12). Further, we extract these quantities by Equation (7) in the following way. For $\mu(P, \text{SB})$, we consider it with respect to the shift value, and for the rest we substitute them in terms of the timestamp recorded in SB_i (i.e., the last expression in Equation (7)). We have

$$\text{Timestamp}(\text{SB}_1) - r_{\text{SB}_1} - \delta_{P, \text{SB}_1} - \varphi_{P, \text{SB}_1} \leq r_i - r + \text{shift} \leq \text{Timestamp}(\text{SB}_2) - r_{\text{SB}_2} - \delta_{P, \text{SB}_2} - \varphi_{P, \text{SB}_2}.$$

By letting $\delta_{P, \text{SB}_1} = \Delta$, $\varphi_{P, \text{SB}_1} = \Phi_{\text{clock}}$ and $\delta_{P, \text{SB}_2} = \varphi_{P, \text{SB}_2} = 0$, we get

$$\text{Timestamp}(\text{SB}_1) - r_{\text{SB}_1} - \Phi \leq r_i - r + \text{shift} \leq \text{Timestamp}(\text{SB}_2) - r_{\text{SB}_2}. \quad (8)$$

Note that no clock adjustment on P happened during nominal time $[\min\{r_{\text{SB}_1}, r_{\text{SB}_2}\}, r)$. Moreover, if the adversary does not set a drift and the beacon is delivered to all alert parties immediately

after it was mined, we will have $\mathbf{r}_i - r = \text{Timestamp}(\text{SB}_i) - r_{\text{SB}_i}$. Since the adversarial drift is upper bounded by the absolute term Φ_{clock} , and the network is Δ -bounded delay, we learn that the difference of $\text{Timestamp}(\text{SB}_i) - r_{\text{SB}_i}$ and $\mathbf{r}_i - r$ is bounded by $\Phi_{\text{clock}} + \Delta = \Phi$. Therefore, we get

$$|(\text{Timestamp}(\text{SB}_i) - r_{\text{SB}_i}) - (\mathbf{r}_i - r)| \leq \Phi. \quad (9)$$

After combining Equations (8) and (9), we get the desired range for **shift**, namely, $-2\Phi < \text{shift} < \Phi$. $\square$

Lemma 14. *For a typical execution in a (γ, s) -respecting environment, if all predicates in Definition 5 hold till $r - 1$, then at round r , the local time of alert parties in different interval differs by at most 2Φ .*

Proof. Consider two alert parties P_1, P_2 that P_1 finishes running **SyncProcedure** at nominal time r and P_2 does not. Note that before the synchronization, $\mathbf{r}_1 - \Phi \leq \mathbf{r}_2 < \mathbf{r}_1$ holds because they are alert and hence skew predicate satisfies (and, since P_1 runs **SyncProcedure** first, its local time is larger than P_2). Let $\mathbf{r}'_1 = \mathbf{r}_1 + \text{shift}_1$ denote the logical time that P_1 owns after synchronization, we have

$$|\mathbf{r}_2 - \mathbf{r}'_1| \leq 2\Phi$$

after combining $\mathbf{r}_1 - \Phi \leq \mathbf{r}_2 < \mathbf{r}_1$ and $-2\Phi < \text{shift}_1 < \Phi$ from Lemma 13. $\square$

Lemma 15. *For a typical execution in a (γ, s) -respecting environment, if all predicates in Definition 5 hold till $r - 1$, then for a newly joining party who concludes the join procedure at round r , the following properties hold.*

- (a) Let $\mathcal{C}_{\text{join}}$ denote the chain held by P_{join} at nominal round $r' \leq r$ and P_{join} is in Phase C or D; let $\mathcal{C}_{\text{alert}}$ denote a chain held by P_{alert} at the same nominal time. Then $\mathcal{C}_{\text{alert}}^{\text{K}} \preceq \mathcal{C}_{\text{join}}$.
- (b) The index value i^* set in Line 30 of its joining procedure **JoinProc** satisfies $i^* \geq 1$.
- (c) When the joining party becomes alert, it will report a logical clock that is at most Φ -rounds apart from any other alert party that is in the same interval.

Proof. Regarding claim(a), notice that the heaviest chain selection rule used in **SelectChain** (Protocol 10) does not involve the local time **localTime** of the party executing it. P_{join} and P_{alert} would make the same decision except that P_{join} may consider and adopt those chains in the **futureChains** set of P_{alert} . Therefore, it suffices to show that no chain with future timestamps and more accumulated difficulties than $\mathcal{C}_{\text{alert}}$ can have a fork from $\mathcal{C}_{\text{alert}}$ that is too earlier with respect to r' .

Assume at nominal round r' , P_{join} is in Phase C or D holding a chain $\mathcal{C}_{\text{join}}$ such that $\text{diff}(\mathcal{C}_{\text{join}}) > \text{diff}(\mathcal{C}_{\text{alert}})$ and $\mathcal{C}_{\text{alert}}^{\text{K}} \not\preceq \mathcal{C}_{\text{join}}$. Consider a *virtual execution* for party P_{alert} (cf. [BGK⁺18]). This is an artificial random experiment that consists of the execution of the protocol with an additional “virtual” party P_{virt} that participates from the beginning, and is always alert. P_{virt} does not query the random oracle thus is passive. Starting from the point of execution where P_{join} joins the system, P_{virt} advances exactly like P_{join} and receives the same messages in the same round and order as P_{join} . When P_{join} adopts a chain $\mathcal{C}_{\text{join}}$ that P_{virt} does not adopt at nominal round r' , since it contains more difficulties, we have $\mathcal{C}_{\text{join}} \in \mathcal{S}_{r'}$. By Lemma 5 we know that the creation time of $\text{head}(\mathcal{C}_{\text{join}} \cap \mathcal{C}_{\text{virt}})$ is larger than $r' - (\ell + 2\Delta + 2\Phi)$. With the similar argument in Corollary 6 (note that a chain received by P_{virt} will be considered by P_{alert} within $(\Delta + \Phi)$ nominal rounds) we get $\mathcal{C}_{\text{alert}}^{\text{K}} \preceq \mathcal{C}_{\text{join}}$.

Moving to claim(b), in order to prove $i^* \geq 1$, we show that P_{join} has observed at least one full synchronization interval that started at least t_{pre} rounds after the beginning of Phase C. Let $r_{\text{start}}^{(j)}$ denote the nominal time such that at least one alert party passes local round $\langle j, (j - 1) \cdot R + K \rangle$ (in

other words, enters the beacon mining and inclusion phase w.r.t. interval j). Since $\text{GOODSKEW}(r-1)$ and $\text{GOODSHIFT}(r-1)$ holds, we have

$$r_{\text{start}}^{(j+1)} - r_{\text{start}}^{(j)} \leq R + 3\Phi.$$

Let i^* denote the minimal i such that $r_{\text{start}}^{(i)} \geq t_{\text{join}} + t_{\text{off}} + t_{\text{pre}}$ where t_{join} is the nominal time such that P_{join} start to run `JoinProc`. In other words, $r_{\text{start}}^{(i^*)}$ occurs at least t_{pre} rounds after the beginning of P_{join} 's Phase C. We have

$$r_{\text{start}}^{(i^*)} \leq t_{\text{join}} + t_{\text{off}} + t_{\text{pre}} + R + 3\Phi \leq t_{\text{join}} + t_{\text{off}} + t_{\text{gather}} - R.$$

The first inequality comes from the upper bound on the distance of $r_{\text{start}}^{(j)}$ and $r_{\text{start}}^{(j+1)}$; the next inequality follows from the values of t_{gather} and t_{pre} .

Since the length of a beacon mining and inclusion phase is $R - 2K$, $r_{\text{start}}^{(i^*)}$ is at least R nominal rounds before the end of Phase C. On the other hand, it takes at most $\Delta + \Phi$ rounds to diffuse the beacons. P_{join} will record the arrival time of all beacons in interval i^* . Note that the length of t_{pre} and the upper bound on duration of pre-mining stage in Lemma 10 guarantees that the adversary cannot produce valid beacons w.r.t. interval i^* before the beginning of Phase C.

Finally, for claim(c), we show that every time P_{join} will adopt the same beacon set as P_{alert} to update its local clock, and then they will update to a local time that maintains a good skew. Recall that in claim(b) we show that $r_{\text{start}}^{(i^*)}$ is at least R nominal rounds before the end of Phase C, at the end of Phase C the blocks in beacon mining and inclusion phase w.r.t. interval i^* went into the settled blockchain. This also concludes that P_{join} will use the same beacon set as P_{alert} . Regarding $i \geq i^*$ they can be proved in the same way.

The rest of the proof follows that in Lemma 12. Note that in Equation (7), the new local time after adjustment is irrelevant to its previous local time, hence after the computation, the distance of P_{alert} and P_{join} 's local clocks will differ for at most Φ rounds. $\square$

Lemma 16. $\text{GOODSKEW}(r-1) \wedge \text{GOODBEACONS}(r-1) \implies \text{GOODSKEW}(r)$.

Proof. The proof directly follows from Lemmas 12, 14 and 15. $\square$

Theorem 17. *For a typical execution in a $(\gamma, M + 2(\ell + 2\Delta + 7\Phi))$ -respecting environment, if Condition (C1) and Condition (C2) are satisfied, then all predicates in Definition 5 hold.*

Finally, we are able to prove our promised goal:

Theorem 18. *Consider an execution of Timekeeper in a $(\gamma, M + 2(\ell + 2\Delta + 7\Phi))$ -respecting environment. If Conditions (C1) and (C2) are satisfied, then the protocol achieves clock synchronization (Definition 1) with parameter values*

$$\text{Skew} = 2\Phi, \quad \text{shiftLB} = 3\Phi/R, \quad \text{shiftUB} = 2\Phi/R,$$

except with probability negligibly small in κ and λ .

Proof. The proof of bounded skews and the value of Skew directly follow from Lemma 16. Regarding linear envelope, consider a nominal round $r > R$ and an alert party $P \in \mathcal{P}_{\text{alert}}[r]$. Consider a *virtual execution* for party P_{alert} (cf. [BGK⁺18]) with an additional party P_{virt} who keeps alert from the beginning of the execution. Intuitively, the largest local time of P_{virt} is achieved when P_{virt} keeps skipping Φ rounds at the beginning of each interval; the lowest local time will be reported when

P_{virt} always set backward his local time for 2Φ rounds instead. In other words, when the local clock of P_{virt} passes one interval, the nominal time elapsed is bounded by $R - \Phi$ and $R + 2\Phi$. Now, consider the local time r that P_{alert} will report at nominal round r . We have

$$r - 2\Phi \cdot \left\lceil \frac{r - R}{R + 2\Phi} \right\rceil \leq r \leq r + \Phi \cdot \left\lceil \frac{r - R}{R - \Phi} \right\rceil.$$

Note that there is no adjustment at the beginning of first interval. By dropping the ceil and rearranging we get

$$\frac{1}{1 + 3\Phi/R} \cdot r \leq r \leq \frac{1}{1 - 2\Phi/R} \cdot r.$$

We then conclude an alert party's local clock stays in the (U, L) -linear envelope with parameters $\text{shiftLB} = 3\Phi/R$ and $\text{shiftUB} = 2\Phi/R$. $\square$

Acknowledgements

Juan Garay's research has been supported in part by NSF grants no. 2001082 and 2055694. Yu Shen's research has been supported by Input Output (iohk.io) through their funding of the University of Edinburgh Blockchain Technology Lab.

References

- [ADD⁺19] Ittai Abraham, Srinivas Devadas, Danny Dolev, Kartik Nayak, and Ling Ren. Synchronous byzantine agreement with expected $O(1)$ rounds, expected $O(n^2)$ communication, and optimal resilience. In Ian Goldberg and Tyler Moore, editors, *FC 2019: 23rd International Conference on Financial Cryptography and Data Security*, volume 11598 of *Lecture Notes in Computer Science*, pages 320–334, Frigate Bay, St. Kitts and Nevis, February 18–22, 2019. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-32101-7_20.
- [Bah13] Lear Bahack. Theoretical Bitcoin attacks with less than half of the computational power (draft). Cryptology ePrint Archive, Report 2013/868, 2013. URL: <https://eprint.iacr.org/2013/868>.
- [BGK⁺18] Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Ouroboros genesis: Composable proof-of-stake blockchains with dynamic availability. In David Lie, Mohammad Mannan, Michael Backes, and XiaoFeng Wang, editors, *ACM CCS 2018: 25th Conference on Computer and Communications Security*, pages 913–930, Toronto, ON, Canada, October 15–19, 2018. ACM Press. <https://doi.org/10.1145/3243734.3243848>.
- [BGK⁺21] Christian Badertscher, Peter Gazi, Aggelos Kiayias, Alexander Russell, and Vassilis Zikas. Dynamic ad hoc clock synchronization. In Anne Canteaut and François-Xavier Standaert, editors, *Advances in Cryptology – EUROCRYPT 2021, Part III*, volume 12698 of *Lecture Notes in Computer Science*, pages 399–428, Zagreb, Croatia, October 17–21, 2021. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-77883-5_14.
- [BKT⁺19] Vivek Kumar Bagaria, Sreeram Kannan, David Tse, Giulia C. Fanti, and Pramod Viswanath. Prism: Deconstructing the blockchain to approach physical limits. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019: 26th Conference on Computer and Communications Security*, pages 585–602, London, UK, November 11–15, 2019. ACM Press. <https://doi.org/10.1145/3319535.3363213>.
- [BMTZ17] Christian Badertscher, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. Bitcoin as a transaction ledger: A composable treatment. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*,

- pages 324–356, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-63688-7_11.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd Annual Symposium on Foundations of Computer Science*, pages 136–145, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. <https://doi.org/10.1109/SFCS.2001.959888>.
 - [DHS86] Danny Dolev, Joseph Y Halpern, and Raymond H Strong. On the possibility and impossibility of achieving clock synchronization. *J. Comput. Syst. Sci.*, 32(2):230–250, apr 1986.
 - [DLS88] Cynthia Dwork, Nancy Lynch, and Larry Stockmeyer. Consensus in the presence of partial synchrony. *J. ACM*, 35(2):288–323, apr 1988. <https://doi.org/10.1145/42282.42283>.
 - [DP09] Devdatt P. Dubhashi and Alessandro Panconesi. *Concentration of Measure for the Analysis of Randomized Algorithms*. Cambridge University Press, 2009. <https://doi.org/10.1017/CB09780511581274>.
 - [GK20] Juan A. Garay and Aggelos Kiayias. SoK: A consensus taxonomy in the blockchain era. In Stanislaw Jarecki, editor, *Topics in Cryptology – CT-RSA 2020*, volume 12006 of *Lecture Notes in Computer Science*, pages 284–318, San Francisco, CA, USA, February 24–28, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-40186-3_13.
 - [GKL15] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol: Analysis and applications. In Elisabeth Oswald and Marc Fischlin, editors, *Advances in Cryptology – EUROCRYPT 2015, Part II*, volume 9057 of *Lecture Notes in Computer Science*, pages 281–310, Sofia, Bulgaria, April 26–30, 2015. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-662-46803-6_10.
 - [GKL17] Juan A. Garay, Aggelos Kiayias, and Nikos Leonardos. The bitcoin backbone protocol with chains of variable difficulty. In Jonathan Katz and Hovav Shacham, editors, *Advances in Cryptology – CRYPTO 2017, Part I*, volume 10401 of *Lecture Notes in Computer Science*, pages 291–323, Santa Barbara, CA, USA, August 20–24, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-63688-7_10.
 - [GKL20] Juan Garay, Aggelos Kiayias, and Nikos Leonardos. Full analysis of nakamoto consensus in bounded-delay networks. Cryptology ePrint Archive, Report 2020/277, 2020. URL: <https://eprint.iacr.org/2020/277>.
 - [GKO⁺20] Juan A. Garay, Aggelos Kiayias, Rafail M. Ostrovsky, Giorgos Panagiotakos, and Vassilis Zikas. Resource-restricted cryptography: Revisiting MPC bounds in the proof-of-work era. In Anne Canteaut and Yuval Ishai, editors, *Advances in Cryptology – EUROCRYPT 2020, Part II*, volume 12106 of *Lecture Notes in Computer Science*, pages 129–158, Zagreb, Croatia, May 10–14, 2020. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-030-45724-2_5.
 - [HSSD84] Joseph Y. Halpern, Barbara Simons, H. Raymond Strong, and Danny Dolev. Fault-tolerant clock synchronization. In Robert L. Probert, Nancy A. Lynch, and Nicola Santoro, editors, *3rd ACM Symposium Annual on Principles of Distributed Computing*, pages 89–102, Vancouver, BC, Canada, August 27–29, 1984. Association for Computing Machinery. <https://doi.org/10.1145/800222.806739>.
 - [KMTZ13] Jonathan Katz, Ueli Maurer, Björn Tackmann, and Vassilis Zikas. Universally composable synchronous computation. In Amit Sahai, editor, *TCC 2013: 10th Theory of Cryptography Conference*, volume 7785 of *Lecture Notes in Computer Science*, pages 477–498, Tokyo, Japan, March 3–6, 2013. Springer Berlin Heidelberg, Germany. https://doi.org/10.1007/978-3-642-36594-2_27.
 - [Lam78] Leslie Lamport. Time, clocks, and the ordering of events in a distributed system. *Commun. ACM*, 21(7):558–565, jul 1978. <https://doi.org/10.1145/359545.359563>.

- [LL22] Christoph Lenzen and Julian Loss. Optimal clock synchronization with signatures. In Alessia Milani and Philipp Woelfel, editors, *41st ACM Symposium Annual on Principles of Distributed Computing*, pages 440–449, Salerno, Italy, July 25–29, 2022. Association for Computing Machinery. <https://doi.org/10.1145/3519270.3538444>.
- [LMS84] Leslie Lamport and P. M. Melliar-Smith. Byzantine clock synchronization. In Robert L. Probert, Nancy A. Lynch, and Nicola Santoro, editors, *3rd ACM Symposium Annual on Principles of Distributed Computing*, pages 68–74, Vancouver, BC, Canada, August 27–29, 1984. Association for Computing Machinery. <https://doi.org/10.1145/800222.806737>.
- [Nak08] Satoshi Nakamoto. Bitcoin: A peer-to-peer electronic cash system, Dec 2008. URL: <https://bitcoin.org/bitcoin.pdf>.
- [PS17a] Rafael Pass and Elaine Shi. FruitChains: A fair blockchain. In Elad Michael Schiller and Alexander A. Schwarzmann, editors, *36th ACM Symposium Annual on Principles of Distributed Computing*, pages 315–324, Washington, DC, USA, July 25–27, 2017. Association for Computing Machinery. <https://doi.org/10.1145/3087801.3087809>.
- [PS17b] Rafael Pass and Elaine Shi. Rethinking large-scale consensus. In Boris Köpf and Steve Chong, editors, *CSF 2017: IEEE 30th Computer Security Foundations Symposium*, pages 115–129, Santa Barbara, CA, USA, August 21–25, 2017. IEEE Computer Society Press. <https://doi.org/10.1109/CSF.2017.37>.
- [PSs17] Rafael Pass, Lior Seeman, and abhi shelat. Analysis of the blockchain protocol in asynchronous networks. In Jean-Sébastien Coron and Jesper Buus Nielsen, editors, *Advances in Cryptology – EUROCRYPT 2017, Part II*, volume 10211 of *Lecture Notes in Computer Science*, pages 643–673, Paris, France, April 30 – May 4, 2017. Springer, Cham, Switzerland. https://doi.org/10.1007/978-3-319-56614-6_22.
- [ST87] T. K. Srikanth and Sam Toueg. Optimal clock synchronization. *J. ACM*, 34(3):626–645, jul 1987. <https://doi.org/10.1145/28869.28876>.
- [WL88] Jennifer Lundelius Welch and Nancy Lynch. A new fault-tolerant algorithm for clock synchronization. *Inf. Comput.*, 77(1):1–36, apr 1988. [https://doi.org/10.1016/0890-5401\(88\)90043-0](https://doi.org/10.1016/0890-5401(88)90043-0).

A Mathematical Facts

Fact 1 captures some facts (Especially, the relation between the total number of hash queries in S and the number of queries in a specific round) within a (γ, s) -respecting environment.

Fact 1. *Let U be a set of at most s consecutive rounds in a (γ, s) -respecting environment and $S \subseteq U$.*

(a) *For any $h \in \{h_r : r \in U\}$, $\frac{h}{\gamma} < \frac{h(S)}{|S|} \leq \gamma h$.*

(b) *$h(U) \leq (1 + \frac{\gamma|U \setminus S|}{|S|})h(S)$.*

(c) *$|S| \sum_{r \in S} (ph_r)^2 \leq \gamma(ph(S))^2$.*

Definition 8. [DP09, Definition 5.3] *A sequence of random variables $(X_0, X_1, \dots)$ is a martingale with respect to the sequence $(Y_0, Y_1, \dots)$, if, for all $n \geq 0$, X_n is determined by $Y_0, \dots, Y_n$ and $\mathbb{E}[X_{n+1} \mid Y_0, \dots, Y_n] = X_n$.*

Theorem 19. *Let $(X_0, X_1, \dots)$ be a martingale with respect to the sequence $(Y_0, Y_1, \dots)$. Suppose an event G implies*

$$\forall k \in \mathbb{N}^+, X_k - X_{k-1} \leq b \text{ and } V = \sum_k \text{Var}[X_k - X_{k-1} \mid Y_1, \dots, Y_{k-1}] \leq v.$$

Then, for non-negative n and t ,

$$\Pr[X_n \geq X_0 + t \wedge G] \leq \exp \left\{ -\frac{t^2}{2v + 2bt/3} \right\}.$$

Fact 2. Let $(a_i)_{i=1}^n$ and $(b_i)_{i=1}^n$ be two sequences of n integers each, with the property that $\forall i \in [n], |a_i - b_i| \leq \Phi$. Then we have $|\text{med}((a_i)_{i=1}^n) - \text{med}((b_i)_{i=1}^n)| \leq \Phi$.

B Model and Building Blocks (Cont'd)

Imperfect local clock functionality. Functionality $\mathcal{G}_{\text{ImpClock}}$ maintains an internal variable τ_{sid} (the nominal time) to record how many times the sub-procedure *Round-Update* resets the round status for suitable honest parties and other functionalities. This variable will never be revealed to neither honest parties nor registered functionalities, which captures the fact that clocks are local, and it is only accessible by $\mathcal{G}_{\text{ImpClock}}$, the environment $\mathcal{Z}$, the adversary $\mathcal{A}$ and the (wrapped) random oracle functionality $\mathcal{W}(\mathcal{F}_{\text{RO}})$, after they send a `CLOCK-READ` command. Notably, $\mathcal{W}(\mathcal{F}_{\text{RO}})$ has to know the exact nominal time in order to coordinate with the environment so as to apply restrictions on adversarial queries and model dynamic participation. (More details on the wrapped RO functionality and the notion of a respecting environment below.)

$\mathcal{G}_{\text{ImpClock}}$ also keeps track of the set of honest parties $\mathcal{P}$ and the registered functionalities F . For all honest parties and functionalities, $\mathcal{G}_{\text{ImpClock}}$ associates each of them with a bit variable $d \in \{0, 1\}$ which tracks if the corresponding party/functionality has claimed finishing its local round, and returns a response to command `CLOCK-TICK` to indicate whether the requester should update its local clock. Importantly, every honest party P is associated with a (possibly negative) integer b_P which indicates the “tick budget” allocated for P during the current nominal round. If no adversarial clock drift is set, at the end of a nominal round b_P is reset to 1 by *Round-Update* and consumed by the party when P sends `CLOCK-UPDATE` in the next nominal round. This captures the fact that, without adversarial manipulation, local clocks proceed with exactly the same speed. Additionally, a (possibly negative) integer `drift` is associated with P to record the total drift that is applied to its local clock.

An honest party P sends the `CLOCK-UPDATE` command to $\mathcal{G}_{\text{ImpClock}}$ to indicate conclusion of its current local round. The main purpose of this command is to set the variable d associated with P to 1 and consume one tick from its budget (i.e., $b_P \leftarrow b_P - 1$). When party P is activated, it sends `(CLOCK-TICK, sid)` to $\mathcal{G}_{\text{ImpClock}}$ to check if its corresponding indicator d has been reset to 0. If this happens, P updates its local clock and enters the next local round; otherwise, this implies that P need to wait for other participants.

Regarding the *Round-Update* procedure, $\mathcal{G}_{\text{ImpClock}}$ will update the nominal time only when (i) all honest parties and registered functionalities have claimed finishing the current round (i.e., the corresponding indicator $d = 1$), and (ii) all the tick budgets allocated to honest parties have been consumed (i.e., $b_P \leq 0$ for every party). When the party/functionality statuses pass the above checks, *Round-Update* moves the nominal time forward ($\tau_{\text{sid}} \leftarrow \tau_{\text{sid}} + 1$). Then, *Round-Update* issues a new tick budget for all honest parties (i.e., all budget variables b are increased by 1); afterwards, for all registered functionalities and honest parties whose b_P is positive, their d indicators are reset to 0.

Next, we elaborate on the “imperfect” aspect of the clock and on the adversarial manipulation of clock drifts. Specifically, in addition to *Round-Update*, we also allow the adversary to set some drifts to parties’ local clocks, which will accelerate or stall their local clock; such values are globally bounded by Φ_{clock} . This assumption allows local clocks to proceed at “roughly” the same speed.

Further, the adversary $\mathcal{A}$ can adaptively manipulate the drift of honest parties' clocks by sending CLOCK-FORWARD and CLOCK-BACKWARD¹³ after they conclude the current round. If $\mathcal{A}$ issues CLOCK-FORWARD for party P , it will enter a new local round before $\mathcal{G}_{\text{ImpClock}}$ updates the nominal time, and this can be repeated as long as P 's drift is not Φ_{clock} rounds larger than other honest parties. On the other hand, if $\mathcal{A}$ issues CLOCK-BACKWARD, it will set P 's budget to a negative value, thus preventing $\mathcal{G}_{\text{ImpClock}}$ from updating d_P at the end of the nominal round. I.e., P will still be in the same logical round during these two nominal rounds. Again, this process can be repeated by $\mathcal{A}$ as long as the drift on P is not Φ_{clock} rounds smaller than others. As a consequence, the targeted party's local clock may remain static for several nominal rounds.

Functionality $\mathcal{G}_{\text{ImpClock}}$

This functionality maintains state variables as follows.

State Variable	Description
$\mathcal{P} \leftarrow \emptyset$	The set of registered parties $P = (\text{pid}, \text{sid})$.
$F \leftarrow \emptyset$	The set of registered functionalities (together with their session identifier).
$\tau_{\text{sid}} \leftarrow 0$	The nominal-time variable for session sid .
$d_P \leftarrow 0$	The clock-update variable for $P = (\text{pid}, \text{sid}) \in \mathcal{P}$. d_P is set to 1 after P finishes a round.
$b_P \leftarrow 1$	The tick-budget variable for $P = (\text{pid}, \text{sid}) \in \mathcal{P}$.
$\text{drift}_P \leftarrow 0$	The accumulated additive drifts for $P = (\text{pid}, \text{sid}) \in \mathcal{P}$.
$d(\mathcal{F}, \text{sid}) \leftarrow 0$	The clock-update variable for $(\mathcal{F}, \text{sid}) \in F$.

Clock capabilities:

- Upon receiving (CLOCK-UPDATE, sid_C) from some party $P \in \mathcal{P}$ set $d_P \leftarrow 1$ and $b_P \leftarrow b_P - 1$; execute *Time-Update* and forward (CLOCK-UPDATE, sid_C, P) to $\mathcal{A}$.
- Upon receiving (CLOCK-UPDATE, sid_C) from some functionality $\mathcal{F}$ in a session sid such that $(\mathcal{F}, \text{sid}) \in F$ set $d(\mathcal{F}, \text{sid}) \leftarrow 1$, execute *Time-Update* and return (CLOCK-UPDATE, $\text{sid}_C, \mathcal{F}$) to this instance of $\mathcal{F}$.
- Upon receiving (CLOCK-FORWARD, $\text{sid}_C, P = (\text{pid}, \text{sid})$) from $\mathcal{A}$ where $P \in \mathcal{P}$, if $d_P = 0$ or $\text{drift}_P \geq \Phi_{\text{clock}}$ or $\text{drift}_P - \min_{P'=(\cdot, \text{sid})} \{\text{drift}_{P'}\} \geq \Phi_{\text{clock}}$, ignore the message. Otherwise, update $d_P = 0$, $b_P \leftarrow b_P + 1$ and $\text{drift}_P \leftarrow \text{drift}_P + 1$; return (CLOCK-FORWARD-OK, sid_C, P) to $\mathcal{A}$.
- Upon receiving (CLOCK-BACKWARD, sid_C, P) from $\mathcal{A}$ where $P \in \mathcal{P}$, if $d_P = 0$ or $\text{drift}_P \leq -\Phi_{\text{clock}}$ or $\max_{P'=(\cdot, \text{sid})} \{\text{drift}_{P'}\} - \text{drift}_P \geq \Phi_{\text{clock}}$, ignore the message. Otherwise, update $b_P \leftarrow b_P - 1$ and $\text{drift}_P \leftarrow \text{drift}_P - 1$; return (CLOCK-BACKWARD-OK, sid_C, P) to $\mathcal{A}$.
- Upon receiving (CLOCK-TICK, sid_C) from any participant P — including the environment on behalf of a party — or the adversary on behalf of a corrupted party P (resp. from any ideal—shared or local—functionality $\mathcal{F}$), execute procedure *Time-Update*, return (CLOCK-TICK, sid_C, d_P) (resp. (CLOCK-TICK, $\text{sid}_C, d(\mathcal{F}, \text{sid})$)) to the requestor (where sid is the session id of the calling instance).
- Upon receiving (CLOCK-READ, sid_C) from the adversary or the wrapper functionalities, return (CLOCK-READ, $\text{sid}_C, \tau_{\text{sid}}$) to the requestor (where sid is the session id of the calling instance).

¹³As such, our clock functionality is a more natural model of the real world compared to [BGK⁺21]'s, as it allows $\mathcal{A}$ to manipulate the clock in both directions, backward, and forward; in [BGK⁺21], only forward manipulation is allowed. Nonetheless, this does not result in a more powerful adversary.

Procedure Time-Update: For each session sid do: If (i) $d_{(\mathcal{F}, \text{sid})} = 1$ for all $\mathcal{F} \in F$, and (ii) $d_P = 1$ and $b_P \leq 0$ for all honest parties $P = (\cdot, \text{sid}) \in \mathcal{P}$, then do the following:

1. Update $\tau_{\text{sid}} \leftarrow \tau_{\text{sid}} + 1$.
2. Update $d_{(\mathcal{F}, \text{sid})} \leftarrow 0$ and $b_P \leftarrow b_P + 1$ for all parties $P = (\cdot, \text{sid}) \in \mathcal{P}$.
3. For all parties $P = (\cdot, \text{sid}) \in \mathcal{P}$ with $b_P > 0$, update $d_P \leftarrow 0$.

Functionality 1: *The imperfect local clock.*

Global random oracle functionality and its wrapper. By convention, we model parties' calls to the hash function used to generate proofs of work as assuming access to a random oracle; this is captured by the functionality $\mathcal{F}_{\text{RO}}$. $\mathcal{F}_{\text{RO}}$ internally maintains an updatable table H ; it is parameterized with security parameter κ as H 's output length. Upon receiving a query $(\text{EVAL}, \text{sid}, x)$, if $H(x) = \perp$ (i.e., no pair of the form $(x, \cdot)$ is in H), a value y is chosen uniformly at random from $\{0, 1\}^\kappa$ and returned to the party ($\mathcal{F}_{\text{RO}}$ also updates $H(x) = y$). If $H(x) \neq \perp$ (i.e., x has been queried before), the corresponding y is returned.

Functionality $\mathcal{F}_{\text{RO}}$

The functionality is parameterized by a security parameter κ .

State Variable	Description
$\mathcal{P} \leftarrow \emptyset$	The set of registered parties.
$H \leftarrow \emptyset$	A dynamically updatable function table where $H[x] = \perp$ denotes the fact that no pair of the form $(x, \cdot)$ is in H .

- **Eval.** Upon receiving $(\text{EVAL}, \text{sid}, x)$ from some party $P \in \mathcal{P}$ (or from $\mathcal{A}$ on behalf of a corrupted P), do the following:
 1. If $H[x] = \perp$ sample a value y uniformly at random from $\{0, 1\}^\kappa$ and set $H[x] \leftarrow y$.
 2. Return $(\text{EVAL}, \text{sid}, x, H[x])$ to the requestor.

Functionality 2: *The random oracle.*

Notice that with regards to bounding access to real-world resources, functionality $\mathcal{F}_{\text{RO}}$ as defined fails to limit the adversary on making a certain number of queries per round. Hence, we adopt a functionality wrapper [BMTZ17, GKO⁺20] $\mathcal{W}(\mathcal{F}_{\text{RO}})$ that wraps the corresponding resource to capture such restrictions. We highlight that our wrapper $\mathcal{W}(\mathcal{F}_{\text{RO}})$ improves on previous wrappers in two aspects, in order to provide a more natural model of the real world:

- We capture the pre-mining stage by letting the adversary query the RO with no restrictions before the CRS is released. More specifically, $\mathcal{W}(\mathcal{F}_{\text{RO}})$ is initialized with internal time counter $\tau = 0$, and the adversary can make as many queries as he wants (albeit polynomially bounded) and the wrapper simply forwards all these queries to $\mathcal{F}_{\text{RO}}$ as long as $\tau = 0$. This pre-mining stage ends when the time counter τ is set to 1, which happens immediately after $\mathcal{W}(\mathcal{F}_{\text{RO}})$ receives $(\text{RETRIEVED}, \text{sid})$ from the CRS functionality $\mathcal{F}_{\text{CRS}}$.

This pre-mining stage captures the fact that a hash function is usually available before the

protocol execution begins. Nonetheless, thanks to the CRS with sufficiently high entropy, all adversarial queries made during the pre-mining stage can benefit the adversary in the later execution only with negligible probability.

- The wrapper limits adversarial access per nominal round by bounding the total number of queries that $\mathcal{A}$ can make. After the CRS is released, $\mathcal{W}(\mathcal{F}_{\text{RO}})$ keeps track of the corrupted party set. Upon receiving queries from corrupted parties, $\mathcal{W}(\mathcal{F}_{\text{RO}})$ first checks if $\mathcal{G}_{\text{ImpClock}}$ advances the nominal time ($\mathcal{W}(\mathcal{F}_{\text{RO}})$ can directly access $\mathcal{G}_{\text{ImpClock}}$'s internal time counter). If $\mathcal{G}_{\text{ImpClock}}$ enters the next round, $\mathcal{W}(\mathcal{F}_{\text{RO}})$ updates its internal clock counter as well and resets the RO counter h_τ to 0. As long as $q_{\mathcal{A}} < h_\tau$, where h_τ is a predetermined value provided by the environment $\mathcal{Z}$ for nominal round τ , $\mathcal{W}(\mathcal{F}_{\text{RO}})$ forwards the queries to $\mathcal{F}_{\text{RO}}$. When $q_{\mathcal{A}} \geq h_\tau$, which implies that the adversary has consumed all its allowed budget, the wrapper stops interaction with the adversary.

This “migration” from the q -bounded adversarial model ([GKL15] and follow-ups) to bounding the total number of queries that $\mathcal{A}$ can make allows us to dispose of the “flat” computational model considered in previous works. We now define the computational power in terms of the number of RO queries per round, which makes it possible to further refine the notion of a “respecting environment” that is suited for imperfect local clocks.

Functionality $\mathcal{W}(\mathcal{F}_{\text{RO}})$

This functionality maintains state variables as follows.

State Variable	Description
$\mathcal{P} \leftarrow \emptyset$	The set of registered parties; the current set of corrupted parties is denoted by $\mathcal{P}'$.
$\tau \leftarrow 0$	The (real-time) clock tick counter.
h_τ	An upper bound which restricts the $\mathcal{F}$ -evaluations of all alert parties at time τ .
$q_{\mathcal{H}}, q_{\mathcal{A}} \leftarrow 0$	The alert/adversary evaluation counter.

Pre-mining attack handling (executed only if $\tau = 0$):

- Upon receiving (EVAL, sid, x) from $\mathcal{A}$ on behalf of a corrupted party $P \in \mathcal{P}'$, forward the request to $\mathcal{F}_{\text{RO}}$ and return to $\mathcal{A}$ whatever $\mathcal{F}_{\text{RO}}$ returns.
- Upon receiving (RETRIEVED, sid) from $\mathcal{F}_{\text{CRS}}$, set $\tau = 1$.

Relaying inputs to the random oracle:

- Upon receiving (EVAL, sid, x) from $\mathcal{A}$ on behalf of a corrupted party $P \in \mathcal{P}'$ or a de-synchronized party P , first execute *Round Reset*, then do the following.
 1. Set $q_{\mathcal{A}} \leftarrow q_{\mathcal{A}} + 1$.
 2. **If** $q_{\mathcal{A}} < h_\tau$ **then** forward the request to $\mathcal{F}_{\text{RO}}$ and return to $\mathcal{A}$ whatever $\mathcal{F}_{\text{RO}}$ returns.
- Upon receiving (EVAL, sid, x) from an alert party P , first execute *Round Reset*, then do the following.
 1. Set $q_{\mathcal{H}} \leftarrow q_{\mathcal{H}} + 1$.
 2. **If** $q_{\mathcal{H}} \leq h_\tau$ **then** forward the request to $\mathcal{F}_{\text{RO}}$ and return to P whatever $\mathcal{F}_{\text{RO}}$ returns.
 3. **If** $q_{\mathcal{H}} \geq h_\tau$ **then** send (CLOCK-UPDATE, sid_C) to $\mathcal{G}_{\text{ImpClock}}$.

Corruption handling:

- Upon receiving (CORRUPT, sid, P) from the adversary, set $\mathcal{P}' \leftarrow \mathcal{P}' \cup P$.

Procedure Round-Reset: Send (CLOCK-READ, sid_C) to $\mathcal{G}_{\text{ImpClock}}$ and receive (CLOCK-READ, sid_C, τ') from $\mathcal{G}_{\text{ImpClock}}$. If $|\tau - \tau'| > 0$, then do the following.

1. Set $q_{\mathcal{H}}, q_{\mathcal{A}} \leftarrow 0$ and $\tau \leftarrow \tau'$.
2. Send (NEXT-ROUND) to $\mathcal{A}$ and receive as response (NEXT-ROUND, $h_{\tau'}^*$). **IF** $(h_1, h_2, \dots, h_{\tau'}^*)$ is (γ, s) -respecting **THEN** set $h_{\tau'} = h_{\tau'}^*$; **ELSE** set $h_{\tau'} = h_{\tau'-1}$.

Functionality 3: *The random oracle wrapper.*

Common reference string. We model a public setup by the CRS functionality $\mathcal{F}_{\text{CRS}}$.

Functionality $\mathcal{F}_{\text{CRS}}$

The functionality is parameterized by a distribution $\mathcal{D}$.

- **Retrieve.** Upon receiving (RETRIEVE, sid) from some party P (or from $\mathcal{A}$ on behalf of a corrupted P), do the following:
 1. If activated for the first time, choose a value $d \leftarrow \mathcal{D}$, and send (RETRIEVED, sid) to $\mathcal{W}(\mathcal{F}_{\text{RO}})$.
 2. Return (RETRIEVE, d) to P .

Functionality 4: *The common reference string.*

Diffusion functionality. There are three types of messages that are exchanged in the network: blockchain messages (e.g., when a party mines a new block); regular messages (usually transactions), which are diffused to the network when received by the environment; and synchronization beacons—a special message that parties will use to perform the local time adjustment. For simplicity, we follow the convention that each type of messages is diffused by its own network. We denote the network functionality disseminating blockchain information by $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$; the one diffusing transactions by $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$; and the one circulating synchronization beacons by $\mathcal{F}_{\text{Diffuse}}^{\text{sync}}$, respectively. Parties will communicate with $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ and $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ to receive new chains and transactions in `FetchInformation`, and they will receive synchronization beacons by sending requests to $\mathcal{F}_{\text{Diffuse}}^{\text{sync}}$ in `ProcessBeacons`. Refer to Protocols 5 and 9 for further details.

Functionality $\mathcal{F}_{\text{Diffuse}}$

This functionality maintains state variables as follows.

State Variable	Description
$\Delta \leftarrow 0$	The maximum network latency.
$\mathcal{P} \leftarrow \emptyset$	The set of registered parties.
$\vec{M} \leftarrow []$	A dynamically updatable list of quadruples $(m, \text{mid}, D_{\text{mid}}, P)$ where D_{mid} denotes the fetch counter.

Setting the delay:

- Upon receiving (SET-DELAY, sid, d) from the adversary $\mathcal{A}$, **if** SET-DELAY has never been received

then set $\Delta = d$. Return (SET-DELAY, sid, ok) to $\mathcal{A}$.

Network capabilities:

- Upon receiving (DIFFUSE, sid, m) from some $P_s \in \mathcal{P}$, where $\mathcal{P} = \{P_1, \dots, P_n\}$ denotes the current party set, do:
 1. Choose n new unique message-IDs $\text{mid}_1, \dots, \text{mid}_n$.
 2. Initialize $2n$ new variables $D_{\text{mid}_1} := D_{\text{mid}_1}^{\text{MAX}} \dots := D_{\text{mid}_n} := D_{\text{mid}_n}^{\text{MAX}} := 1$ and a per message-delay $\Delta_{\text{mid}_i} = \Delta$ for $i \in [n]$.
 3. Set $\vec{M} := \vec{M} \parallel (m, \text{mid}_1, D_{\text{mid}_1}, P_1) \parallel \dots \parallel (m, \text{mid}_n, D_{\text{mid}_n}, P_n)$.
 4. Send (DIFFUSE, sid, $m, P_s, (P_1, \text{mid}_1), \dots, (P_n, \text{mid}_n)$) to the adversary.
- Upon receiving (FETCH, sid) from $P \in \mathcal{P}$, or from $\mathcal{A}$ on behalf of a corrupted party P :
 1. For all tuples $(m, \text{mid}, D_{\text{mid}}, P) \in \vec{M}$, set $D_{\text{mid}} := D_{\text{mid}} - 1$.
 2. Let $\vec{M}_0^P$ denote the subvector $\vec{M}$ including all tuples of the form $(m, \text{mid}, D_{\text{mid}}, P)$ with $D_{\text{mid}} \leq 0$ (in the same order as they appear in $\vec{M}$). Delete all entries in $\vec{M}_0^P$ from $\vec{M}$ and in case some $(m, \text{mid}, D_{\text{mid}}, P)$ is in $\vec{M}_0^P$, where P is honest, set $\Delta_{\text{mid}'} = \Delta$ for any $(m, \text{mid}', D_{\text{mid}'}, (\cdot, \text{sid}))$ in $\vec{M}$ and replace this record by $(m, \text{mid}', \min\{D_{\text{mid}'}, \Delta\}, P')$.
 3. Output $\vec{M}_0^P$ to P (if P is corrupted, send $\vec{M}_0^P$ to $\mathcal{A}$).

Additional adversarial capabilities:

- Upon receiving (DIFFUSE, sid, m) from some corrupted $P_s \in \mathcal{P}$ (or from $\mathcal{A}$ on behalf of P_s if corrupted), execute it the same way as an honest-sender diffuse, with the only difference that $\Delta_{\text{mid}_i} = \infty$.
- Upon receiving (DELAYS, sid, $(T_{\text{mid}_{i_1}}, \text{mid}_{i_1}), \dots, (T_{\text{mid}_{i_\ell}}, \text{mid}_{i_\ell})$) from the adversary do the following for each pair $(T_{\text{mid}_{i_j}}, \text{mid}_{i_j})$: if $D_{\text{mid}_{i_j}}^{\text{MAX}} + T_{\text{mid}_{i_j}} \leq \Delta_{\text{mid}_{i_j}}$ and mid_{i_j} is a message-ID of receiver $P = (\cdot, \text{sid})$ registered in the current $\vec{M}$, set $D_{\text{mid}_{i_j}} := D_{\text{mid}_{i_j}} + T_{\text{mid}_{i_j}}$ and set $D_{\text{mid}_{i_j}}^{\text{MAX}} := D_{\text{mid}_{i_j}}^{\text{MAX}} + T_{\text{mid}_{i_j}}$; otherwise, ignore this pair.
- Upon receiving (SWAP, sid, mid, mid') from the adversary, if mid and mid' are message-IDs registered in the current $\vec{M}$, then swap the triples $(m, \text{mid}, D_{\text{mid}}, (\cdot, \text{sid}))$ and $(m, \text{mid}', D_{\text{mid}'}, (\cdot, \text{sid}))$ in $\vec{M}$. Return (SWAP, sid) to the adversary.
- Upon receiving (GET-REG, sid) from $\mathcal{A}$, return the response (GET-REG, sid, $\mathcal{P}$) to $\mathcal{A}$.

Functionality 5: *The diffusion network.*

C The Timekeeper Protocol

In this section we give the full specification of the Timekeeper protocol. The description is presented in UC-like notation.

The main protocol instance. We first introduce the main Timekeeper protocol instance that dispatches to the relevant subprocesses.

Protocol Timekeeper($P, \text{sid}; \mathcal{G}_{\text{Ledger}}, \mathcal{G}_{\text{ImpClock}}, \mathcal{F}_{\text{RO}}, \mathcal{F}_{\text{Diffuse}}$)

Global Variables:

- Read-only: $R, M, t_{\text{off}}, t_{\text{gather}}, t_{\text{pre}}$
- Read-write: $\text{localTime}, \text{ep}, r, C_{\text{loc}}, T_{\text{P}}^{\text{ep}}, \text{isInit}, t_{\text{work}}, \text{buffer}, \text{futureChains}, \text{isSync}, \text{fetchCompleted}, \text{lastTimeAlert}, \text{arrivalTime}_{SB}(\cdot)$.

Registration / Deregistration:

- Upon receiving input (REGISTER, $\mathcal{R}$), where $\mathcal{R} \in \{\mathcal{G}_{\text{Ledger}}, \mathcal{G}_{\text{ImpClock}}\}$ execute protocol Registration-Timekeeper($P, \text{sid}, \text{Reg}, \mathcal{R}$).
- Upon receiving input (DE-REGISTER, $\mathcal{R}$), where $\mathcal{R} \in \{\mathcal{G}_{\text{Ledger}}, \mathcal{G}_{\text{ImpClock}}\}$ execute protocol Deregistration-Timekeeper($P, \text{sid}, \text{Reg}, \mathcal{R}$).
- Upon receiving input (IS-REGISTERED, sid) return (REGISTER, $\text{sid}, 1$) if the local registry **Reg** indicates that this party has successfully completed a registration with $\mathcal{R} = \mathcal{G}_{\text{Ledger}}$ (and did not de-register since then). Otherwise, return (REGISTER, $\text{sid}, 0$).

Interacting with the Ledger: Upon receiving a ledger-specific input $I \in \{(\text{SUBMIT}, \dots), (\text{READ}, \dots), (\text{MAINTAIN-LEDGER}, \dots)\}$ verify first that all resources are available. **If** not all resources are available, **then** ignore the input; **else** (i.e., the party is operational and time-aware) execute one of the following steps depending on the input I :

- **If** $I = (\text{SUBMIT}, \text{sid}, \text{tx})$ **then** set **buffer** $\leftarrow$ **buffer** $\parallel$ **tx**, and send (DIFFUSE, sid, tx) to $\mathcal{F}_{\text{Diffuse}}$.
- **If** $I = (\text{MAINTAIN-LEDGER}, \text{sid}, \text{minerID})$ **then** invoke protocol LedgerMaintenance(P, sid); **if** LedgerMaintenance halts **then** halt the protocol execution (all future input is ignored).
- **If** $I = (\text{READ}, \text{sid})$ **then** invoke protocol ReadState(P, sid).
- **If** $I = (\text{EXPORT-TIME}, \text{sid})$ **then** do the following: **If** **isSync** or **isInit** is false, then return (EXPORT-TIME, $\text{sid}, \perp$) to the caller. Otherwise, call UpdateTime(P, R) and return (EXPORT-TIME, $\text{sid}, \text{localTime}$) to the caller.

Handling calls to the shared setup:

- Upon receiving (CLOCK-TICK, sid_C), forward it to $\mathcal{G}_{\text{ImpClock}}$ and output $\mathcal{G}_{\text{ImpClock}}$'s response.
- Upon receiving (CLOCK-UPDATE, sid_C), record that a clock-update was received in the current round. **If** the party is registered to all its setups, then do nothing further. Otherwise, do the following operations *before concluding this round*:
 1. **If** this instance is currently time-aware but otherwise stalled or offline, then call UpdateTime(P, R) to update **localTime**. **If** the party has passed a synchronization interval, then set **isSync** $\leftarrow$ false.
 2. **If** this instance is only stalled but **isSync** = true, then additionally execute FetchInformation(P, sid), extract all new synchronization beacons from the fetched chains and record their arrival times and set **fetchCompleted** $\leftarrow$ true. Also, any unfinished interruptible execution of this round is marked as completed.
 3. Forward (CLOCK-UPDATE, sid_C) to $\mathcal{G}_{\text{ImpClock}}$ to finally conclude the round.

Protocol 1: *The main protocol instance of Timekeeper*

Registration and de-registration. In order to perform basic operations, a party P needs to register to all resources. Note that P is aware whether he is not synchronized not and will set the bit variable **isSync** correspondingly.

Protocol Registration-Timekeeper($P, \text{sid}, \text{reg}, \mathcal{G}$)

- 1: **if** $\mathcal{G} = \mathcal{G}_{\text{ImpClock}}$ **then** send (REGISTER, sid) to $\mathcal{G}$, set registration status to registered with $\mathcal{G}$, and output the value received by $\mathcal{G}$.
- 2: **if** $\mathcal{G} = \mathcal{G}_{\text{Ledger}}$ **then**
- 3: **if** the party is not registered with $\mathcal{G}_{\text{ImpClock}}$ or already registered with all setups **then**
- 4: ignore this input.
- 5: **else**

```

6:      Send (CLOCK-TICK, sidC) to  $\mathcal{G}_{\text{ImpClock}}$  and receive (CLOCK-TICK, sidC, tick).
7:      Send (REGISTER, sid) to  $\mathcal{F}_{\text{Diffuse}}$ .
8:      Set localTime :=  $\langle 1, 1 \rangle$  and isSync  $\leftarrow$  false.
9:      If this is the first registration invocation for this ITI, then set isInit  $\leftarrow$  false.
10:     Output (REGISTER, sid, P) once completing the registration with all above resources  $\mathcal{F}$ .
11:   end if
12: end if

```

Protocol 2: *Parties register on necessary resources to run Timekeeper.*

The deregistration process is an analogous action. Note that parties will record the last alert time which might be used to synchronize if it is only stalled.

Protocol Deregistration-Timekeeper(P, sid, reg, $\mathcal{G}$)

```

1: If the party is alert, set lastTimeAlert  $\leftarrow$  localTime.
2: if  $\mathcal{G} = \mathcal{G}_{\text{ImpClock}}$  then
3:   Set isSync  $\leftarrow$  false
4:   Send (DE-REGISTER, sid) to  $\mathcal{G}$  and set registration status as de-registered with  $\mathcal{G}$ 
5:   Output the value received by  $\mathcal{G}$ 
6: end if
7: if  $\mathcal{G} = \mathcal{G}_{\text{Ledger}}$  then
8:   Set isSync  $\leftarrow$  false
9:   Send (DE-REGISTER, sid) to  $\mathcal{F}_{\text{Diffuse}}$ , set its registration status as de-registered with  $\mathcal{F}_{\text{Diffuse}}$  and
   output (DE-REGISTER, sid, P).
10: end if

```

Protocol 3: *Parties de-register from corresponding resources in Timekeeper.*

Ledger Maintenance We group all the steps in the main ledger operation in LedgerMaintenance. Note that what a party might execute depends on its status.

Protocol LedgerMaintenance(P, sid)

```

▷ The following steps are executed in an (MAINTAIN-LEDGER, sid, minerID)-interruptible manner:
1: if isInit = false then
2:   Send (RETRIEVE, sid) to  $\mathcal{F}_{\text{CRS}}$  and receive (RETRIEVED, CRS) and set isInit  $\leftarrow$  true.
3: end if
4: if isSync and stalled before (and now up and running) then
5:   Call SimulateClockAdjustments(P, sid)
6: end if
7: if not isSync then Call JoinProc(P, sid)
   ▷ Normal operation when alert
8: Call FetchInformation(P, sid) and denote the output by  $(\mathcal{C}_1, \dots, \mathcal{C}_N), (\mathbf{tx}_1, \dots, \mathbf{tx}_k)$ 
9: Set buffer  $\leftarrow$  buffer  $\parallel (\mathbf{tx}_1, \dots, \mathbf{tx}_k)$  and define futureChains  $\leftarrow$  futureChains  $\parallel (\mathcal{C}_1, \dots, \mathcal{C}_N)$ 
10: Call UpdateTime(P, sid)
   ▷ Ensures the processing of new beacons arrived in chains only.

```

```

11: Extract beacons  $B \leftarrow \{SB_1, \dots, SB_n\}$  contained in  $\mathcal{C}_1, \dots, \mathcal{C}_N$  and not yet contained in
    syncBuffer.
12: Call ProcessBeacons( $P, \text{sid}, B$ )
13: Let  $\mathcal{N}_0$  be the subsequence of futureChains s.t.  $\mathcal{C} \in \mathcal{N}_0 :\Leftrightarrow \forall \mathcal{B} \in \mathcal{C} : \text{Timestamp}(\mathcal{B}) \leq \text{localTime}$ 
14: Remove each  $\mathcal{C} \in \mathcal{N}_0$  from futureChains.
15: fetchCompleted  $\leftarrow$  true
16: Call SelectChain( $P, \text{sid}, \mathcal{C}_{\text{loc}}, \mathcal{N}_0, R$ ) to update  $\mathcal{C}_{\text{loc}}$ 
17: if  $t_{\text{work}} < \text{localTime}$  then
18:   Call UpdateMiningTarget( $P, M$ ) to update  $T_{\text{ep}}^{\mathcal{C}}$ 
19:   Call UpdateFreshRandomness( $P, R$ ) to update  $\eta_{\text{itvl}}$ 
20:   Call MiningProcedure( $P, \text{ep}, r, \text{buffer}, \mathcal{C}_{\text{loc}}$ )
21:   Set  $t_{\text{work}} \leftarrow \text{localTime}$ 
22:   if  $r = \text{itvl} \cdot R$  then Call SyncProcedure( $P, \text{sid}, R$ )
23: end if
24: Call FinishRound( $P$ )

```

◁ Mark normal round actions as finished.

Protocol 4: *The main operations for parties to maintain the ledger.*

Chain verification. A core procedure is to distinguish valid from invalid chains. The procedure is depicted below. It adopts the verification rule of the main blockchain structure similar to that in Bitcoin (cf. [GKL17, GKL20]), and extends it with beacon verifications.

Recall that $I_{\text{sync}}(\text{itvl})$ defined in Equation (2) is a useful function which extracts the valid timestamp set in beacon mining and inclusion phase w.r.t. interval itvl . In addition, isvalidate is a predicate that checks whether transactions in the blockchain achieves a valid ledger state (for details, see [BMTZ17]).

Algorithm IsValidChain($\mathcal{C}$)

```

1: if  $\mathcal{C}$  contains empty intervals or starts with a block with hash reference other than CRS, or
   isvalidstate( $\vec{\text{st}}) = 0$  then return false
2: if isSync and  $(\exists \mathcal{B} \in \mathcal{C} : \text{Timestamp}(\mathcal{B}) > \text{localTime})$  then return false
3: for each interval  $\text{itvl}'$  do
4:    $\text{ep}' \leftarrow \text{TargetRecalcEpoch}(\langle \text{itvl}', \cdot \rangle)$ 
     ▷ Derive target and randomness for interval  $\text{itvl}'$ 
5:   Set  $T_{\text{ep}'}^{\mathcal{C}}$  to be the target for epoch  $\text{ep}'$  in  $\mathcal{C}$ .
6:   Set  $\eta_{\text{itvl}'} \leftarrow G(\eta_{\text{itvl}'-1} \parallel \text{itvl}' \parallel v)$  where  $v$  is the concatenation of all block hash in interval
      $\text{itvl}' - 1$ , and  $\eta_1 \triangleq \text{CRS}$ .
7:   for each block  $\mathcal{B}$  in  $\mathcal{C}$  from interval  $\text{itvl}'$  do
     ▷ check 2-for-1 PoW
8:     Parse  $\mathcal{B}$  as  $\langle \text{ctr}, h, \text{st}, \langle \text{itvl}', r' \rangle, \text{txLabel} \rangle$ .
     ▷ Check hash
9:     Set badhash  $\leftarrow (h \neq H(\mathcal{B}^{-1}))$ , where  $\mathcal{B}^{-1}$  is the last block in  $\mathcal{C}$  before  $\mathcal{B}$ .
     ▷ Check nonce
10:    Set badnonce  $\leftarrow (H(\text{ctr}, h, \text{st}, \langle \text{itvl}', r' \rangle, \text{txLabel}) < T_{\text{ep}'}^{\mathcal{C}}) \wedge (\text{ctr} < 2^{32})$ 
     ▷ Check beacons
11:    if  $\exists \text{SB} \in \mathcal{B}$  and  $\text{Timestamp}(\text{SB}) \notin I_{\text{sync}}(\text{itvl}')$  then
12:      Set badBeacon  $\leftarrow$  true
13:    else if  $\exists \text{SB} \in \mathcal{B} : \text{Timestamp}(\text{SB}) > \text{Timestamp}(\mathcal{B})$  then
14:      Set badBeacon  $\leftarrow$  true

```

```

15:   else
16:     Parse SB as  $\langle \langle \text{itv1}', \mathbf{r}' \rangle, \mathbf{P}', \eta_{\text{SB}}, \text{ctr}, \text{blockLabel} \rangle$ 
17:     if  $\mathcal{C}$  contains more than one beacon with  $(\mathbf{r}', \mathbf{P}', \cdot)$  then set  $\text{badBeacon} \leftarrow \text{true}$ 
18:     if  $\eta_{\text{SB}} \neq \eta_{\text{itv1}'}$  or  $\mathbf{r}' \notin I_{\text{sync}}(\text{itv1}')$  then set  $\text{badBeacon} \leftarrow \text{true}$ 
19:      $u \leftarrow H(\text{ctr}, \mathbf{r}', \text{blockLabel}, \langle \text{itv1}', \mathbf{r}' \rangle, \mathbf{P}', \eta_{\text{itv1}'})$ 
20:     Set  $\text{badBeacon} \leftarrow ([u]^{\text{R}} < T_{\text{ep}'}^{\mathcal{C}}) \wedge (\text{ctr}' < 2^{32})$ 
21:   end if
22:   if  $(\text{badhash} \vee \text{badnonce} \vee \text{badBeacon})$  then return false
23: end for
24: end for
25: return true

```

Algorithm 1: *The chain validation procedure.*

The beacon validity predicate. Beacons validity is related to chain validity as one need the corresponding target as well as fresh randomness to check beacon validity. The details can be found below.

Algorithm ValidSB(SB, $\mathcal{C}$)

```

  ▷ Precondition: Chain  $\mathcal{C}$  is valid. Returns true if the beacon is a valid beacon w.r.t.  $\mathcal{C}$ , undecided
  if no judgement is possible, and false if the beacon is invalid w.r.t.  $\mathcal{C}$ .
1: Parse SB as  $\langle \langle \text{itv1}', \mathbf{r}' \rangle, \mathbf{P}', \eta_{\text{SB}}, \text{ctr}, \text{blockLabel} \rangle$ 
2: if  $\mathcal{C}$  contains no block in interval  $\text{itv1}'$  then
3:   return undecided                                     ◁ no judgement possible for this beacon
4: end if
  ▷ Derive target and randomness for interval  $\text{itv1}'$  as indicated by  $\mathcal{C}$ 
5:  $\text{ep}' \leftarrow \text{TargetRecalcEpoch}(\langle \text{itv1}', \mathbf{r}' \rangle)$ 
6: Set  $T_{\text{ep}'}^{\mathcal{C}}$  to be the target for epoch  $\text{ep}'$  in  $\mathcal{C}$ .
7: Set  $\eta_{\text{itv1}'} \leftarrow G(\eta_{\text{itv1}'-1} \parallel \text{itv1}' \parallel v)$  where  $v$  is the concatenation of all block hash in interval
    $\text{itv1}' - 1$ , and  $\eta_1 \triangleq \text{CRS}$ .
  ▷ Check nonce value and freshness
8:  $u \leftarrow H(\text{ctr}, \mathbf{r}', \text{blockLabel}, \langle \text{itv1}', \mathbf{r}' \rangle, \mathbf{P}', \eta_{\text{itv1}'})$ 
9: if  $([u]^{\text{R}} < T_{\text{ep}'}^{\mathcal{C}}) \wedge (\text{ctr}' < 2^{32}) \wedge (\eta_{\text{itv1}'} = \eta_{\text{SB}})$  then return true
10: return false

```

Algorithm 2: *The beacon validation procedure.*

Fetch information. Parties fetch information from two diffusion network — $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ and $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ — to learn new chains and transactions.

Protocol FetchInformation(P, sid)

```

1: if fetchCompleted then return
  ▷ Fetch on  $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ 
2: Send (FETCH, sid) to  $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ ; denote the response from  $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$  by (FETCH, sid, b).

```

```

3: Extract chains  $\mathcal{C}_1, \dots, \mathcal{C}_k$  from  $b$ .
   ▷ Fetch on  $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ 
4: Send (FETCH, sid) to  $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$ ; denote the response from  $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$  by (FETCH, sid,  $b$ ).
5: Extract received transactions  $\text{tx}_1, \dots, \text{tx}_k$  from  $b$ .
6: if not isSync or  $P$  is stalled then
7:    $\text{buffer} \leftarrow \text{buffer} \parallel (\text{tx}_1, \dots, \text{tx}_n)$ 
8:    $\text{futureChains} \leftarrow \text{futureChains} \cup \{\mathcal{C}_1, \dots, \mathcal{C}_n\}$ 
9: end if
OUTPUT: The protocol outputs  $(\mathcal{C}_1, \dots, \mathcal{C}_k)$  and  $(\text{tx}_1, \dots, \text{tx}_k)$  to its caller (but not to  $\mathcal{Z}$ ).

```

Protocol 5: *Fetching new blocks and transactions from $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ and $\mathcal{F}_{\text{Diffuse}}^{\text{tx}}$.*

Update mining target, fresh randomness and local time. Following Equation (5) and the rule regarding dampening filter τ , the targets for each epoch ep are computed as following. Recall that EpochBlockCount defined in Equation (4) outputs the number of blocks in the corresponding epoch, and T_0 is a pre-determined target value (hardcoded in the protocol) that guarantees a “safe” start in our assumption.

Protocol UpdateMiningTarget(P , sid)

```

1:  $\text{ep} \leftarrow \text{TargetRecalcEpoch}(\text{localTime})$ 
2: if  $\text{ep} = 1$  then
3:    $T_{\text{ep}} \leftarrow T_0$ 
4: else
5:    $\Lambda = |\text{EpochBlocks}(\mathcal{C}_{\text{loc}}, \text{ep} - 1)|$ 
6:    $\Lambda = \min\{\max\{\Lambda, \Lambda_{\text{epoch}}/\tau\}, \Lambda_{\text{epoch}} \cdot \tau\}$ 
7:    $T_{\text{ep}} \leftarrow (\Lambda_{\text{epoch}}/\Lambda) \cdot T_{\text{ep}-M/R}$ 
8: end if
OUTPUT: The protocol outputs  $T_{\text{ep}}$  to its caller (but not to  $\mathcal{Z}$ ).

```

Protocol 6: *Updating mining target based on local time and chain.*

Parties will extract the fresh randomness in every synchronization interval from the block hash in previous epoch. Note that $G(\cdot)$ is another hash function different from $H(\cdot)$ therefore it does not interact with the random oracle $\mathcal{F}_{\text{RO}}$ in the mining procedure.

Protocol UpdateFreshRandomness(P , sid)

```

1: Set  $\eta_{\text{itv}l} \leftarrow G(\eta_{\text{itv}l-1} \parallel \text{itv}l \parallel v)$  where  $v$  is the concatenation of all block hash in interval  $\text{itv}l - 1$ ,
   and  $\eta_1 \triangleq \text{CRS}$ .
OUTPUT: The protocol outputs  $\eta_{\text{itv}l}$  to its caller (but not to  $\mathcal{Z}$ ).

```

Protocol 7: *Updating mining target based on local chain*

Parties will send CLOCK-TICK to $\mathcal{G}_{\text{ImpClock}}$ to check if it receives a tick = 0, which indicates the

beginning of a new (local) round. Note that alert parties will never change the interval index `itv1` here when adding 1 to `localTime` (regarding the rules of addition, see Section 3.1); they will only adjust `itv1` in `SyncProcedure` (code in Protocol 12). Meanwhile, for those stalled parties, their local time will increase as if no adjustment happens.

Protocol UpdateTime(P, R)

▷ Precondition: Only executed if time-aware.

```

1: Send (CLOCK-TICK, sidC) to  $\mathcal{G}_{\text{ImpClock}}$  and receive (CLOCK-TICK, sidC, tick)
2: if tick = 0 then
3:   localTime  $\leftarrow$  localTime + 1
4:   fetchCompleted  $\leftarrow$  false
5: end if
6: ep  $\leftarrow$  TargetRecalcEpoch(localTime)
OUTPUT: The protocol outputs localTime, ep to its caller (but not to  $\mathcal{Z}$ ).

```

Protocol 8: Updating local time from $\mathcal{G}_{\text{ImpClock}}$.

Process beacons and arrival times. The following procedure processes incoming beacons, bookkeeps their arrival times and filters out invalid as well as duplicate beacons. The predicate to verify beacons is presented in Algorithm 2. Regarding the duplicate beacons recording the same timestamp and miner identity, only one with the earliest arrival time will be preserved.

Protocol ProcessBeacons(P, sid, B)

```

1: if fetchCompleted then return
2: Send (FETCH, sid) to  $\mathcal{F}_{\text{Diffuse}}^{\text{sync}}$ . denote the response from  $\mathcal{F}_{\text{Diffuse}}^{\text{sync}}$  by (FETCH, sid, b)
3: Extract all received beacons (SB1, ..., SBk) contained in  $b \cup B$ .
4: for each SBi with arrivalTimeSB(SBi) =  $\perp$  do
5:   syncBuffer  $\leftarrow$  syncBuffer  $\cup$  {SBi}
6:   Let itv1' be the interval index Timestamp(SBi) belongs to
7:   if isSync  $\wedge$  (itv1  $\geq$  itv1') then
8:     Set arrivalTimeSB(SBi)  $\leftarrow$  (localTime, final) ◁ The measurement is final.
9:   else ◁ Will be adjusted upon next time shift.
10:    arrivalTimeSB(SBi)  $\leftarrow$  (localTime, temp)
11:   end if
12: end for
    ▷ Buffer cleaning. Keep one representative arrival time.
13: if isSync then
14:   Remove from syncBuffer all beacons s.t. ValidSB(P, sid, SB, Cloc, R) returns false
15:   syncBuffervalid  $\leftarrow$  {SB'  $\in$  syncBuffer | ValidSB(P, sid, SB', Cloc, R) = true}
16:   Let L = (SB1, ..., SBn) be a canonical ordering of syncBuffervalid
17:   for each SB =  $\langle \langle \text{itv1}', r' \rangle, P', \eta_{\text{SB}}, \text{ctr}, \text{blockLabel} \rangle \in L$  do
18:     QSB  $\leftarrow$  {SB' =  $\langle \langle \text{itv1}'', r'' \rangle, P'', \cdot, \cdot, \cdot \rangle \in L$  | P' = P''  $\wedge$   $\langle \text{itv1}', r' \rangle = \langle \text{itv1}'', r'' \rangle$ }
19:     minSB  $\leftarrow$  min{arrivalTime(SB') | SB'  $\in$  QSB}
20:     SB'  $\leftarrow$  min{SB''  $\in$  QSB | arrivalTime(SB'') = minSB} ◁ Min w.r.t. ordering in L
21:     Remove from syncBuffer all beacons  $\langle \langle \text{itv1}', r' \rangle, P', \cdot, \cdot, \cdot \rangle$  except SB'
22:   end for
23: end if

```


OUTPUT: ok to its caller (but not to $\mathcal{Z}$).

Protocol 9: *Parties filter invalid synchronization beacons.*

Chain selection. Parties drop invalid chains, and then select the a chain by *heaviest difficulty* chain selection rule (cf. `maxvalid` in [GKL17]). More specifically, $\max(\mathcal{C}_1, \mathcal{C}_2)$ will return the *most difficult* of the two. In case $\text{diff}(\mathcal{C}_1) = \text{diff}(\mathcal{C}_2)$, $\max(\cdot, \cdot)$ always return the first operand to reflect the fact that parties adopt the first chain they obtain from the network.

Protocol SelectChain(P, sid, $\mathcal{N} = \{\mathcal{C}_1, \dots, \mathcal{C}_n\}$)

```

1: Set  $\mathcal{C}_{\max} \leftarrow \mathcal{C}_{\text{loc}}$ 
2: for  $i = 1$  to  $n$  do
3:   if IsValidChain( $\mathcal{C}_i$ ) returns true then  $\mathcal{C}_{\max} \leftarrow \max(\mathcal{C}_{\max}, \mathcal{C}_i)$ 
4: end for
5: Replace  $\mathcal{C}_{\text{loc}}$  by  $\mathcal{C}_{\max}$ .
OUTPUT: The protocol outputs  $\mathcal{C}_{\max}$  to its caller (but not to  $\mathcal{Z}$ ).
```

Protocol 10: *Parties filter invalid chains and apply the heaviest chain selection rule.*

Mining Procedure Once a party P has prepared all information and updated its state, it can run the core mining procedure formally given below. When `localTime` reports a timestamp that satisfies $\mathbf{r} \in I_{\text{sync}}(\text{itv1})$ (i.e., P stays in the beacon mining and inclusion phase), P will include the fresh beacons and check if he succeeds in the beacon mining procedure.

Note that, in the “prepare block content” part, we follow [BMTZ17] and adopt several predicates such as `blockifyOC` and `validTXOC` to generate the Merkle root (`st`) of the block content. `blockifyOC` is used to compute the root value, and `validTXOC` can justify whether an incoming transaction is valid w.r.t. the current blockchain state. This part is not relevant to the topic of this paper; for further discussion, we refer to [BMTZ17].

Protocol MiningProcedure(P, sid)

```

▷ The following steps are executed in an (MAINTAIN-LEDGER, sid, minerID)-interruptible manner:
1: if  $h \neq H(\text{head}(\mathcal{C}_{\text{loc}}))$  then  $h \leftarrow H(\text{head}(\mathcal{C}_{\text{loc}}))$  ◁ Check if switch to a new chain.
   ▷ Prepare block content.
2: Set  $\text{buffer}' \leftarrow \text{buffer}$ ,  $\mathbf{N} \leftarrow \text{tx}_P^{\text{base-tx}}$ , and  $\text{st} \leftarrow \text{blockify}_{\text{OC}}(\mathbf{N})$ .
3: repeat
4:   Parse  $\text{buffer}'$  as sequence  $(\text{tx}_1, \dots, \text{tx}_n)$ 
5:   for  $i = 1$  to  $n$  do
6:     if validTXOC( $\text{tx}_i, \vec{\text{st}} \parallel \text{st}$ ) = 1 then
7:        $\mathbf{N} \leftarrow \mathbf{N} \parallel \text{tx}_i$ 
8:       Remove  $\text{tx}'$  from  $\text{buffer}'$ 
9:       Set  $\text{st} \leftarrow \text{blockify}_{\text{OC}}(\mathbf{N})$ 
10:    end if
11:  end for
```

```

12: until  $N$  does not increase any more
    ▷ Check if a beacon should be included.
13: if  $I_{\text{sync}}(\text{localTime}) = \text{true}$  then
14:    $B \leftarrow \{SB' \in \text{syncBuffer} \mid \text{ValidSB}(P, \text{sid}, SB', \mathcal{C}_{\text{loc}}, R) = \text{true}\}$ 
15:   Remove from  $B$  all beacons  $SB = \langle \langle \text{itvl}', r' \rangle, P', \cdot, \cdot, \cdot \rangle$  that satisfy  $(\text{Timestamp}(SB) > \text{localTime}) \vee (I_{\text{sync}}(\text{Timestamp}(SB)) = \text{true}) \vee \mathcal{C}_{\text{loc}}$  contains a beacon  $\langle \langle \text{itvl}', r' \rangle, P', \cdot, \cdot, \cdot \rangle$ .
16:    $N \leftarrow N \parallel B$ 
17:   Set  $\text{st} \leftarrow \text{blockify}_{\text{OC}}(N)$ 
18: end if
    ▷ prepare 2-for-1 PoW.
19:  $\text{blockLabel} \leftarrow h \parallel \text{st}, \text{txLabel} \leftarrow P \parallel \eta_{\text{itvl}}$ 
20:  $u \leftarrow H(\text{ctr}, h, \text{st}, \text{localTime}, P, \eta_{\text{itvl}})$ 
    ▷ Check if block mining succeed.
21: if  $(u < T_{\text{ep}}^{\text{C}}) \wedge (\text{ctr} < 2^{32})$  then
22:   Set  $\mathcal{B} \leftarrow \langle h, \text{st}, \text{localTime}, \text{ctr}, \text{txLabel} \rangle$  and update  $\mathcal{C}_{\text{loc}} \leftarrow \mathcal{C}_{\text{loc}} \parallel \mathcal{B}$ 
23:   Send (DIFFUSE,  $\text{sid}, \mathcal{C}_{\text{loc}}$ ) to  $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$ 
24:   Set anchor and proceed from here upon next activation of this procedure.
25: end if
    ▷ Check if a PoW timestamp transaction mining succeed.
26: if  $([u]^R < T_{\text{ep}}^{\text{C}}) \wedge (\text{ctr} < 2^{32}) \wedge (I_{\text{sync}}(\text{localTime}) = \text{true})$  then
27:    $SB \leftarrow \langle \text{localTime}, P, \text{ctr}, \eta_{\text{itvl}}, \text{blockLabel} \rangle$ 
28:   Send (DIFFUSE,  $\text{sid}, SB$ ) to  $\mathcal{F}_{\text{Diffuse}}^{\text{sync}}$ 
29:   Set anchor at the end of the procedure to resume on next maintenance activation.
30: end if
31:  $\text{ctr} \leftarrow \text{ctr} + 1$ 

```

Protocol 11: *The mining procedure of Timekeeper.*

Synchronization Procedure The synchronization procedure is called when party's local clock enters a clock synchronization interval boundary (i.e., $\text{localTime} = \langle \text{itvl}, \text{itvl} \cdot R \rangle$). Note that thanks to the timestamp scheme in Timekeeper, parties will only pass the interval boundary for once.

Protocol SyncProcedure(P, sid)

```

    ▷ Only called when:  $P$  is alert,  $\text{localTime} = \langle \text{itvl}, \text{itvl} \cdot R \rangle$  and  $\text{itvl} > 0$ 
1:  $B \leftarrow \{ \mathcal{B} \mid (\mathcal{B} \in \mathcal{C}_{\text{loc}}) \wedge (\text{Timestamp}(\mathcal{B}) = \langle \text{itvl}, \cdot \rangle) \wedge (I_{\text{sync}}(\text{Timestamp}(\mathcal{B})) = \text{true}) \}$ 
2:  $SB \leftarrow \{ SB \mid (SB \in \mathcal{B} \in B) \wedge (\text{Timestamp}(SB) = \langle \text{itvl}, \cdot \rangle) \wedge (I_{\text{sync}}(\text{Timestamp}(SB)) = \text{true}) \}$ 
    ▷ Find representative beacon and compute recommendation.
3: for each  $SB = \langle \langle \text{itvl}', r' \rangle, P', \text{ctr}, \eta_{\text{SB}}, \text{blockLabel} \rangle \in SB$  do
4:   Find unique  $SB' = \langle \langle \text{itvl}', r' \rangle, P', \cdot, \cdot, \cdot \rangle \in \text{syncBuffer}$ . If inexistent, set  $SB' \leftarrow \perp$ .
5:   if  $SB' \neq \perp$  then
6:     Set  $\text{arrivalTime}_{SB}(SB) \leftarrow \text{arrivalTime}_{SB}(SB')$ 
7:      $\text{recom}(SB) \leftarrow \text{Timestamp}(SB) - \text{arrivalTime}(SB)$ 
8:   else
9:      $\mathcal{S} \leftarrow \mathcal{S} \setminus \{SB\}$ 
10:   end if
11: end for
    ◁ Negligible probability event in execution.

```

```

12:  $\text{shift}_i \leftarrow \text{med}\{\text{recom}(\text{SB}) \mid \text{SB} \in \text{SB}\}$ 
13: for each SB with  $\text{arrivalTime}_{\text{SB}}(\text{SB}) = (a, \text{temp})$  do
14:    $\text{arrivalTime}_{\text{SB}}(\text{SB}) \leftarrow (a + \text{shift}_i, \text{final})$ 
15: end for
16: if  $\text{shift}_i = 0$  then  $\text{itvl} \leftarrow \text{itvl} + 1$   $\triangleleft$  No adjustment, simply enter next epoch
17: if  $\text{shift}_i > 0$  then  $\triangleleft$  Move fast forward
18:    $\text{newTime} \leftarrow \langle \text{itvl} + 1, r + \text{shift}_i \rangle, \text{localTime} \leftarrow \langle \text{itvl} + 1, r \rangle, \mathcal{M}_{\text{chains}} \leftarrow \emptyset$ 
19:   while  $\text{localTime} < \text{newTime}$  do
20:      $\text{localTime} \leftarrow \text{localTime} + 1$   $\triangleleft$  increment round counter in localtime by 1
21:     Let  $\mathcal{N}_0$  be the subsequence of  $\text{futureChains}$  s.t.  $\mathcal{C} \in \mathcal{N}_0 : \Leftrightarrow \forall \mathcal{B} \in \mathcal{C} : \text{Timestamp}(\mathcal{B}) < \text{localTime}$ 
22:     Remove each  $\mathcal{C} \in \mathcal{N}_0$  from  $\text{futureChains}$ 
23:     Call  $\text{SelectChain}(\text{P}, \text{sid}, \mathcal{C}_{\text{loc}}, \mathcal{N}_0, \text{R})$  to update  $\mathcal{C}_{\text{loc}}$ 
24:   end while
25:   Send  $(\text{DIFFUSE}, \text{sid}, \mathcal{M}_{\text{chains}})$  to  $\mathcal{F}_{\text{Diffuse}}^{\text{bc}}$  and proceed from here upon next activation of this procedure.
26: end if
27: if  $\text{shift}_i < 0$  then Set  $\text{localTime} \leftarrow (\text{itvl} + 1, r + \text{shift}_i)$   $\triangleleft$  Set clock back
OUTPUT: The protocol outputs ok to its caller (but not to  $\mathcal{Z}$ ).

```

Protocol 12: *Parties update their local clocks.*

Reading the Ledger State In order to read the ledger state, the party P first processes all relevant information and then extracts the state (the settled ledger).

Protocol ReadState(P, sid)

```

1: if  $\text{isInit} = \text{false} \vee \text{isSync} = \text{false}$  then
2:   Output the empty state  $(\text{READ}, \text{sid}, \varepsilon)$  (to  $\mathcal{Z}$ ).
3: else
4:   Call  $\text{FetchInformation}(\text{P}, \text{sid})$  and denote the output by  $(\mathcal{C}_1, \dots, \mathcal{C}_N), (\text{tx}_1, \dots, \text{tx}_k)$ 
5:   Set  $\text{buffer} \leftarrow \text{buffer} \parallel (\text{tx}_1, \dots, \text{tx}_k)$  and define  $\mathcal{N} \leftarrow \{\mathcal{C}_1, \dots, \mathcal{C}_N\}$ 
6:   Call  $\text{UpdateTime}(\text{P}, \text{R})$ 
7:   Call  $\text{ProcessBeacons}(\text{P}, \text{sid})$ 
8:   Let  $\mathcal{N}_0 := \{\mathcal{C} \in \mathcal{N} \cup \text{futureChains} \mid \forall \mathcal{B} \in \mathcal{C} : \text{Timestamp}(\mathcal{B}) \leq \text{localTime}\}$ 
9:   Let  $\mathcal{N}_1 := \{\mathcal{C} \in \mathcal{N} \mid \exists \mathcal{B} \in \mathcal{C} : \text{Timestamp}(\mathcal{B}) > \text{localTime}\}$ 
10:   $\text{futureChains} \leftarrow (\text{futureChains} \setminus \mathcal{N}_0) \cup \mathcal{N}_1$ 
11:   $\text{fetchCompleted} \leftarrow \text{true}$ 
12:  Call  $\text{SelectChain}(\text{P}, \text{sid}, \mathcal{C}_{\text{loc}}, \mathcal{N}_0, \text{R})$  to update  $\mathcal{C}_{\text{loc}}$ 
13:  Extract the state  $\text{st}$  from the current local chain  $\mathcal{C}_{\text{loc}}$ 
14:  Output  $(\text{READ}, \text{sid}, \text{st}^{[k]})$  (to  $\mathcal{Z}$ )  $\triangleleft$   $\text{st}^{[k]}$  denotes the prefix of  $\text{st}$  by pruning blocks with timestamps reporting the last  $k$  rounds
15: end if

```

Protocol 13: *Parties read the ledger state.*

Simulate Clock Adjustments If parties are merely de-registered from the random oracle $\mathcal{F}_{\text{RO}}$ (namely, stalled for a limited time), they can bootstrap easily to the reliable state and time. Note

that if parties are stalled, their `localTime` is still updated in `UpdateTime` and by computing the distance of round numbers in `lastTimeAlert` and `localTime` we get the exact number of rounds that have elapsed during the stall period.

Protocol SimulateClockAdjustment(P, sid)

```

1:  $\text{simulatedTime} \leftarrow \text{lastTimeAlert}$ 
2: Set  $r'$  as round index in  $\text{simulatedTime}$ 
3: for  $r - r'$  iterations do
4:   Let  $\mathcal{N}_0$  be the subsequence of  $\text{futureChains}$  s.t.  $\mathcal{C} \in \mathcal{N}_0 :\Leftrightarrow \forall \mathcal{B} \in \mathcal{C} : \text{Timestamp}(\mathcal{B}) \leq \text{simulatedTime}$ 
5:   Remove each  $\mathcal{C} \in \mathcal{N}_0$  from  $\text{futureChains}$ .
6:   Emulate  $\text{SelectChain}(P, \text{sid}, \mathcal{C}_{\text{loc}}, \mathcal{N}_0, R)$  with simulated time  $\text{simulatedTime}$  (instead of  $\text{localTime}$ ) to update  $\mathcal{C}_{\text{loc}}$ 
7:   if  $\text{simulatedTime} = \langle \text{itv1}, \text{itv1} \cdot R \rangle$  for interval  $\text{itv1}$  then
8:     Emulate  $\text{SyncProcedure}(P, \text{sid}, R)$  on simulated time  $\text{simulatedTime}$  (instead of  $\text{localTime}$ )
9:   end if
10: end for
11: Set  $\text{localTime} \leftarrow \text{simulatedTime}$ 
12: Set  $t_{\text{work}} \leftarrow \text{localTime} - 1$ 
OUTPUT: The protocol outputs ok to its caller (but not to  $\mathcal{Z}$ ).

```

Protocol 14: *Stalled parties simulate clock adjustments.*

Round finish procedure. Once a party P has done its actions in a round, P claims finishing current round by calling `FinishRound` and sending `CLOCK-UPDATE` to $\mathcal{G}_{\text{ImpClock}}$. For details of the way to interact with $\mathcal{G}_{\text{ImpClock}}$ in a UC treatment, see [BGK⁺21].

Protocol FinishRound(P, sid)

```

1: while A (CLOCK-UPDATE,  $\mathcal{Z}$ ) has not been received during the current round do
2:   Give up activation (set the anchor here)
3: end while
4: Send (CLOCK-UPDATE,  $\text{sid}_C$ ) to  $\mathcal{G}_{\text{ImpClock}}$ .

```

◁ Party will lose its activation here.

Protocol 15: *Finishing a round.*

The Joining Procedure Another main procedure of Timekeeper is `JoinProc`, where newly joint parties synchronize with other alert parties by passively listening to the protocol execution, building blockchain and processing beacons to derive a local time that is close to all alert parties. The default value of the parameters in each phase are summarized in Table 2.

Protocol JoinProc(P, sid)

```

▷ Phase A, state-reset
1: Call  $\text{UpdateTime}(P, R)$ 
2: if  $\text{localTime} > \langle 1, 1 \rangle$  then

```

◁ Align with newest round.

```

3:   Set  $\text{localTime} \leftarrow \langle 1, 1 \rangle$ 
4:    $\text{fetchCompleted} \leftarrow \text{false}$ ,  $\text{futureChains} \leftarrow \emptyset$ ,  $\text{buffer} \leftarrow \emptyset$ 
5:   Set beacon arrival timetable as empty array
6: end if
    $\triangleright$  Phase B, chain-convergence
7: while  $\text{localTime} < \langle 1, 1 \rangle + t_{\text{off}}$  do
8:   if  $\text{fetchCompleted} = \text{false}$  then
9:     Call  $\text{FetchInformation}(P, \text{sid})$  and denote fetched chains by  $\mathcal{N} := (\mathcal{C}_1, \dots, \mathcal{C}_N)$ 
10:    Call  $\text{SelectChain}(P, \text{sid}, \mathcal{C}_{\text{loc}}, \mathcal{N}, R)$  to update  $\mathcal{C}_{\text{loc}}$ 
11:     $\text{fetchCompleted} \leftarrow \text{true}$ 
12:     $\text{FinishRound}(P)$ 
13:   end if
14:   Call  $\text{UpdateTime}(P, R)$  to update  $\text{localTime}$ 
15: end while
    $\triangleright$  Phase C, beacon-gathering
16: while  $\text{localTime} \leq \langle 1, 1 \rangle + t_{\text{off}} + t_{\text{gather}}$  do
17:   if  $\text{fetchCompleted} = \text{false}$  then
18:     Call  $\text{FetchInformation}(P, \text{sid})$  and denote the output by  $(\mathcal{C}_1, \dots, \mathcal{C}_N), (\text{tx}_1, \dots, \text{tx}_k)$ 
19:     Set  $\text{buffer} \leftarrow \text{buffer} \parallel (\text{tx}_1, \dots, \text{tx}_k)$ 
20:     Set  $\text{futureChains} \leftarrow \text{futureChains} \parallel (\mathcal{C}_1, \dots, \mathcal{C}_N)$ 
21:     Call  $\text{ProcessBeacons}$  to collect new beacons in this round.  $\triangleleft$  All arrival times are
        temporary
22:     Call  $\text{SelectChain}(P, \text{sid}, \mathcal{C}_{\text{loc}}, \text{futureChains}, R)$  to update  $\mathcal{C}_{\text{loc}}$ 
23:      $\text{fetchCompleted} \leftarrow \text{true}$ 
24:      $\text{FinishRound}(P)$ 
25:   end if
26:   Call  $\text{UpdateTime}(P, R)$  to update  $\text{localTime}$ 
27: end while
    $\triangleright$  Phase D, shift-computation
28:  $\text{syncBuffer}_{\text{valid}} \leftarrow \{\text{SB}' \in \text{syncBuffer} \mid \text{ValidSB}(P, \text{sid}, \text{SB}', \mathcal{C}_{\text{loc}}, R) = \text{true}\}$ 
29: Initialize  $i := 0$ .
30: Set  $i$  to be the minimum positive integer such that  $\forall \text{SB} \in \mathcal{B} \in B : \text{SB} \in \text{syncBuffer}_{\text{valid}} \wedge$ 
     $\text{arrivalTime}(\text{SB}) \geq \langle 1, 1 \rangle + t_{\text{off}} + t_{\text{pre}}$  where  $B \leftarrow \{\mathcal{B} \mid (\mathcal{B} \in \mathcal{C}_{\text{loc}}) \wedge (\text{Timestamp}(\mathcal{B}) = \langle i, \cdot \rangle) \wedge$ 
     $(I_{\text{sync}}(\text{Timestamp}(\mathcal{B})) = \text{true})\}$ .  $\triangleleft$  if no interval exists,  $i$  is unchanged.
31: if  $i \geq 1$  then
32:   for at most  $(t_{\text{gather}} \text{ div } R)$  iterations do
33:      $B \leftarrow \{\mathcal{B} \mid (\mathcal{B} \in \mathcal{C}_{\text{loc}}) \wedge (\text{Timestamp}(\mathcal{B}) = \langle i, \cdot \rangle) \wedge (I_{\text{sync}}(\text{Timestamp}(\mathcal{B})) = \text{true})\}$ 
34:      $SB \leftarrow \{\text{SB} \mid \text{SB} \in \mathcal{B} \in B \wedge (\text{Timestamp}(\text{SB}) = \langle i, \cdot \rangle) \wedge (I_{\text{sync}}(\text{Timestamp}(\text{SB})) = \text{true})\}$ 
35:     for each  $\text{SB} = \langle \langle \text{itvl}', \text{r}' \rangle, P', \text{ctr}, \eta_{\text{SB}}, \text{blockLabel} \rangle \in SB$  do
36:        $Q_{\text{SB}} \leftarrow \{\text{SB}' = \langle \langle \text{itvl}'', \text{r}'' \rangle, P'', \cdot, \cdot, \cdot \rangle \in \text{syncBuffer}_{\text{valid}} \mid P' = P'' \wedge \langle \text{itvl}', \text{r}' \rangle =$ 
         $\langle \text{itvl}'', \text{r}'' \rangle\}$ 
37:       if  $Q_{\text{SB}} \neq \emptyset$  then
38:          $\min_{\text{SB}} \leftarrow \min\{\text{arrivalTime}(\text{SB}) \mid \text{SB} \in Q_{\text{SB}}\}$ 
39:          $\text{arrivalTime}_{\text{SB}}(\text{SB}) \leftarrow (\min_{\text{SB}}, \text{final})$ 
40:          $\text{recom}(\text{SB}) \leftarrow \text{Timestamp}(\text{SB}) - \text{arrivalTime}(\text{SB})$ 
41:       else
42:          $\mathcal{S} \leftarrow \mathcal{S} \setminus \{\text{SB}\}$   $\triangleleft$  Negligible probability event in execution
43:       end if
44:     end for
45:      $\text{shift}_i \leftarrow \text{med}\{\text{recom}(\text{SB}) \mid \text{SB} \in SB\}$ 

```

```

46:   for each SB with  $\text{arrivalTime}_{SB}(\text{SB}) = (a, \text{temp})$  do
47:      $\text{arrivalTime}_{SB}(\text{SB}) \leftarrow (a + \text{shift}_i, \text{temp})$ 
48:   end for
49:   if  $r + \text{shift}_i \leq (i + 1)R$  then
50:     Set  $\text{localTime} \leftarrow \langle i + 1, r + \text{shift}_i \rangle$ 
51:     Break
52:   else
53:     Set  $\text{localTime} \leftarrow \langle i + 1, r + \text{shift}_i \rangle$        $\triangleleft$  Temporarily invalid; will be adjust later.
54:     set  $i \leftarrow i + 1$                                  $\triangleleft$  continue iteration.
55:   end if
56: end for
57:  $\text{isSync} \leftarrow \text{true}$ ; run  $\text{SelectChain}(P, \text{sid}, \mathcal{C}_{\text{loc}}, \text{futureChains}, R)$  to update  $\mathcal{C}_{\text{loc}}$ ;  $t_{\text{work}} \leftarrow \langle \text{itvl}, r - 1 \rangle$ 
58:   for each beacon  $\text{SB} \in \text{syncBuffer}_{\text{valid}}$  with  $\text{Timestamp}(\text{SB}) = \langle \text{itvl}, \cdot \rangle$  do
59:     Parse  $\text{arrivalTime}_{SB}(\text{SB})$  as  $(a, \text{temp})$ . Define  $\text{arrivalTime}_{SB}(\text{SB}) \leftarrow (a, \text{final})$ 
60:   end for
61: end if

```

Protocol 16: *The procedure for a fresh new party to join Timekeeper.*

D Proofs Omitted from the Main Body

Proof of Lemma 1. If two blocks are obtained at nominal rounds which are at distance at least $\Delta + \Phi$, then we are certain that the later block increased the accumulated difficulty. To be precise, assume $S^* \subseteq S$ is such that, for all $i, j \in S^*$, $|i - j| \geq \Delta + \Phi$ and $Y_i > 0$. We argue that, by round v , every honest party has a chain of difficulty at least

$$d + \sum_{r \in S^*} Y_r \geq d + Q(S).$$

Observe first that every honest party will receive the chain of difficulty d by round $u + \Delta + \Phi$ and so the first block obtained in S^* extends a chain of weight at least d . Next, note that if a block obtained in S^* is the head of a chain of weight at least d' , then the next block in S^* extends a chain of weight at least d' . $\square$

Proof of Lemma 2. For the proof of Lemma 2(a)(b)(c), we refer to [GKL20]. Regarding Lemma 2(d), note that we are under the same condition as that in Lemma 2(c), we have

$$A(J') < A(J) < (1 - \delta + 3\epsilon)Q(S) \leq (1 - \delta + 3\epsilon)(1 + \epsilon)ph(S) < (1 - \delta + 3\epsilon)(1 + \epsilon)(1 - \delta)p|J'| < (1 + \epsilon)p|J'|.$$

The second inequality follows Lemma 2(c); the third one comes from Definition 7(a); the last inequality is a consequence of Condition (C2).

By combining the upper bound on $A(J')$ with $D(S) < (1 + \epsilon)ph(S)$ (which comes from the definitions) we get the desired inequality. $\square$

Proof of Theorem 3. Definition 7 extends the definition of typical executions in [GKL20] by adding lower bound on $D(S)$ in item (a). Hence regarding the proof of the rest part in Definition 7, we refer to [GKL20]. In this proof we only consider the probability of violating the lower bound on $D(S)$ and the eventual error probability.

Fix a set of $R \geq \ell$ consecutive rounds $S = s_1, s_2, \dots, s_R$ and let $h_j, j \in [R]$ denote the number of honest queries made during round s_j . We work on per query that alert parties make during S . Let J denote the queries in S , $\nu = |J| = h(S)$, and Z_i the difficulty of any block obtained from query i . Consider the sequence of random variables

$$X_0 = 0; X_k = \sum_{i \in [k]} Z_i - \sum_{i \in [k]} \mathbb{E}[Z_i | \mathcal{E}_{i-1}], k \in [\nu].$$

This is a martingale sequence with respect to sequence $(\mathcal{E}_0, \mathcal{E}_1, \dots, \mathcal{E}_\nu)$ in that

$$\mathbb{E}[X_k | \mathcal{E}_{k-1}] = \mathbb{E}[Z_k - \mathbb{E}[Z_k | \mathcal{E}_{k-1}] | \mathcal{E}_{k-1}] + \mathbb{E}[X_{k-1} | \mathcal{E}_{k-1}] = X_{k-1}.$$

The last equation follows the linearity of conditional expectation and the fact that X_{k-1} is a deterministic function with respect to $\mathcal{E}_{k-1}$.

Regarding the details relevant to Theorem 19, we pick t as

$$\epsilon \sum_{i \in [\nu]} \mathbb{E}[Z_i | \mathcal{E}_{i-1} = E_{i-1}] \leq \epsilon \sum_{j \in [R]} ph_j = \epsilon ph(S) \stackrel{\text{def}}{=} t$$

and consider an execution satisfying G_t . Fix $i \in [\nu]$, let j be the round s_j that query i belongs to. Let

$$Z_i - \mathbb{E}[Z_i | \mathcal{E}_{i-1} = E_{i-1}] \leq \frac{1}{T_j^{\min}} = \frac{ph_j}{ph_j T_j^{\min}} \leq \frac{\gamma ph(S)}{ph_j T_j^{\min} R} \leq \frac{\gamma ph(S)}{fR/(2\gamma^2)} = \frac{2\gamma^3 t}{\epsilon f R} \stackrel{\text{def}}{=} b$$

and we see that the event G implies $X_k - X_{k-1} \leq b$. To get the bound on V , note that

$$\mathbf{Var}(X_k - X_{k-1} | \mathcal{E}_{k-1}) = \mathbb{E}[(Z_i - \mathbb{E}[Z_i | \mathcal{E}_{i-1}])^2 | \mathcal{E}_{k-1}] = \mathbb{E}[Z_i^2 | \mathcal{E}_{k-1}] - (\mathbb{E}[Z_k | \mathcal{E}_{k-1}])^2 \leq \mathbb{E}[Z_i^2 | \mathcal{E}_{k-1}].$$

Hence, based on the independence of random variables as well as Fact 1(c), we pick v as

$$\begin{aligned} \sum_{k \in [\nu]} \mathbb{E}[Z_i^2 | \mathcal{E}_{k-1} = E_{k-1}] &\leq \sum_{j \in [R]} \sum_{i \in [h_j]} \frac{1}{T_i^2} \cdot pT_i = \sum_{j \in [R]} \frac{ph_j}{T_j^{\min}} = \sum_{j \in [R]} \frac{(ph_j)^2}{ph_j T_j^{\min}} \\ &\leq \frac{2\gamma^2}{f} \sum_{j \in [R]} (ph_j)^2 \leq \frac{2\gamma^3}{fR} \cdot (ph(S))^2 \leq \frac{2\gamma^3 t^2}{\epsilon^2 f R} \stackrel{\text{def}}{=} v. \end{aligned}$$

In view of these bounds (note that $bt = \epsilon v$), by Theorem 19 and Condition (C1), we have

$$\Pr[-X_\nu \geq t \wedge G_t] \leq \exp \left\{ -\frac{t^2}{2v(1 + \frac{\epsilon}{3})} \right\} \leq \exp \left\{ -\frac{\epsilon^2 f R}{4\gamma^3(1 + \frac{\epsilon}{3})} \right\} \leq e^{-\lambda}. \quad (10)$$

Combining Equation (10) and those error probabilities in [GKL20], we get asymptotically the same result. I.e., the probability that “ $\mathcal{E}$ is not typical” is bounded by $O(L^2)(e^{-\lambda} + 2^{-\kappa})$. $\square$

Proof of Lemma 5. Suppose $\text{head}(\mathcal{C} \cap \mathcal{C}')$ was created in round v and let $u \leq v$ be the greatest round in which an honest party computed a block on $\mathcal{C} \cap \mathcal{C}'$. Let $U = \{i : u < i \leq r\}$, $S = \{i : u + \Delta + \Phi \leq i \leq r - (\Delta + \Phi)\}$, and let J denote the adversarial queries that correspond to the rounds in U . Consider the following claim.

Below we say that $d \in \mathbb{R}$ is *contained* in a block $\mathcal{B}$ (and write $d \in \mathcal{B}$), when $\mathcal{B}$ extends a chain $\mathcal{C}$ and $\text{diff}(\mathcal{C}) < d \leq \text{diff}(\mathcal{CB})$.

Claim. *If $r - v \geq \ell + 2(\Delta + \Phi)$, then $2Q(S) \leq D(U) + A(J)$.*

Proof. Associate with each $r \in S$ such that $Q_r > 0$ an arbitrary honest block that is computed at round r for difficulty Q_r . Let B be the set of these blocks and note that their difficulties sum to $Q(S)$. We argue the existence of a set of blocks B' computed in U such that $B \cap B' = \emptyset$ and $\{d \in \mathcal{B} : \mathcal{B} \in B\} \subseteq \{d \in \mathcal{B} : \mathcal{B} \in B'\}$. This suffices, because each block in B' contributes either to $D(U) - Q(S)$ or to $A(J)$ and so $Q(S) \leq D(U) - Q(S) + A(J)$.

Consider, then, a block $\mathcal{B} \in B$ extending a chain $\mathcal{C}^*$ and let $d = \text{diff}(\mathcal{C}^* \mathcal{B})$. If $d \leq \text{diff}(\mathcal{C} \cap \mathcal{C}')$ (note that $u < v$ in this case and $\text{head}(\mathcal{C} \cap \mathcal{C}')$ is adversarial), let $\mathcal{B}'$ be the block of $\mathcal{C} \cap \mathcal{C}'$ containing d . Such a block clearly exists and was computed after u . Furthermore, $\mathcal{B}' \notin B$, since $\mathcal{B}'$ was computed by the adversary. If $d > \text{diff}(\mathcal{C} \cap \mathcal{C}')$, note that there is a unique $\mathcal{B} \in B$ such that $d \in \mathcal{B}$ (recall the argument in Chain Growth Lemma 1). Since $\mathcal{B}$ cannot simultaneously be on $\mathcal{C}$ and $\mathcal{C}'$, there is a $\mathcal{B}' \notin B$ either on $\mathcal{C}$ or on $\mathcal{C}'$ that contains d . $\square$

Since $S \geq \ell$ and Lemma 4 implies that neither $\mathcal{C}$ nor $\mathcal{C}'$ are stale, from Lemma 2(a)(c) we get that $D(U) < (1 + 5\epsilon)Q(S)$ and $A(J) < (1 - \delta + 3\epsilon)Q(S)$, which implies $D(U) + A(J) < 2Q(S)$ when $\delta \geq 8\epsilon$ (by Condition (C2)). This contradicts the claim above. Hence, $r - v < \ell + 2(\Delta + \Phi)$. $\square$

E Glossary

Main parameters of Timekeeper. In Table 3, we list and explain all Timekeeper parameters.

Parameter	Description
h_r	The number of honest RO queries in (nominal) round r .
t_r	The number of RO queries by $\mathcal{A}$ in (nominal) round r .
δ	Advantage of honest parties ($t_r < (1 - \delta)h_r$ for all r).
f	The probability at least one honest RO query out of n_0 computes a block for target T_0 .
R	The length of a clock synchronization interval in number of rounds.
M	The length of a target recalculation epoch in number of rounds.
K	The length of convergence phase in a clock synchronization interval in number of rounds.
Δ	Network delay in rounds.
Φ_{clock}	The upper bound of the drift that $\mathcal{A}$ can set.
Φ	The upper bound of the difference between honest parties' local clocks. We require that $\Phi = \Phi_{\text{clock}} + \Delta$.
κ	Security parameter; length of the hash function output.
(γ, s)	Respecting environment parameter.
ϵ	Quality of concentration of random variables.
λ	Security parameter; related to the properties of the protocol.

Table 3: *Main Parameters of Timekeeper.*

Main variables of Timekeeper participants. We summarize the internal state variables of a protocol participant as follows.

Variable	Description
<code>localTime</code>	The party P 's current timestamp in the form of $\langle \text{itvl}, \mathbf{r} \rangle$.
<code>itvl</code>	The interval that $\mathbf{r}$ belongs to.
<code>ep</code>	The epoch that $\mathbf{r}$ belongs to.
$\mathcal{C}_{\text{loc}}$	The local chain the party adopts based on which it does mining and exports the ledger state.
<code>buffer</code>	A buffer of transactions.
<code>futureChains</code>	A buffer to store chains with blocks whose timestamps belong to the future (logical) rounds.
$\text{arrivalTime}_{SB}(\cdot)$	A map that assigns to each synchronization beacon a pair (a, b) , where a is the arrival time and b is an indication of whether a is final or temp .
$\text{arrivalTime}(\cdot)$	Shorthand for the first (arrival time) element of the pair $\text{arrivalTime}_{SB}(\cdot)$.
<code>isSync</code>	A bit variable to store the synchronization status.
<code>fetchCompleted</code>	A variable to store whether the round messages have been fetched.
<code>lastTimeAlert</code>	The timestamp which stores the last alert time. Used for rejoining if the party was only stalled.

Table 4: *Main state variables in Timekeeper.*